

数据结构

第六章 树和二叉树

树的定义

二叉树

树的存储结构

树和二叉树的遍历

哈夫曼树

树和图

树型结构是一类非常重要的非线性结构。直观地，树型结构是以分支关系定义的层次结构。

树在计算机领域中也有着广泛的应用，例如在编译程序中，用树来表示源程序的语法结构；在数据库系统中，可用树来组织信息；在分析算法的行为时，可用树来描述其执行过程等等。

本章将详细讨论树和二叉树数据结构，主要介绍树和二叉树的概念、术语，二叉树的遍历算法。树和二叉树的各种存储结构以及建立在各种存储结构上的操作及应用等。

第一节 树的定义

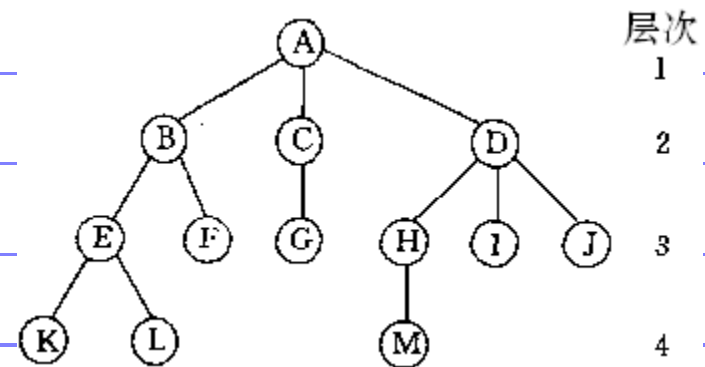
□ 定义：树(tree)是 $n(n>0)$ 个结点的有限集 T ，其中：

- 有且仅有一个特定的结点，称为树的根(root)
- 当 $n>1$ 时，其余结点可分为 $m(m>0)$ 个互不相交的有限集 $T_1, T_2, \dots, T_m$ ，其中每一个集合本身又是一棵树，称为根的子树(subtree)

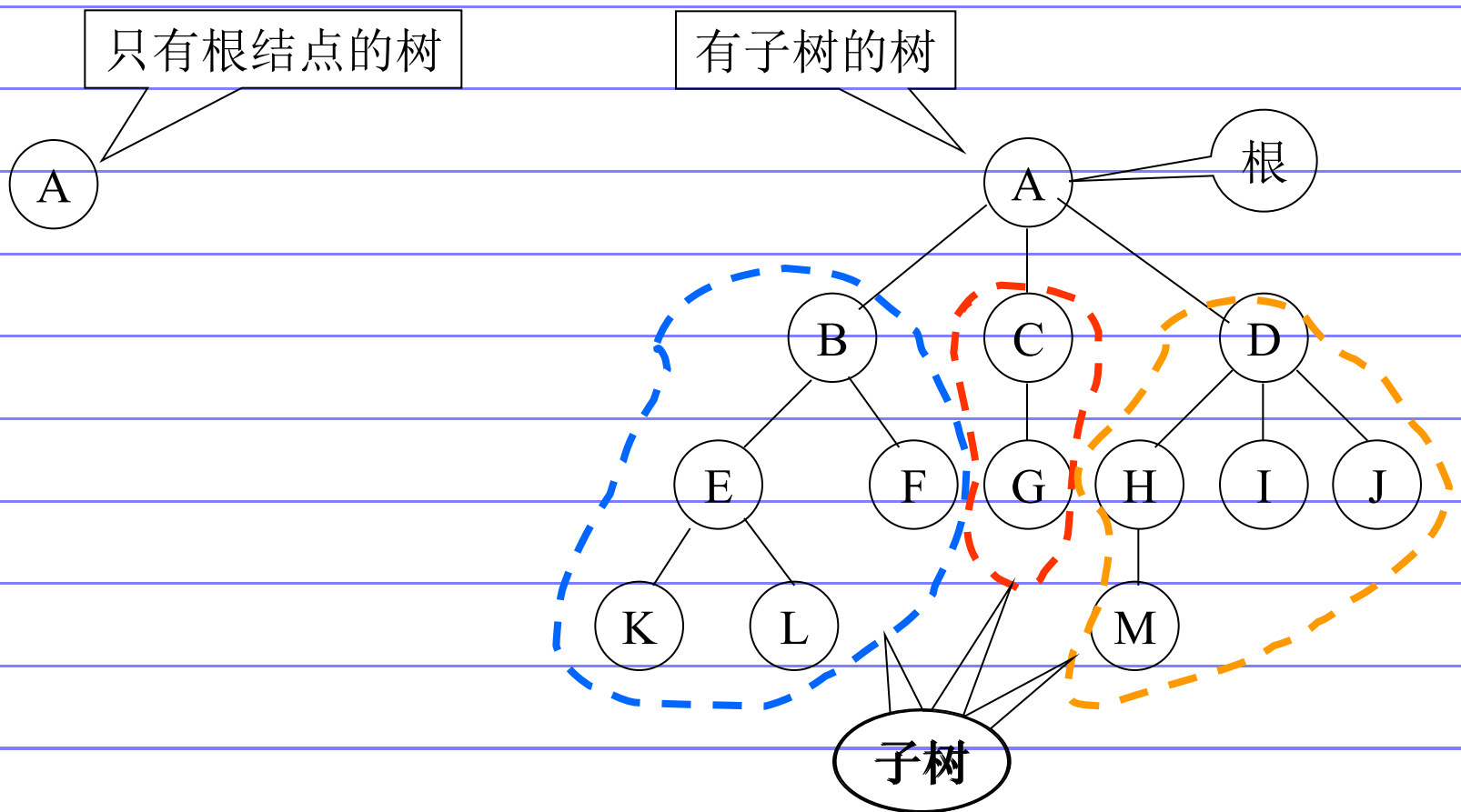
这是树的递归定义，即用树来定义树，而只有一个结点的树必定仅由根组成。

□ 特点：

- 树中至少有一个结点——根
- 树中各子树是互不相交的集合



树的例子



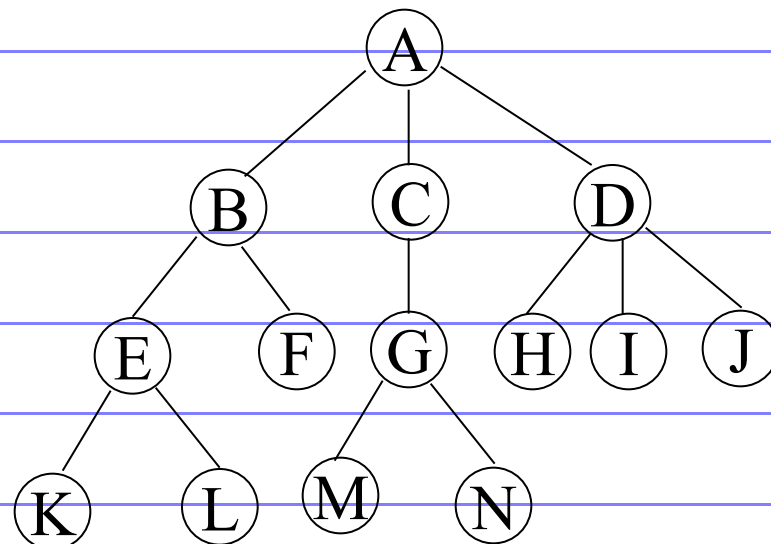
树的基本术语

- (1) **结点(node)**: 一个数据元素及其若干指向其子树的分支。
- (2) **结点的度(degree)**、**树的度**: 结点所拥有的子树的棵数称为结点的度。树中结点度的最大值称为树的度。

如图6-1(b)中结点A的度是3，结点B的度是2，结点M的度是0，树的度是3。



(a) 只有根结点



(b) 一般的树

图6-1 树的示例形式

树的基本术语

(3) **叶子(leef)结点、非叶子结点**：树中度为**0**的结点称为**叶子结点**（或**终端结点**）。相对应地，**度不为0**的结点称为**非叶子结点**（或**非终端结点**或**分支结点**）。除根结点外，分支结点又称为**内部结点**。

如图6-1(b)中结点**H、I、J、K、L、M、N**是叶子结点，而所有其它结点都是分支结点。

(4) **孩子结点、双亲结点、兄弟结点**

一个结点的**子树的根**称为该结点的**孩子结点(child)**或**子结点**；相应地，该结点是其孩子结点的**双亲结点(parent)**或**父结点**。

如图6-1(b)中结点**B、C、D**是结点**A**的子结点，而结点**A**是结点**B、C、D**的父结点；类似地结点**E、F**是结点**B**的子结点，结点**B**是结点**E、F**的父结点。

□ 同一双亲结点的所有子结点互称为兄弟结点。

□ 如图6-1(b)中结点**B、C、D**是兄弟结点；结点**E、F**是兄弟结点。

树的基本术语

(5) 层次、堂兄弟结点

规定树中根结点的层次为1，其余结点的层次等于其双亲结点的层次加1。

若某结点在第 l ($l \geq 1$) 层，则其子结点在第 $l+1$ 层。

双亲结点在同一层上的所有结点互称为堂兄弟结点。如图6-1 (b) 中结点E、F、G、H、I、J。

(6) 结点的层次路径、祖先、子孙

从根结点开始，到达某结点 p 所经过的所有结点成为结点 p 的层次路径（有且只有一条）。

结点 p 的层次路径上的所有结点（ p 除外）称为 p 的祖先(ancestor)。

以某一结点为根的子树中的任意结点称为该结点的子孙结点(descendant)。

树的基本术语

- (7) **树的深度(depth)**: 树中结点的最大层次值, 又称为树的高度, 如图6-1(b)中树的高度为4。
- (8) **有序树和无序树**: 对于一棵树, 若其中每一个结点的子树 (若有) 具有一定的次序, 则该树称为**有序树**, 否则称为**无序树**。
- (9) **森林(forest)**: 是 $m(m \geq 0)$ 棵互不相交的树的集合。显然, 若将一棵树的根结点删除, 剩余的子树就构成了森林。

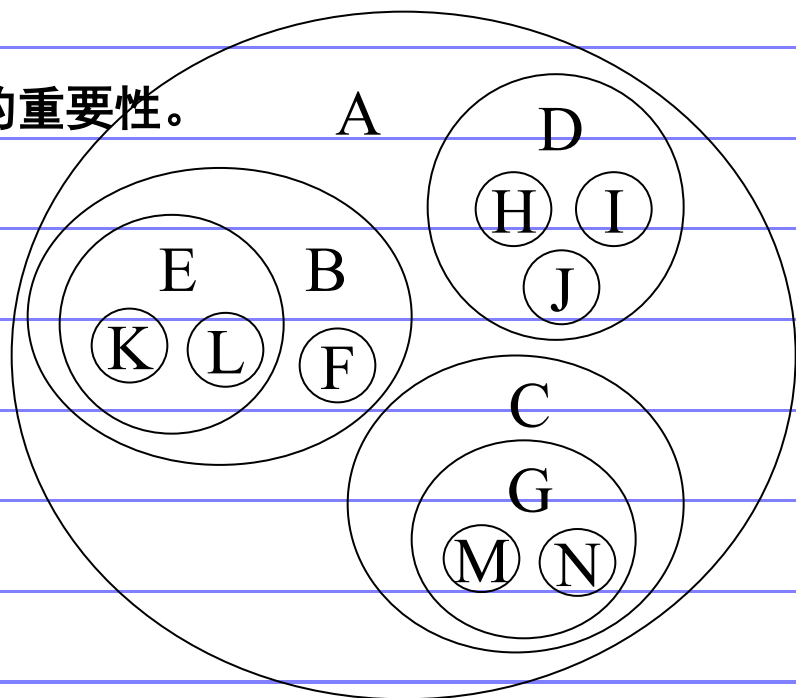
树的表示形式

- (1) **倒悬树**。是最常用的表示形式，如图6-1(b)。
- (2) **嵌套集合**。是一些集合的集体，对于任何两个集合，或者不相交，或者一个集合包含另一个集合。图6-2(a)是图6-1(b)树的嵌套集合形式。
- (3) **广义表形式**。图6-2(b)是树的广义表形式。
- (4) **凹入法表示形式**。见P₁₂₀

□ 树的表示方法的多样化说明了树结构的重要性。

$(A(B(E(K,L),F),C(G(M,N)),D(H,I,J)))$

(b) 广义表形式



(a) 嵌套集合形式

图6-2 树的表示形式

树的抽象数据类型定义

□ 详见p_{118~119}

树是一种数据结构

$$\text{Tree} = (D, R)$$

其中: D 是具有相同特性的数据元素的集合, 若 D 只含一个数据元素, 则 R 为空集, 否则 R 是 D 上某个二元关系 H 的集合, 即 $R = \{H\}$ 。 H 为如下描述的二元关系:

- (1) 在 D 中存在唯一的称为根的数据元素 root , 它在关系 H 下无前驱;
- (2) 若 $D - \{\text{root}\} \neq \Phi$, 则存在 $D - \{\text{root}\}$ 的一个划分 $D_1, D_2, \dots, D_m (m > 0)$, 对任意一对 $j \neq k (1 \leq j, k \leq m)$ 有 $D_j \cap D_k = \Phi$, 且对任意的 $i (1 \leq i \leq m)$, 唯一存在数据元素 $x_i \in D_i$, 有 $\langle \text{root}, x_i \rangle \in H$;
- (3) 对应于 $D - \{\text{root}\}$ 的划分, $H - \{\langle \text{root}, x_1 \rangle, \dots, \langle \text{root}, x_m \rangle\}$ 有唯一的一个划分 $H_1, H_2, \dots, H_m (m > 0)$, 对任意一对 $j \neq k (1 \leq j, k \leq m)$ 有, $H_j \cap H_k = \Phi$, 且对任意的 $i (1 \leq i \leq m)$ H_i 是 D_i 上的二元关系, $(D_i, (H_i))$ 是一棵符合本定义棵树, 称为根 root 的子树。

这是一个递归的定义, 即在树的定义中又用到树的概念, 它道出了树的固有特性。

树的基本操作

InitTree(&T);

操作结果:构造空树 T。

DestroyTree(&T);

初始条件:树 T 存在。

操作结果:销毁树 T。

CreateTree(&T, definition);

初始条件:definition 给出树 T 的定义。

操作结果:按 definition 构造树 T。

ClearTree(&T);

初始条件:树 T 存在。

操作结果:将树 T 清为空树。

TreeEmpty(T);

初始条件:树 T 存在。

操作结果:若 T 为空树,则返回 TRUE, 否则 FALSE。

TreeDepth(T);

初始条件:树 T 存在。

操作结果:返回 T 的深度。

Root(T);

初始条件:树 T 存在。

操作结果:返回 T 的根。

Value(T, cur_e);

初始条件:树 T 存在, cur_e 是 T 中某个结点。

操作结果:返回 cur_e 的值。

Assign(T, cur_e, value);

初始条件:树 T 存在, cur_e 是 T 中某个结点。

操作结果:结点 cur_e 赋值为 value。

Parent(T, cur_e);

初始条件:树 T 存在, cur_e 是 T 中某个结点。

操作结果:若 cur_e 是 T 的非根结点,则返回它的双亲,否则函数值为“空”。

LeftChild(T, cur_e);

初始条件:树 T 存在, cur_e 是 T 中某个结点。

操作结果:若 cur_e 是 T 的非叶子结点,则返回它的最左孩子,否则返回“空”。

RightSibling(T, cur_e);

初始条件:树 T 存在, cur_e 是 T 中某个结点。

操作结果:若 cur_e 有右兄弟,则返回它的右兄弟,否则函数值为“空”。

InsertChild(&T, &p, i, c);

初始条件:树 T 存在, p 指向 T 中某个结点, $1 \leq i \leq p$ 所指结点的度 + 1, 非空树 c 与 T 不相交。

操作结果:插入 c 为 T 中 p 指结点的第 i 棵子树。

DeleteChild(&T, &p, i);

初始条件:树 T 存在, p 指向 T 中某个结点, $1 \leq i \leq p$ 指结点的度。

操作结果:删除 T 中 p 所指结点的第 i 棵子树。

TraverseTree(T, Visit());

初始条件:树 T 存在, Visit 是对结点操作的应用函数。

操作结果:按某种次序对 T 的每个结点调用函数 visit() 一次且至多一次。

一旦 visit() 失败, 则操作失败。

第二节 二叉树

□ 定义：二叉树是 $n(n \geq 0)$ 个结点的有限集，它或为空树($n=0$)，或由一个根结点和两棵分别称为左子树和右子树的互不相交的二叉树构成

(1) 有且只有一个特殊的称为树的根(**Root**)结点；

(2) 若 $n > 1$ 时，其余的结点被分成为二个互不相交的子集 T_1, T_2 ，分别称之为左、右子树，并且左、右子树又都是二叉树。

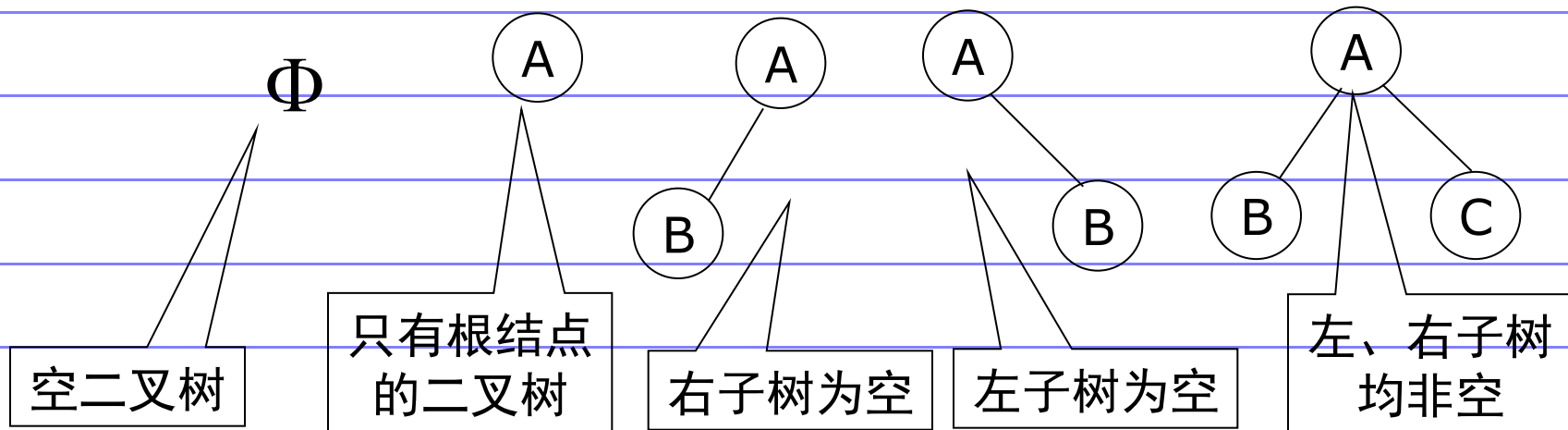
由此可知，二叉树的定义是递归的。

特点

- 每个结点至多有二棵子树(即不存在度大于2的结点)
- 二叉树的子树有左、右之分，且其次序不能任意颠倒

二叉树的基本形态

二叉树在树结构中起着非常重要的作用。因为二叉树结构简单，存储效率高，树的操作算法相对简单，且任何树都很容易转化成二叉树结构。上节中引入的有关树的术语也都适用于二叉树。



□ 有三个节点的树有几种形态？

二叉树性质

□ 性质1：在二叉树的第 i 层上至多有 2^{i-1} 个结点($i \geq 1$)

证明：用归纳法证明之

① $i=1$ 时，只有一个根结点， $2^{i-1} = 2^0 = 1$ 是对的

②假设对所有 $j(1 \leq j < i)$ 命题成立，即第 j 层上至多有 2^{j-1} 个结点

那么，第 $i-1$ 层至多有 2^{i-2} 个结点

又二叉树每个结点的度至多为2

$\therefore$ 第 i 层上最大结点数是第 $i-1$ 层的2倍，即 $2 \cdot 2^{i-2} = 2^{i-1}$

故命题得证

□ 性质2：深度为 k 的二叉树至多有 $2^k - 1$ 个结点($k \geq 1$)

证明：由性质1，可得深度为 k 的二叉树最大结点数是

$$\sum_{i=1}^k (\text{第} i \text{层的最大结点数}) = \sum_{i=1}^k 2^{i-1} = 2^k - 1$$

二叉树性质

□ 性质3：对任何一棵二叉树T，如果其终端结点数为 n_0 ，度为2的结点数为 n_2 ，则 $n_0 = n_2 + 1$

证明： n_1 为二叉树T中度为1的结点数

因为：二叉树中所有结点的度均小于或等于2

所以：其结点总数 $n = n_0 + n_1 + n_2$

又二叉树中，除根结点外，其余结点都
只有一个分支进入

设B为分支总数，则 $n = B + 1$

又：分支由度为1和度为2的结点射出，

$$\therefore B = n_1 + 2n_2$$

于是， $n = B + 1 = n_1 + 2n_2 + 1 = n_0 + n_1 + n_2$

$$\therefore n_0 = n_2 + 1$$

特殊形式的二叉树

□ 满二叉树

- 定义：一深度为 k 且有 2^k-1 个结点的二叉树
- 特点：每一层上的结点数都是最大结点数
- 满二叉树的所有的支结点都有左、右子树。
- 可对满二叉树的结点进行连续编号，若规定从根结点开始，按“自上而下、自左至右”的原则进行。

□ 完全二叉树

- 定义：深度为 k ，有 n 个结点的二叉树当且仅当其每一个结点都与深度为 k 的满二叉树中编号从1至 n 的结点一一对应时，称完全二叉树
- 或深度为 k 的满二叉树中编号从1到 n 的前 n 个结点构成了一棵深度为 k 的完全二叉树。其中 $2^{(k-1)}-1 < n \leq 2^k-1$

特殊形式的二叉树

□ 完全二叉树

完全二叉树是满二叉树的一部分，而满二叉树是完全二叉树的特例。

■ 特点

- ◇ 叶子结点只可能在层次最大的两层上出现（若完全二叉树的深度为 k ，则所有的叶子结点都出现在第 k 层或 $k-1$ 层）
- ◇ 对任一结点，若其右分支下子孙的最大层次为 l ，则其左分支下子孙的最大层次必为 l 或 $l+1$

完全二叉树性质

□ 性质4：具有 n 个结点的完全二叉树的深度为 $\lfloor \log_2 n \rfloor + 1$

其中符号： $\lfloor x \rfloor$ 表示不大于 x 的最大整数。

$\lceil x \rceil$ 表示不小于 x 的最小整数。

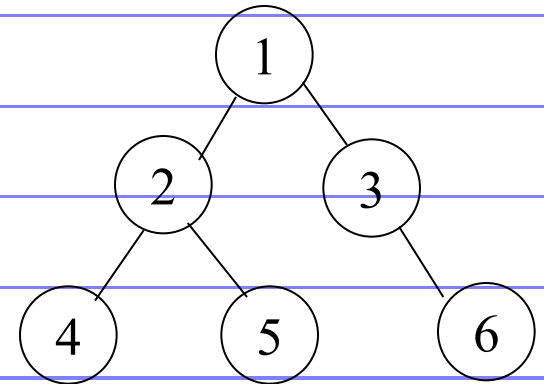
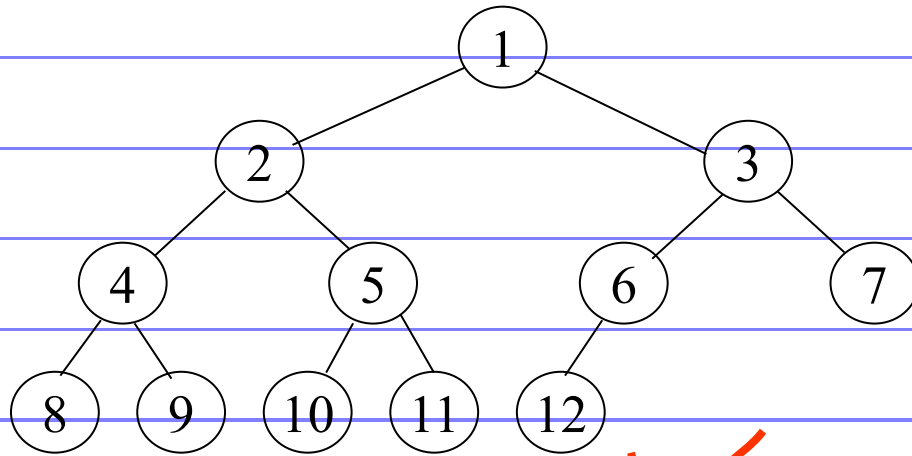
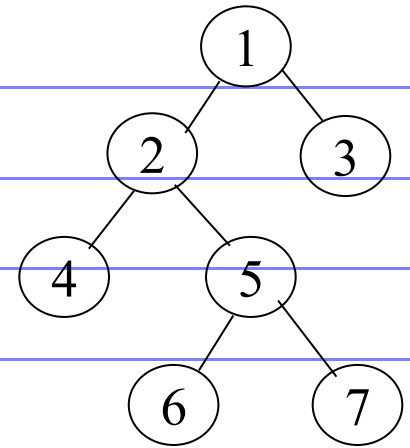
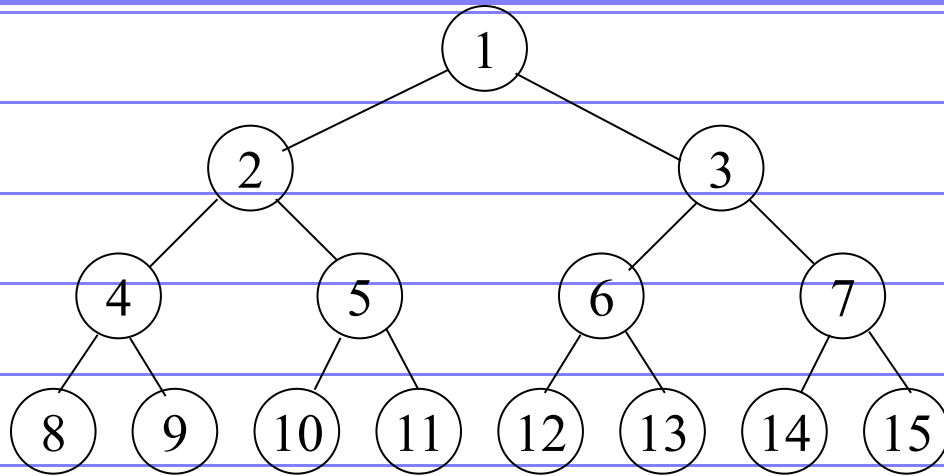
□ 性质5：如果对一棵有 n 个结点的完全二叉树的结点按层序编号，则对任一结点 i ($1 \leq i \leq n$)，有：

(1) 如果 $i=1$ ，则结点 i 是二叉树的根，无双亲；如果 $i>1$ ，则其双亲是 $\lfloor i/2 \rfloor$

(2) 如果 $2i>n$ ，则结点 i 无左孩子；如果 $2i \leq n$ ，则其左孩子是 $2i$

(3) 如果 $2i+1>n$ ，则结点 i 无右孩子；如果 $2i+1 \leq n$ ，则其右孩子是 $2i+1$

二叉树例



二叉树的存储结构

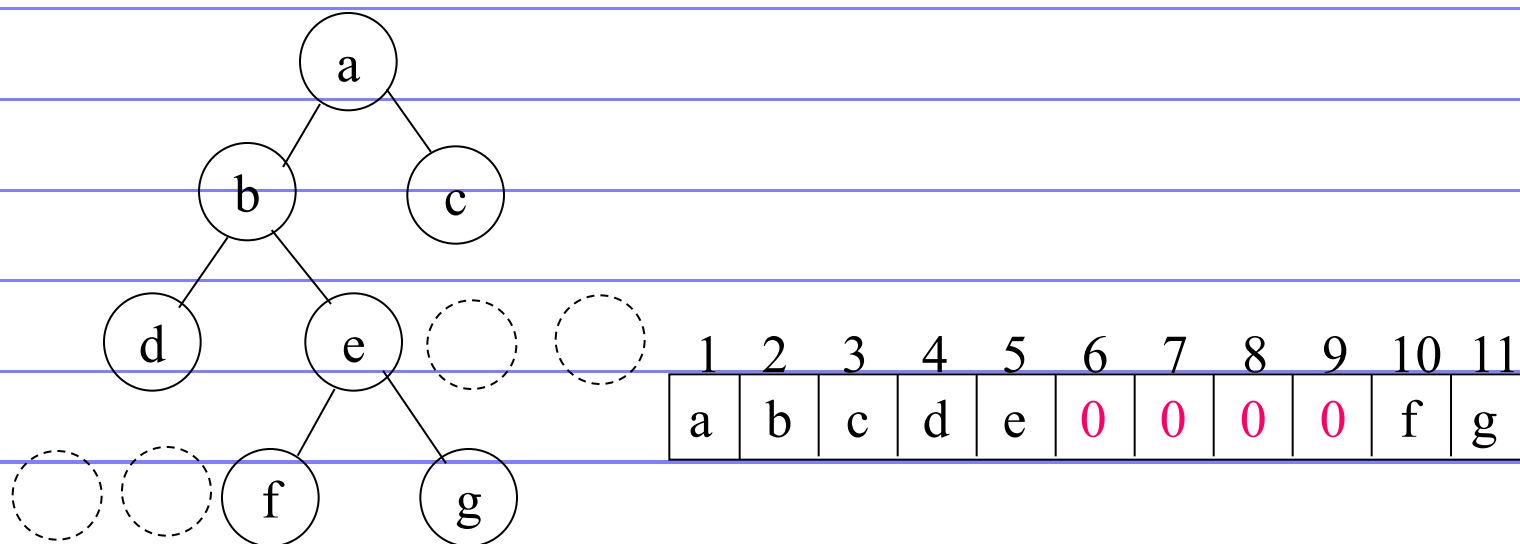
□ 顺序存储结构

■ 实现：按满二叉树的结点层次编号，依次存放二叉树中的数据元素

■ 特点：

◇ 结点间关系蕴含在其存储位置中

◇ 浪费空间，适于存满二叉树和完全二叉树



二叉树的存储结构

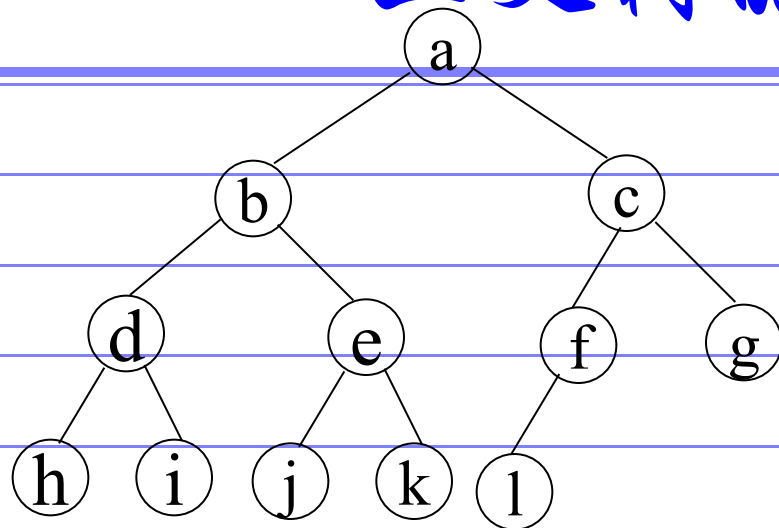
用一组地址连续的存储单元依次“自上而下、自左至右”存储完全二叉树的数据元素。

对于完全二叉树上编号为 i 的结点元素存储在一维数组的下标值为 $i-1$ 的分量中，如图6-6(c)所示。

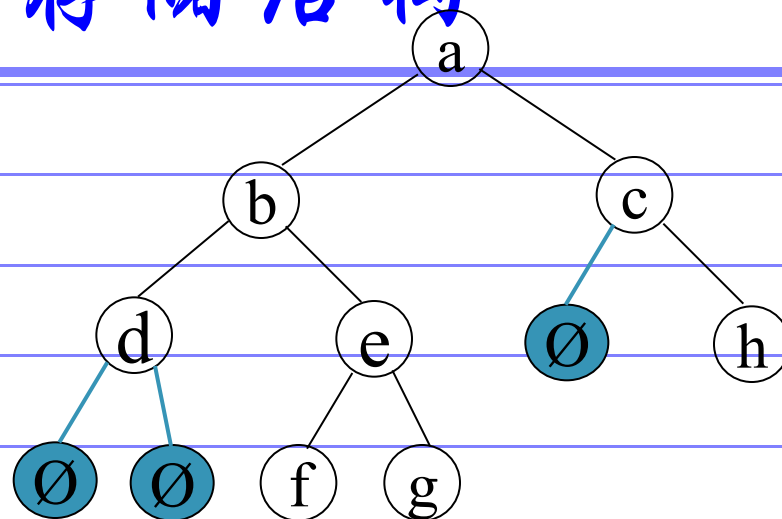
对于一般的二叉树，将其每个结点与完全二叉树上的结点相对照，存储在一维数组中，如图6-6(d)所示。

最坏的情况下，一个深度为 k 且只有 k 个结点的单支树需要长度为 2^k-1 的一维数组。

二叉树的存储结构



(a) 完全二叉树



(b) 非完全二叉树

1	2	3	4	5	6	7	8	9	10	11	12
a	b	c	d	e	f	g	h	i	j	k	l

(c) 完全二叉树的顺序存储形式

1	2	3	4	5	6	7	8	9	10	11
a	b	c	d	e	Ø	h	Ø	Ø	f	g

(d) 非完全二叉树的顺序存储形式

二叉树的链式存储结构

❑ 二叉链表

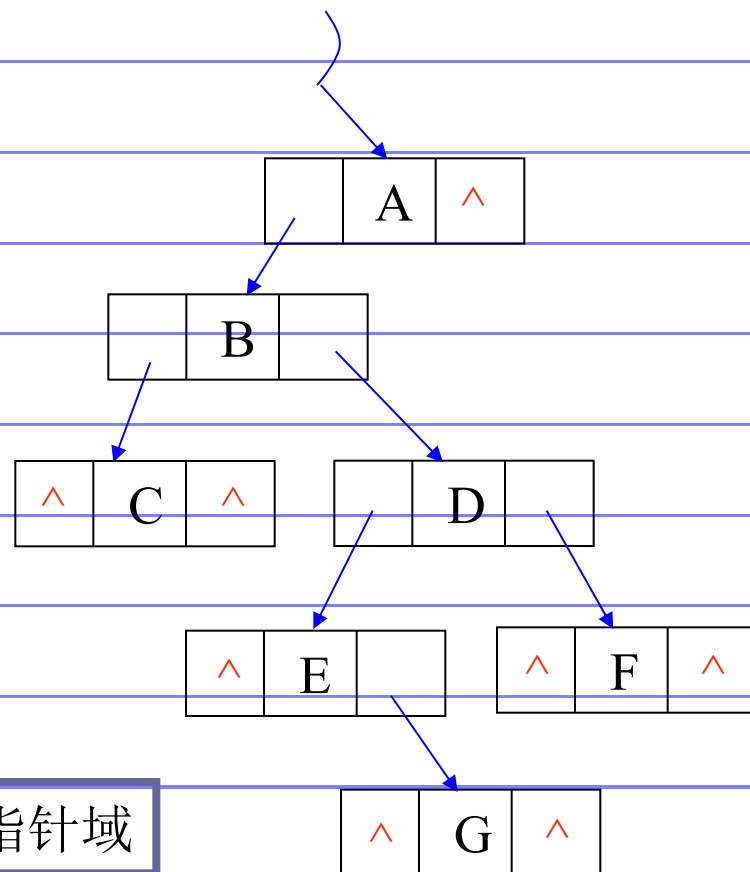
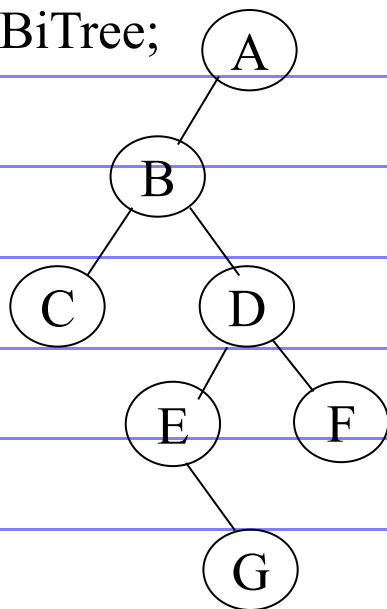
```
typedef struct BiTNode {
```

lchild	data	rchild
--------	------	--------

```
TElemType data;
```

```
Struct BiTNode *lchild, *rchild;
```

```
} BiTNode, *BiTree;
```



在n个结点的二叉链表中，有n+1个空指针域

二叉树的链式存储结构

❑ 三叉链表 除二叉链表的三个域外，再增加一个指针域，用来指向结点的父结点

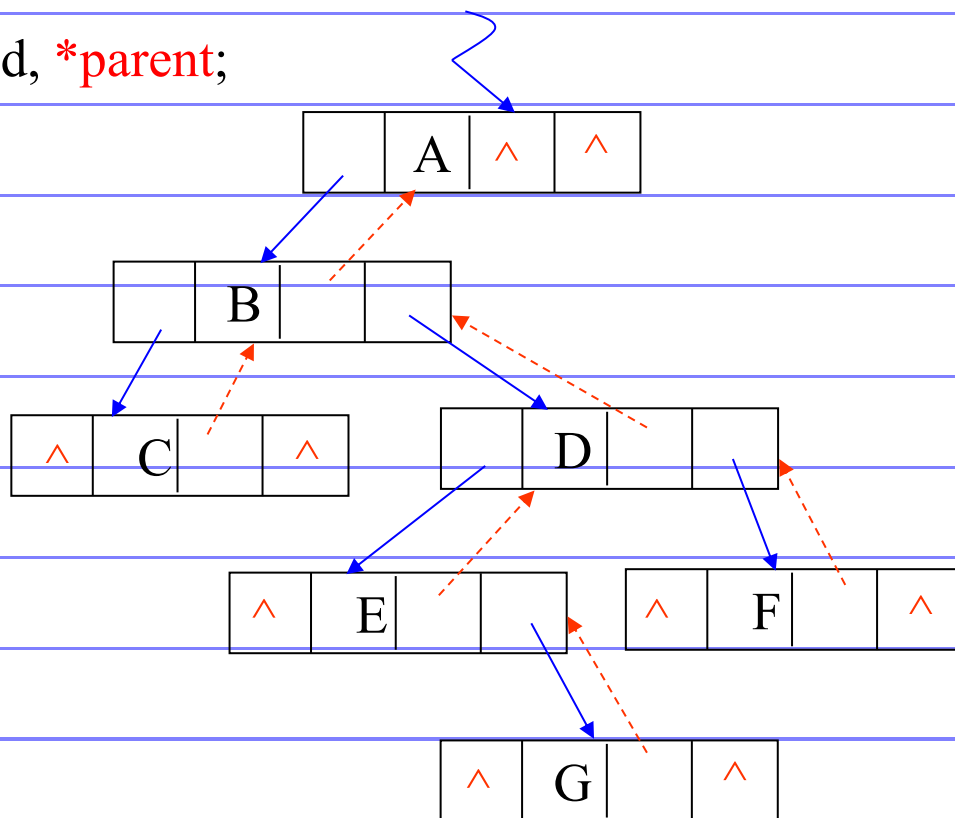
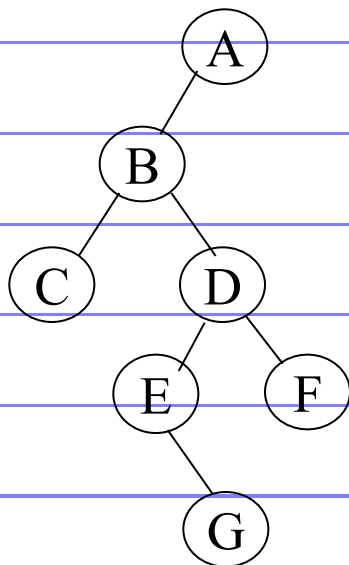
```
typedef struct TrTNode {
```

lchild	data	parent	rchild
--------	------	--------	--------

```
TElemType data;
```

```
Struct TrTNode*lchild, *rchild, *parent;
```

```
} TrTNode, TrTree;
```



二叉树的遍历

遍历二叉树(Traversing Binary Tree): 是指按指定的规律对二叉树中的每个结点访问一次且仅访问一次。

所谓**访问**是指对结点做某种处理。如：输出信息、修改结点的值等。

二叉树是一种非线性结构，每个结点都可能有左、右两棵子树，因此，需要寻找一种规律，使二叉树上的结点能排列在一个线性队列上，从而便于遍历。

二叉树的基本组成：根结点、左子树、右子树。若能依次遍历这三部分，就是遍历了二叉树。

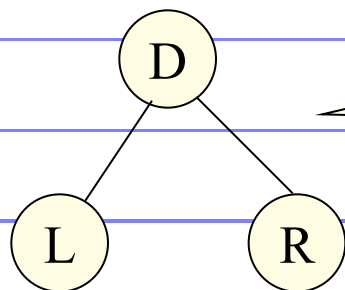
二叉树的遍历

□ 深度优先算法

- 先序遍历：先访问根结点,然后分别先序遍历左子树、右子树
- 中序遍历：先中序遍历左子树，然后访问根结点，最后中序遍历右子树
- 后序遍历：先后序遍历左、右子树，然后访问根结点

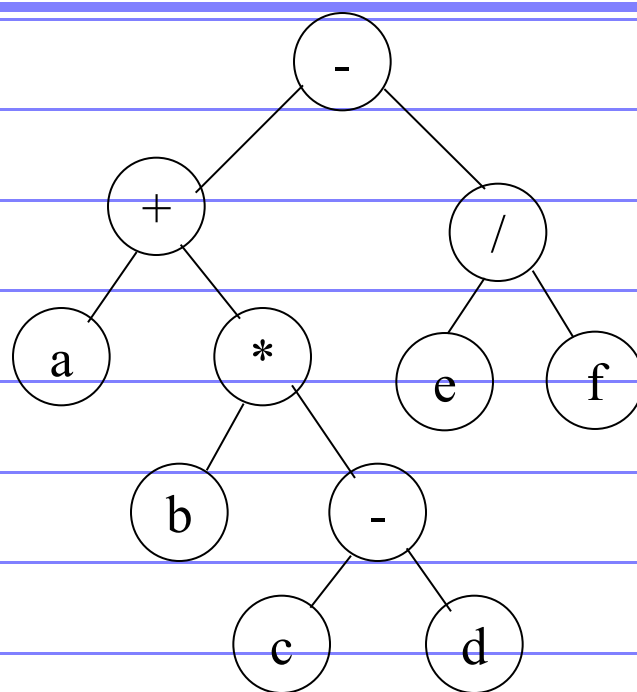
□ 广度优先算法

- 按层次遍历：从上到下、从左到右访问各结点



LDR、LRD、DLR
RDL、RLD、DRL

若以L、D、R分别表示遍历左子树、遍历根结点和遍历右子树，则有六种遍历方案：DLR、LDR、LRD、DRL、RDL、RLD。若规定先左后右，则只有前三种情况



先序遍历（前缀表示——波兰式）： - + a * b - c d / e f

中序遍历（中缀表示）： a + b * c - d - e / f

后序遍历（后缀表示——逆波兰式）： a b c d - * + e f / -

层次遍历： - + / a * e f b - c d

树的遍历算法

对于二叉树的遍历，分别讨论递归遍历算法和非递归遍历算法。递归遍历算法具有非常清晰的结构，但初学者往往难以接受或怀疑，不敢使用。实际上，递归算法是由系统通过使用堆栈来实现控制的。而非递归算法中的控制是由设计者定义和使用堆栈来实现的。

算法的递归定义是：

若二叉树为空，则遍历结束；否则

- (1) 访问根结点；
- (2) 先序遍历左子树(递归调用本算法)；
- (3) 先序遍历右子树(递归调用本算法)。

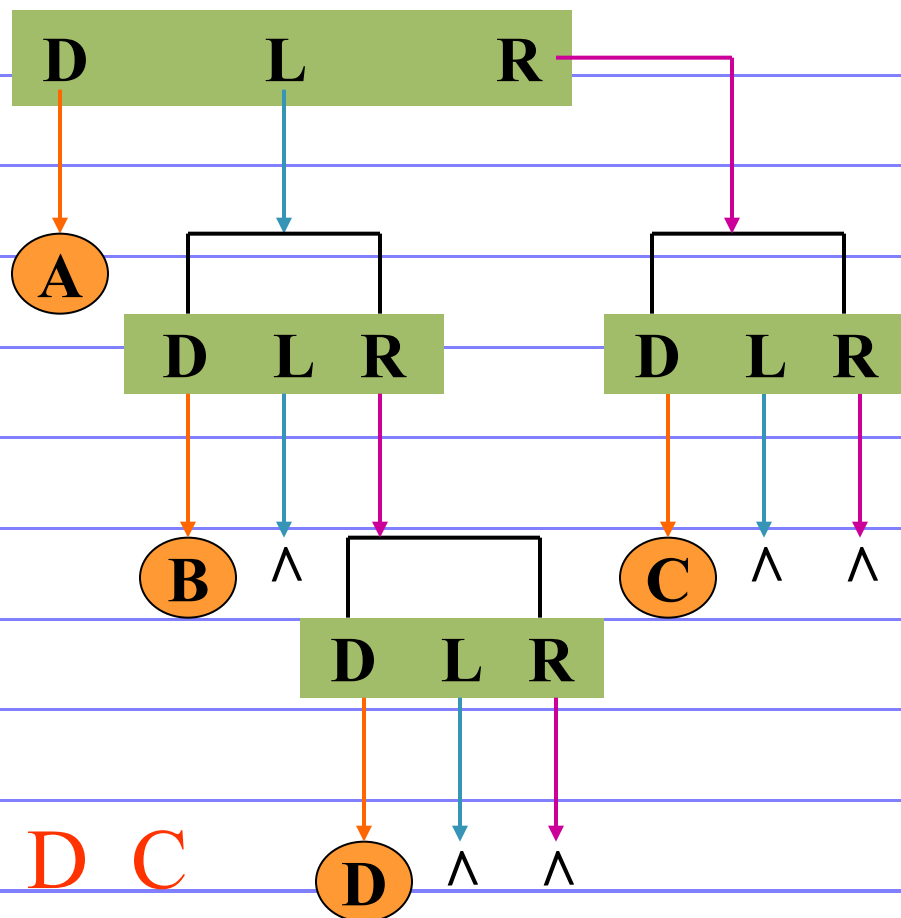
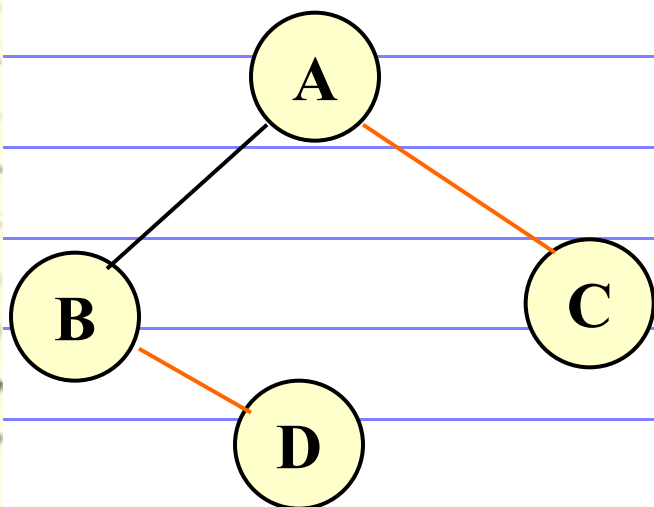
树的遍历算法

```
void preorder(TreeNode *bt)
{
    if(bt!=NULL){
        printf("%d\t",bt->data);
        preorder(bt->lchild);
        preorder(bt->rchild);
    }
}
```

```
void postorder(TreeNode *bt)
{
    if(bt!=NULL){
        postorder(bt->lchild);
        postorder(bt->rchild);
        printf("%d\t",bt->data);
    }
}
```

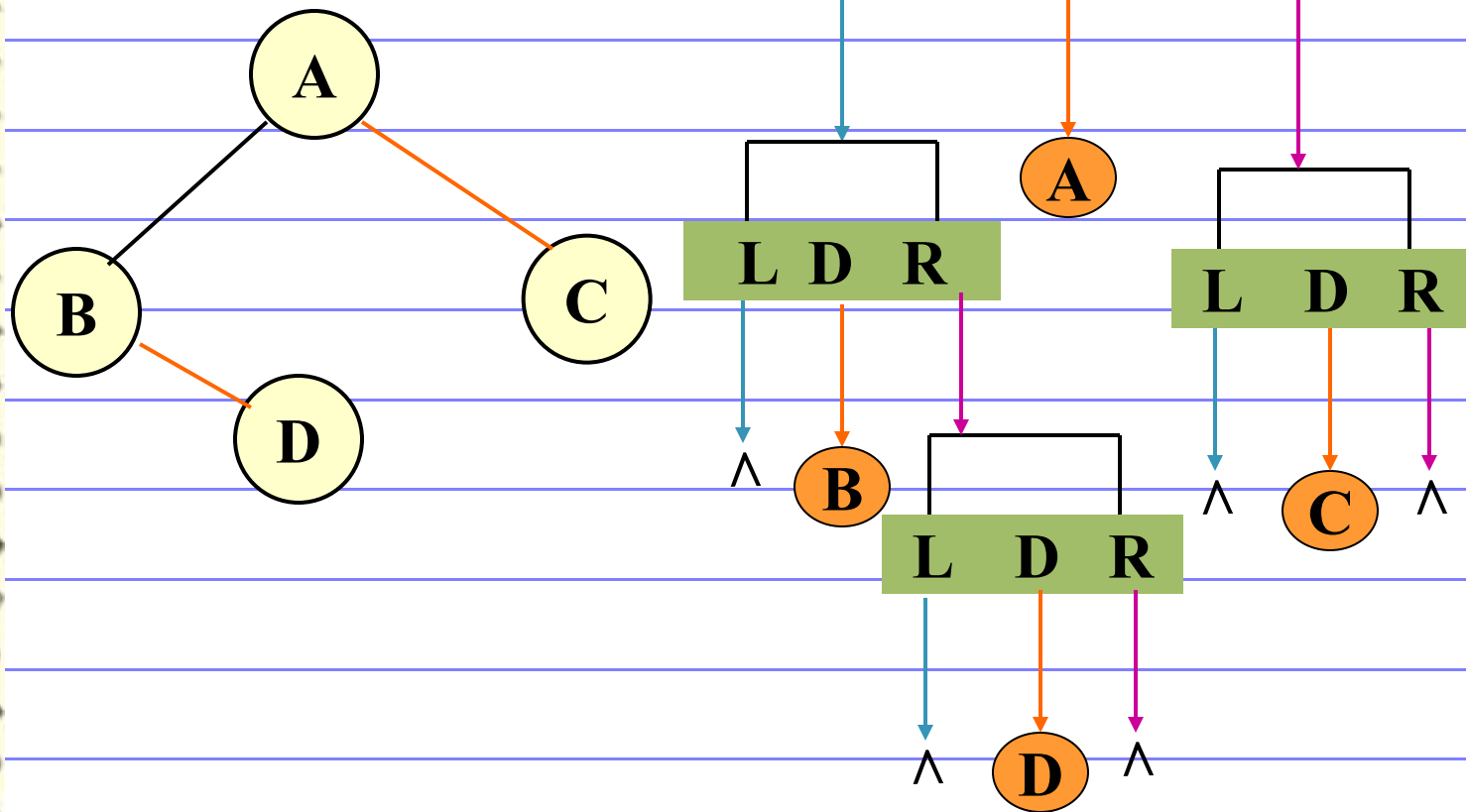
```
void inorder(TreeNode *bt)
{
    if(bt!=NULL) {
        inorder(bt->lchild);
        printf("%d\t",bt->data);
        inorder(bt->rchild);
    }
}
```

先序遍历:



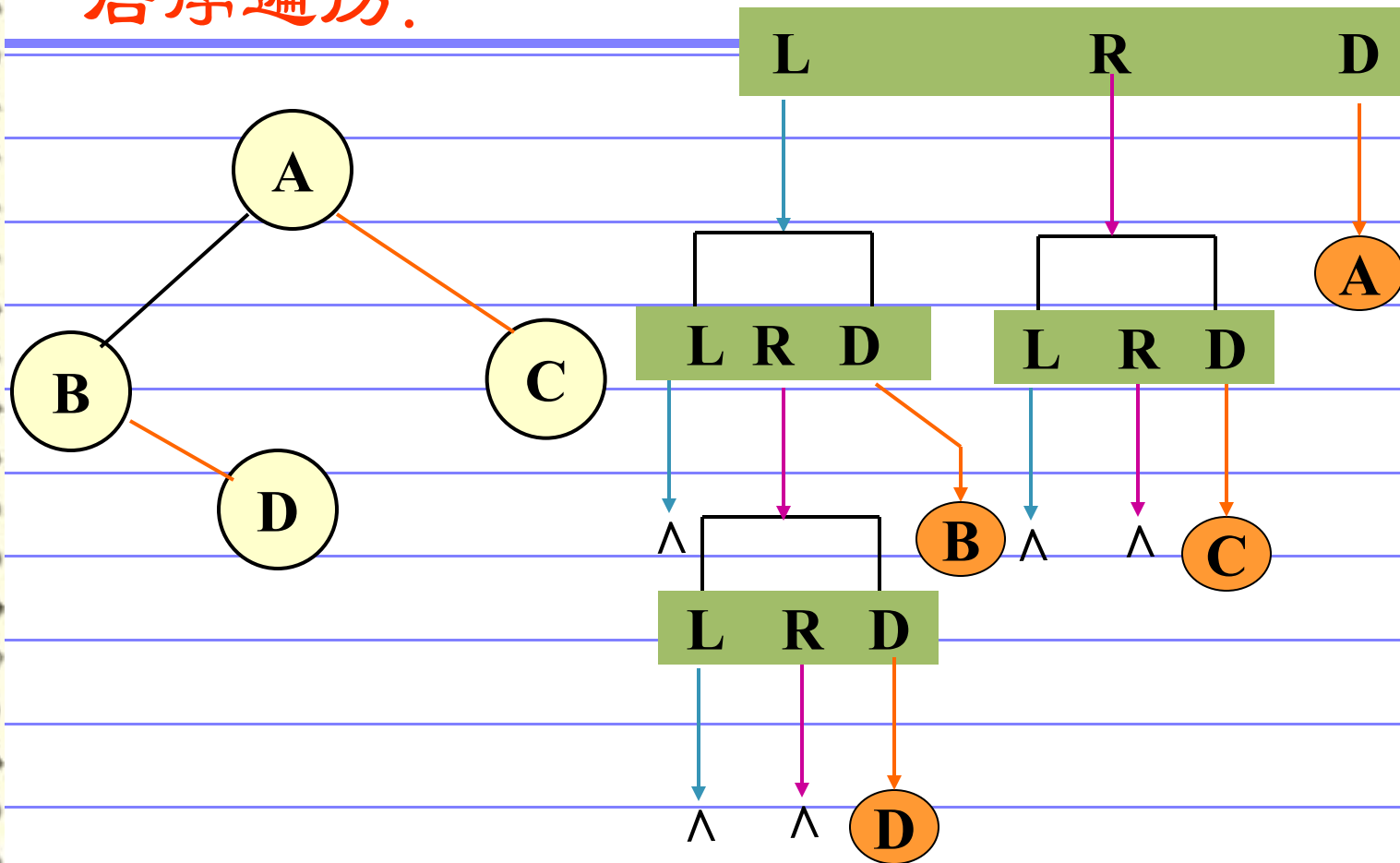
先序遍历序列: A B D C

中序遍历:



中序遍历序列: B D A C

后序遍历:



后序遍历序列: D B C A

```
void preorder(BiTree *bt)
```

```
{ if(bt!=NULL)
```

```
{ printf("%d\t",bt->data);
```

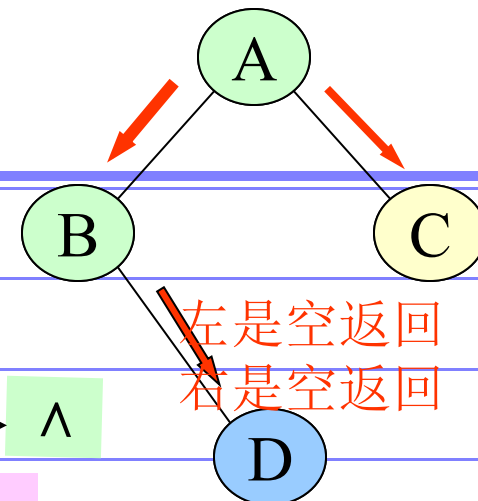
左是空返回

```
preorder(bt->lchild);
```

```
preorder(bt->rchild);
```

```
}
```

```
}
```



左是空返回
右是空返回

左是空返回
右是空返回

主程序

Pre(T)

T → A

printf(A);

pre(T → L);

pre(T → R);

T → B

printf(B);

pre(T → L);

pre(T → R);

T → C

printf(C);

pre(T → L);

pre(T → R);

T → Λ

返回

T → D

printf(D);

pre(T → L);

pre(T → R);

T → Λ

返回

T → Λ

返回

T → Λ

返回

T → Λ

返回

先序序列: A B D C

数据结构

Back

树的非递归遍历

先序遍历

设T是指向二叉树根结点的指针变量，非递归算法是：
若二叉树为空，则返回；否则，令p=T；

- (1) 访问p所指向的结点；
- (2) $q=p \rightarrow Rchild$ ，若q不为空，则q进栈；
- (3) $p=p \rightarrow Lchild$ ，若p不为空，转(1)，否则转(4)；
- (4) 退栈到p，转(1)，直到栈空为止。

树的非递归遍历 算法实现

```
#define MAX_NODE 50

void PreorderTraverse( BTreeNode *T)
{ BTreeNode *Stack[MAX_NODE], *p=T, *q ;
  int top=0 ;
  if (T==NULL) printf(“ Binary Tree is Empty!\n”) ;
  else { do
        { visit( p->data ) ; q=p->Rchild ;
          if ( q!=NULL ) stack[++top]=q ;
          p=p->Lchild ;
          if (p==NULL) { p=stack[top] ; top-- ; }
        }
        while (p!=NULL) ;
      }
}
```

树的非递归遍历

中序遍历

设T是指向二叉树根结点的指针变量，非递归算法是：

若二叉树为空，则返回；否则，令 $p=T$

(1) 若 p 不为空， p 进栈， $p=p \rightarrow Lchild$ ；

(2) 否则(即 p 为空)，退栈到 p ，访问 p 所指向的结点；

(3) $p=p \rightarrow Rchild$ ，转(1)；

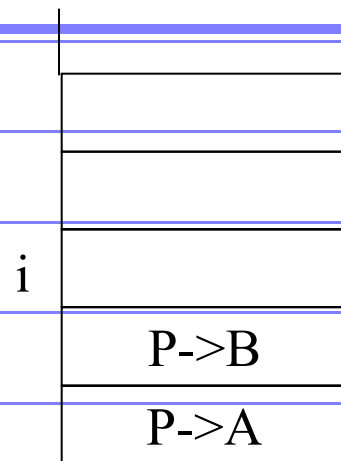
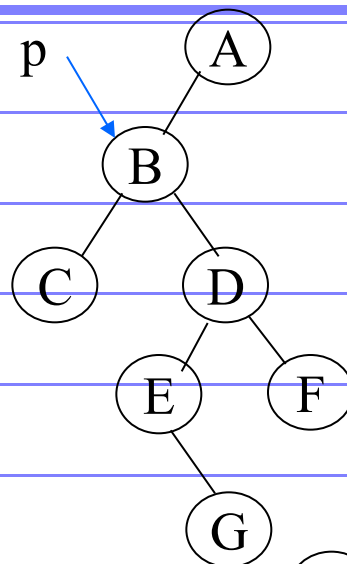
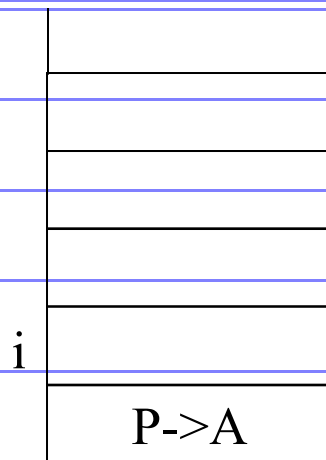
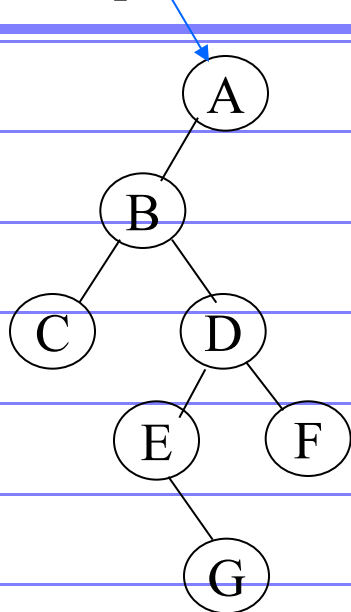
直到栈空为止。

树的非递归遍历 算法实现

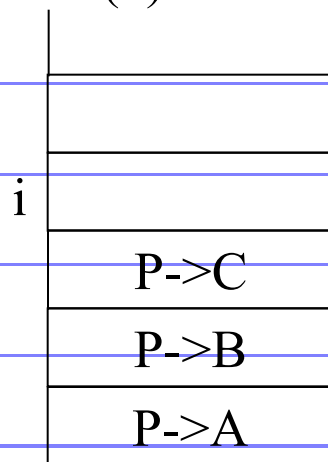
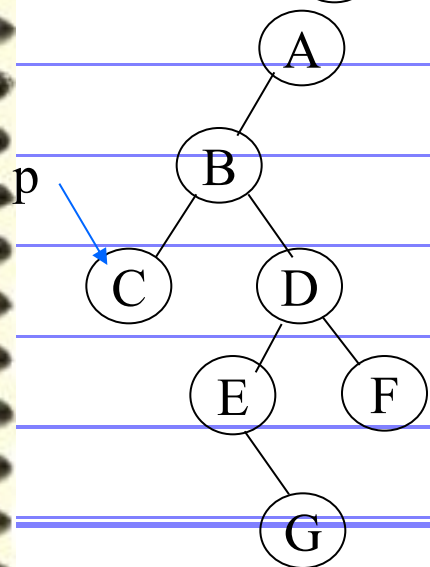
```
#define MAX_NODE 50

void InorderTraverse( BTreeNode *T)
{ BTreeNode *Stack[MAX_NODE], *p=T ;
  int top=0 , bool=1 ;
  if (T==NULL) printf(“ Binary Tree is Empty!\n”) ;
  else { do
    { while (p!=NULL)
      { stack[++top]=p ; p=p->Lchild ; }
      if (top==0) bool=0 ;
      else { p=stack[top] ; top-- ;
             visit( p->data ) ; p=p->Rchild ; }
    } while (bool!=0) ;
  }
}
```

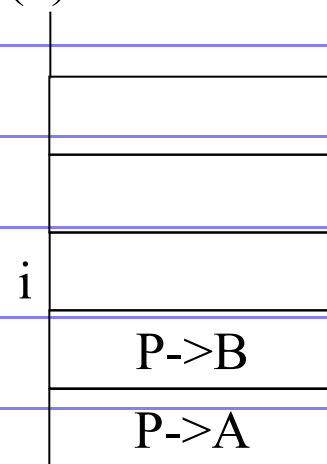
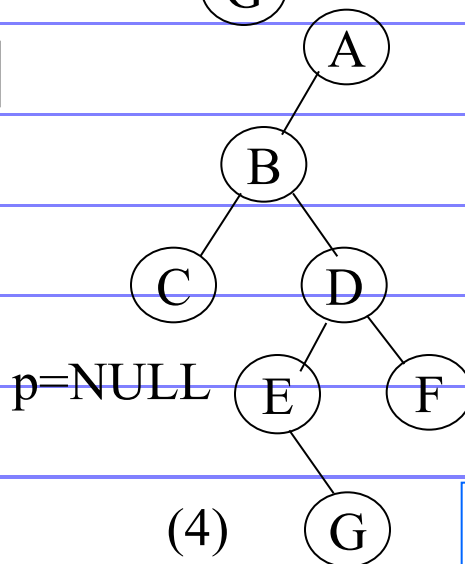
✧非递归算法



(2)

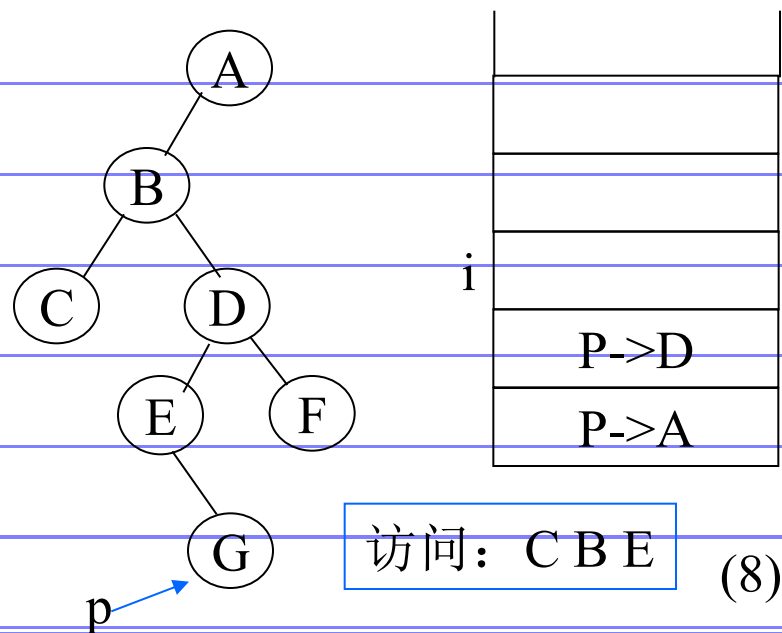
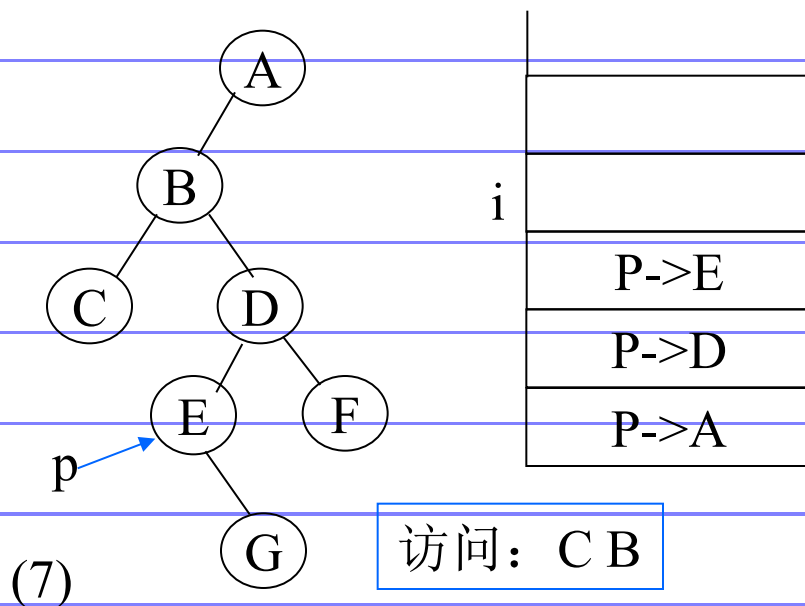
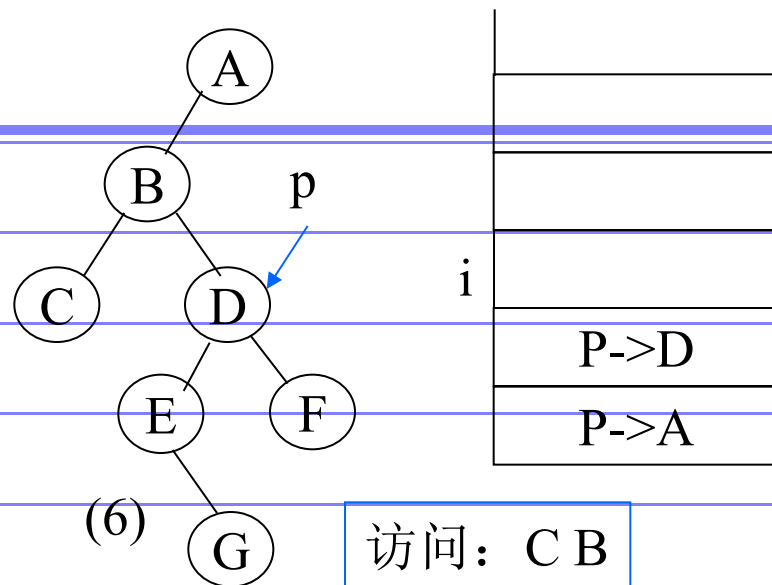
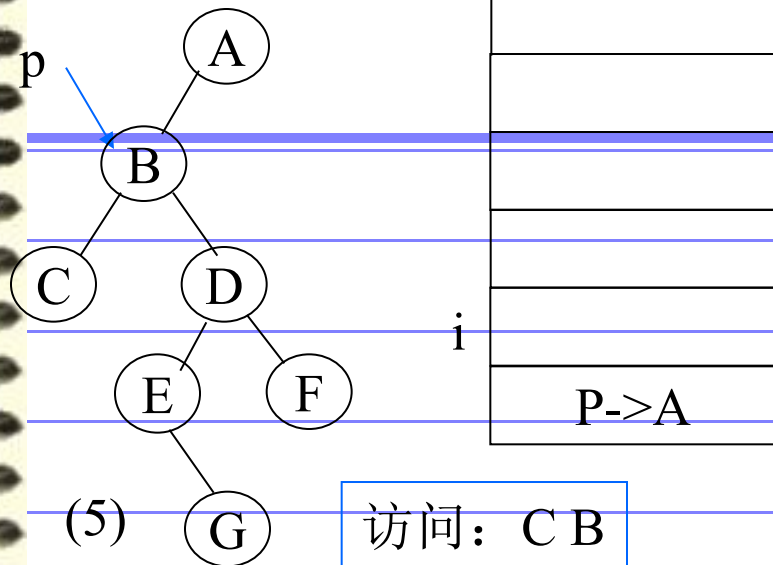


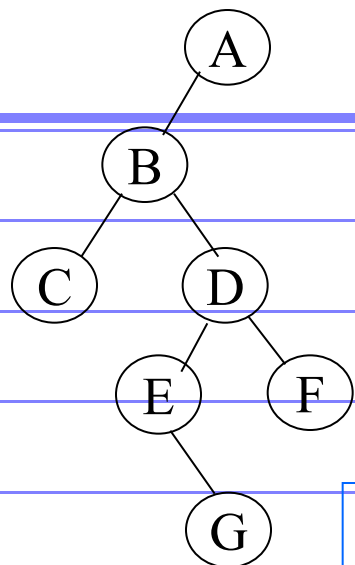
(3)



(4)

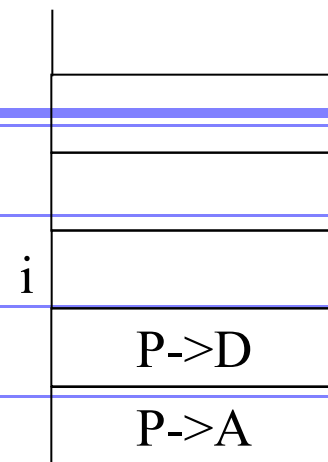
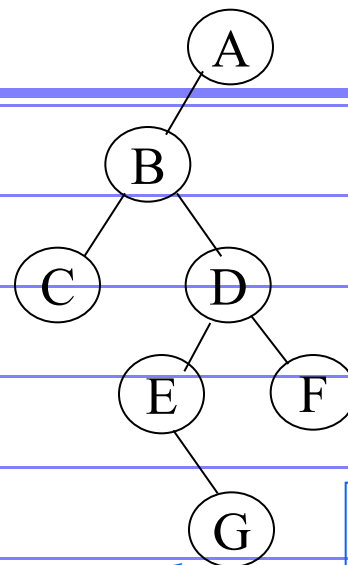
访问: C



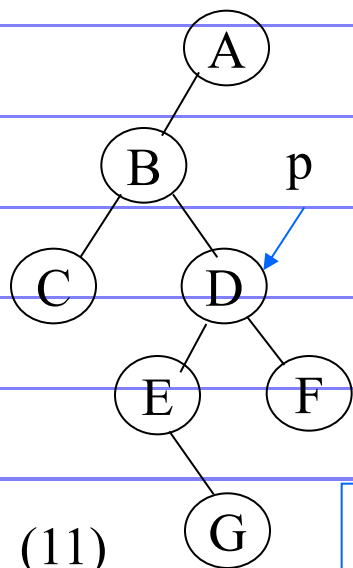


访问: C B E (9)

P=NULL

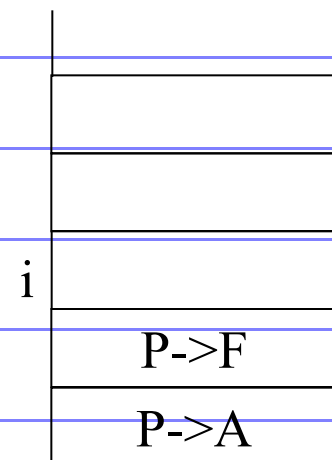
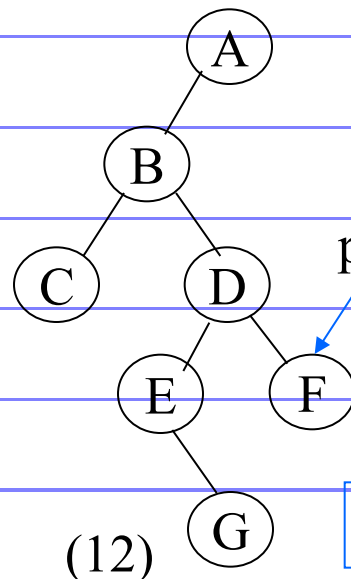


访问: C B E G (10)



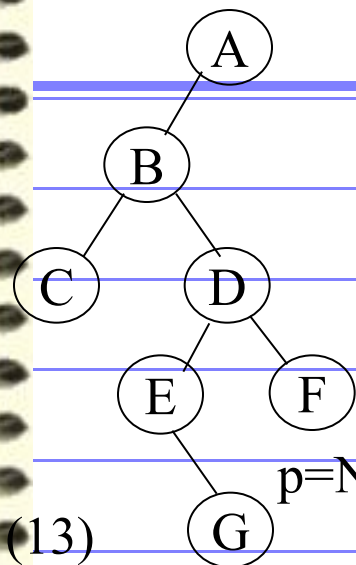
(11)

访问: C B E G D

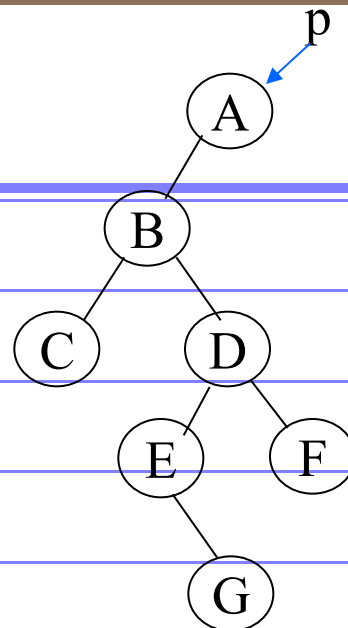


(12)

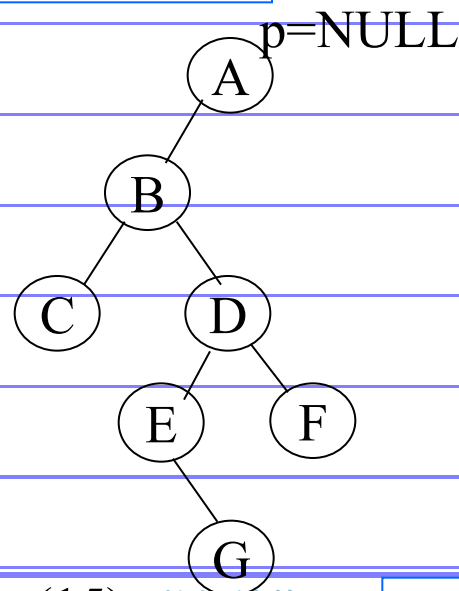
访问: C B E G D



访问: C B E G D F



访问: C B E G D F A



访问: C B E G D F A

树的非递归遍历

后序遍历

在后序遍历中，根结点是被最后访问的。因此，在遍历过程中，当搜索指针指向某一根结点时，不能立即访问，而要先遍历其左子树，此时根结点进栈。当其左子树遍历完后再搜索到该根结点时，还是不能访问，还需遍历其右子树。所以，此根结点还需再次进栈，当其右子树遍历完后再退栈到到该根结点时，才能被访问。

因此，设立一个状态标志变量tag：

$$\text{tag} = \begin{cases} 0 : & \text{结点暂不能访问} \\ 1 : & \text{结点可以被访问} \end{cases}$$

树的非递归遍历

其次，设两个堆栈 S_1 、 S_2 ， S_1 保存结点， S_2 保存结点的状态标志变量tag。 S_1 和 S_2 共用一个栈顶指针。

设T是指向根结点的指针变量，非递归算法是：

若二叉树为空，则返回；否则，令 $p=T$ ；

(1) 第一次经过根结点p，不访问：

p进栈 S_1 ， tag 赋值0， 进栈 S_2 ， $p=p \rightarrow Lchild$ 。

(2) 若p不为空，转(1)， 否则， 取状态标志值tag：

(3) 若tag=0： 对栈 S_1 ， 不访问， 不出栈； 修改 S_2 栈顶元素值(tag赋值1)， 取 S_1 栈顶元素的右子树， 即 $p=S_1[top] \rightarrow Rchild$ ， 转(1)；

(4) 若tag=1： S_1 退栈， 访问该结点；

直到栈空为止。

树的非递归遍历 算法实现

```
#define MAX_NODE 50
void PostorderTraverse( BTreeNode *T)
{ BTreeNode *S1[MAX_NODE] ,*p=T ;
  int S2[MAX_NODE] , top=0 , bool=1 ;
  if (T==NULL) printf("Binary Tree is Empty!\n") ;
  else { do
    { while (p!=NULL)
      { S1[++top]=p ; S2[top]=0 ;
        p=p->Lchild ;
      }
      if (top==0) bool=0 ;
      else if (S2[top]==0)
        { p=S1[top]->Rchild ; S2[top]=1 ; }
      else
        { p=S1[top] ; top-- ;
          visit( p->data ) ; p=NULL ;
          /* 使循环继续进行而不至于死循环 */
        }
    } while (bool!=0) ;
  }
}
```

层次遍历二叉树

层次遍历二叉树，是从根结点开始遍历，按层次次序“自上而下，从左至右”访问树中的各结点。

为保证是按层次遍历，必须设置一个队列，初始化时为空。

设T是指向根结点的指针变量，层次遍历非递归算法是：

若二叉树为空，则返回；否则，令 $p=T$ ， p 入队；

(1) 队首元素出队到 p ；

(2) 访问 p 所指向的结点；

(3) 将 p 所指向的结点的左、右子结点依次入队。直到队空为止。

层次遍历二叉树

算法实现

```
#define MAX_NODE 50

void LevelorderTraverse( BTreeNode *T)
{ BTreeNode *Queue[MAX_NODE], *p=T ;
  int front=0 , rear=0 ;
  if (p!=NULL)
  { Queue[++rear]=p; /* 根结点入队 */
    while (front<rear)
    { p=Queue[++front]; visit( p->data );
      if (p->Lchild!=NULL)
        Queue[++rear]=p; /* 左结点入队 */
      if (p->Rchild!=NULL)
        Queue[++rear]=p; /* 右结点入队 */
    }
  }
}
```

二叉树遍历算法的应用

“遍历”是二叉树最重要的基本操作，是各种其它操作的基础，二叉树的许多其它操作都可以通过遍历来实现。如建立二叉树的存储结构、求二叉树的结点数、求二叉树的深度等。

1 二叉树的二叉链表创建

(1) 按满二叉树方式建立

在此补充按满二叉树的方式对结点进行编号建立链式二叉树。
对每个结点，输入i、ch。

$$\begin{cases} i: & \text{结点编号, 按从小到大的顺序输入} \\ ch: & \text{结点内容, 假设是字符} \end{cases}$$

在建立过程中借助一个一维数组S[n]，编号为i的结点保存在S[i]中。

```
#define MAX_NODE 50
```

```
typedef struct BTreeNode
```

```
{ char data ;
```

```
    struct BTreeNode *Lchild , *Rchild ;
```

```
}BTreeNode ;
```

```
BTreeNode *Create_BTree(void) //建立链式二叉树，返回指向根结点的指针变量
```

```
{ BTreeNode *T , *p , *s[MAX_NODE] ;
```

```
    char ch ; int i , j ;
```

```
    while (1) {
```

```
        scanf("%d", &i) ;
```

```
        if (i==0) break ; /* 以编号0作为输入结束 */
```

```
        else { ch=getchar() ;
```

```
            p=(BTreeNode *)malloc(sizeof(BTreeNode)) ;
```

```
            p->data=ch ;
```

```
            p->Lchild=p->Rchild=NULL ; s[i]=p ;
```

```
            if (i==1) T=p ;
```

```
            else { j=i/2 ; /* j是i的双亲结点编号 */
```

```
                if (i%2==0) s[j]->Lchild=p ;
```

```
                else s[j]->Rchild=p ;
```

```
            }
```

```
        }
```

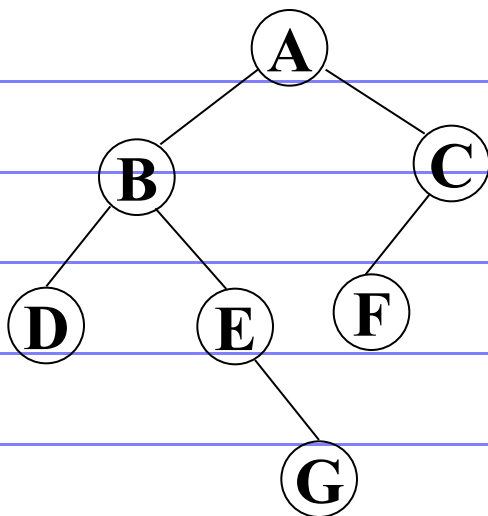
```
    }
```

```
    return(T) ;
```

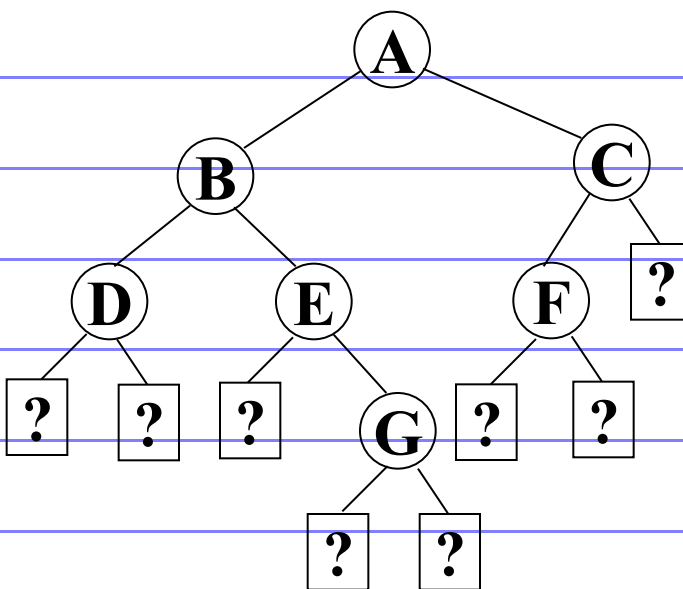
```
}
```

(2) 按先序遍历方式建立

对一棵二叉树进行“扩充”，就可以得到有该二叉树所扩充的二叉树。有两棵二叉树 T_1 及其扩充的二叉树 T_2 如图6-10所示。



(a) 二叉树 T_1



(b) T_1 的扩充二叉树 T_2

图6-10 二叉树 T_1 及其扩充二叉树 T_2

二叉树的扩充方法是：在二叉树中结点的每一个空链域处增加一个扩充的结点(总是叶子结点，用方框“□”表示)。对于二叉树的结点值：

- ◆ 是char类型：扩充结点值为“？”；

- ◆ 是int类型：扩充结点值为0或-1；

下面的算法是二叉树的前序创建的递归算法，读入一棵二叉树对应的扩充二叉树的前序遍历的结点值序列。每读入一个结点值就进行分析：

- ◆ 若是扩充结点值：令根指针为NULL；

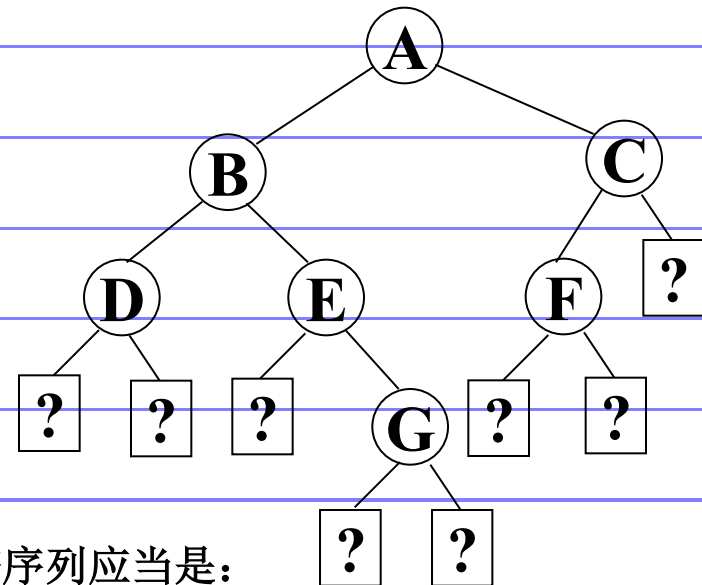
- ◆ 若是(正常)结点值：动态地为根指针分配一个结点，将该值赋给根结点，然后递归地创建根的左子树和右子树。

```

#define NULLKY '?'
#define MAX_NODE 50
typedef struct BTreeNode
{
    char data ;
    struct BTreeNode *Lchild , *Rchild ;
}BTreeNode ;

BTreeNode *Preorder_Create_BTree(BTreeNode *T)
/* 建立链式二叉树，返回指向根结点的指针变量 */
{
    char ch ;
    ch=getchar() ;
    if (ch==NULLKY)
        { T=NULL; return(T) ; }
    else
        { T=(BTreeNode *)malloc(sizeof(BTreeNode)) ;
          T->data=ch ;
          Preorder_Create_BTree(T->Lchild) ;
          Preorder_Create_BTree(T->Rchild) ;
          return(T) ;
        }
}

```



当希望创建图6-10(a)所示的二叉树时，输入的字符序列应当是：

ABD??E?G??CF???

2 求二叉树的叶子结点数

可以直接利用先序遍历二叉树算法求二叉树的叶子结点数。只要将先序遍历二叉树算法中**vist()**函数简单地进行修改就可以。

```
#define MAX_NODE 50
int search_leaves( BTreeNode *T)
{
    BTreeNode *Stack[MAX_NODE] ,*p=T;
    int top=0, num=0;
    if (T!=NULL) {
        stack[++top]=p ;
        while (top>0)
        { p=stack[top--] ;
            if (p->Lchild==NULL&& p->Rchild==NULL) num++ ;
            if (p->Lchild!=NULL )
                stack[++top]=p->Lchild;
            if (p->Rchild!=NULL )
                stack[++top]=p->Rchild;
        }
    }
    return(num) ;
}
```

3 求二叉树的深度

利用层次遍历算法可以直接求得二叉树的深度。

```
#define MAX_NODE 50
int search_depth( BTreeNode *T)
{ BTreeNode *Stack[MAX_NODE] ,*p=T;
  int front=0 , rear=0, depth=0, level ;
  /* level总是指向访问层的最后一个结点在队列的位置 */
  if (T!=NULL)
  { Queue[++rear]=p; /* 根结点入队 */
    level=rear ; /* 根是第1层的最后一个节点 */
    while (front<rear)
    { p=Queue[++front];
      if (p->Lchild!=NULL)
        Queue[++rear]=p; /* 左结点入队 */
      if (p->Rchild!=NULL)
        Queue[++rear]=p; /* 右结点入队 */
      if (front==level)
        /* 正访问的是当前层的最后一个结点 */
        { depth++ ; level=rear ; }
    }
  }
}
```

线索二叉树

遍历二叉树是按一定的规则将树中的结点排列成一个线性序列，即是对非线性结构的线性化操作。如何找到遍历过程中动态得到的每个结点的直接前驱和直接后继(第一个和最后一个除外)?如何保存这些信息?

设一棵二叉树有 n 个结点，则有 $n-1$ 条边(指针连线)，而 n 个结点共有 $2n$ 个指针域(Lchild和Rchild)，显然有 $n+1$ 个空闲指针域未用。则可以利用这些空闲的指针域来存放结点的直接前驱和直接后继信息。

线索二叉树

对结点的指针域做如下规定：

- ◆ 若结点有左孩子，则Lchild指向其左孩子，否则，指向其直接前驱；
 - ◆ 若结点有右孩子，则Rchild指向其右孩子，否则，指向其直接后继；
- 为避免混淆，对结点结构加以改进，增加两个标志域，如图6-10所示。

Lchild	Ltag	data	Rchild	Rtag
--------	------	------	--------	------

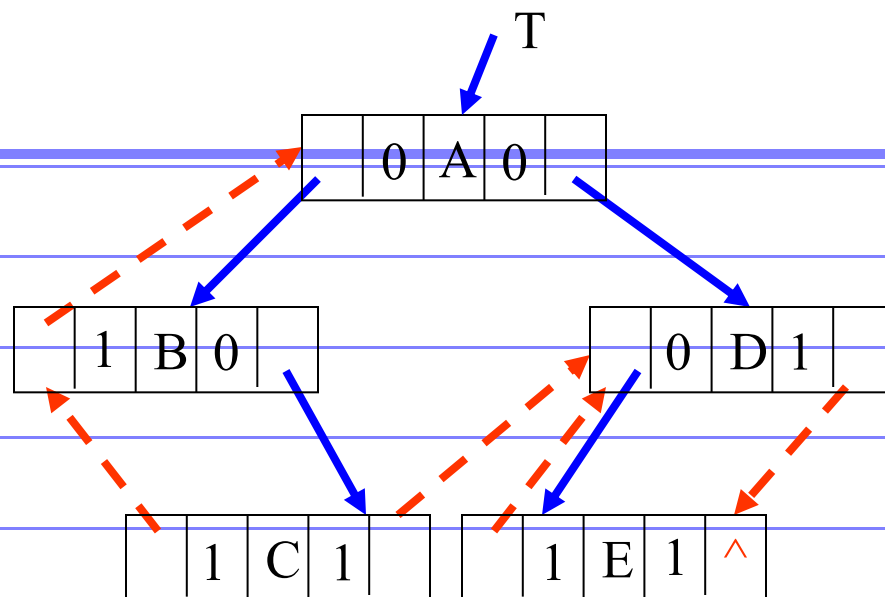
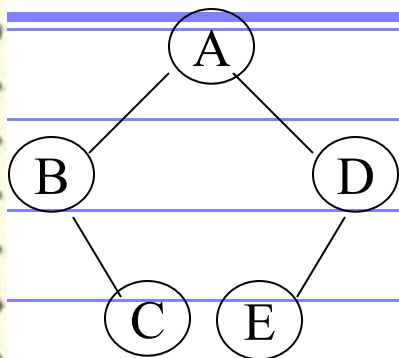
图6-10 线索二叉树的结点结构

$Ltag = \begin{cases} 0: & \text{Lchild域指示结点的左孩子} \\ 1: & \text{Lchild域指示结点的前驱} \end{cases}$

$Rtag = \begin{cases} 0: & \text{Rchild域指示结点的右孩子} \\ 1: & \text{Rchild域指示结点的后继} \end{cases}$

线索二叉树

- ✧ 用这种结点结构构成的二叉树的存储结构；叫做线索链表；
- ✧ 前驱与后继：在二叉树的先序、中序或后序遍历序列中两个相邻的结点互称为~
- ✧ 线索：指向前驱或后继结点的指针称为~
- ✧ 线索二叉树：加上线索的二叉链表表示的二叉树叫~
- ✧ 线索化：对二叉树按某种遍历次序使其变为线索二叉树的过程叫~



先序序列: ABCDE

先序线索二叉树

```
typedef enum PointerTag { Link, Thread};
```

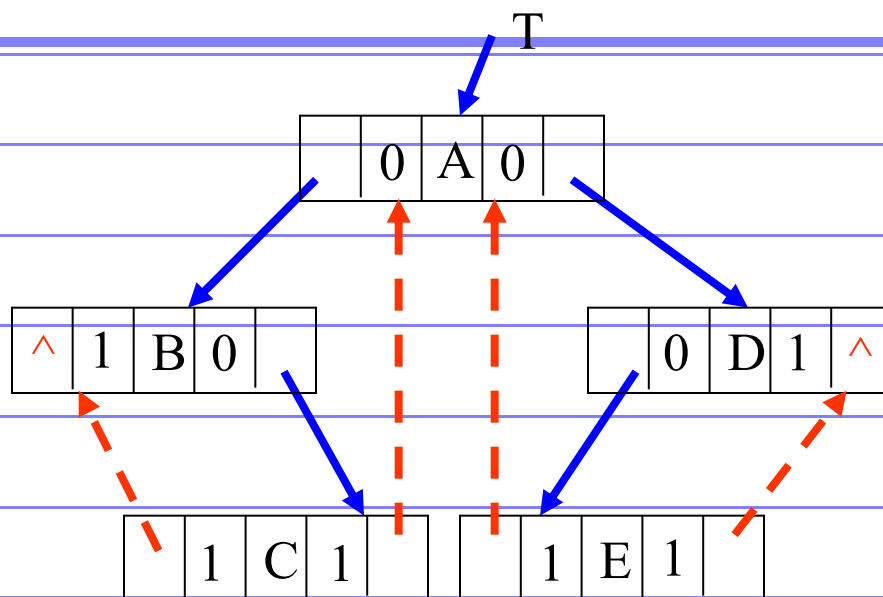
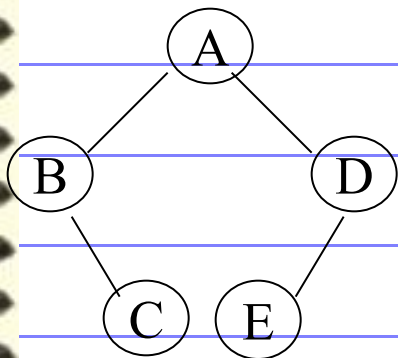
```
typedef struct BiThrNode {
```

```
    TElemType data;
```

```
    PointerTag LTag, RTag;
```

```
    struct BiThrNode *lchild, *rchild;
```

```
} BiThrNode, *BiThrTree;
```



```
typedef enum PointerTag { Link, Thread};
```

```
typedef struct BiThrNode {
```

```
    TElemType data;
```

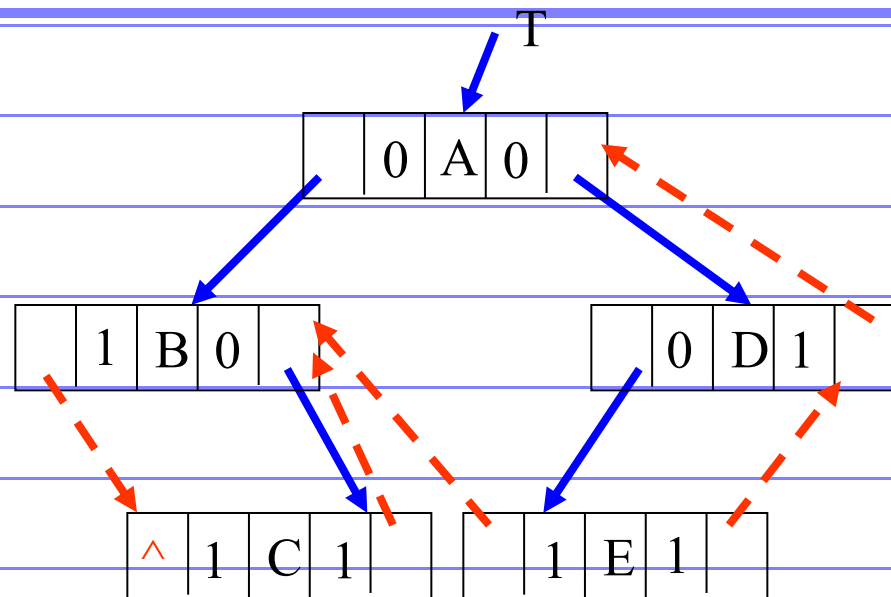
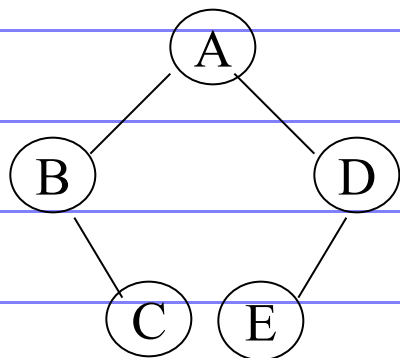
```
    PointerTag LTag, RTag;
```

```
    struct BiThrNode *lchild, *rchild;
```

```
} BiThrNode, *BiThrTree;
```

中序序列: BCAED

中序线索二叉树



```
typedef enum PointerTag { Link, Thread};
```

```
typedef struct BiThrNode {
```

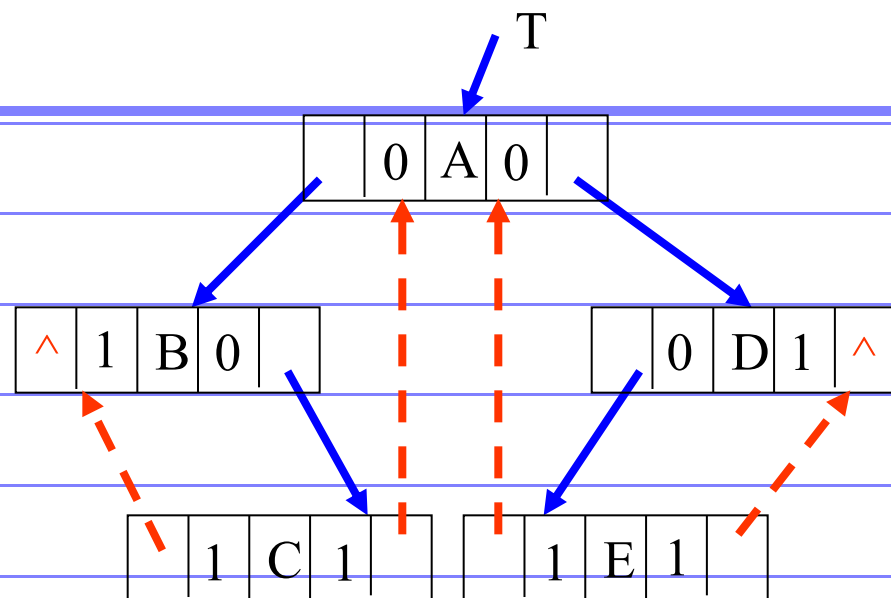
```
    TElemType data;
```

```
    PointerTag LTag, RTag;
```

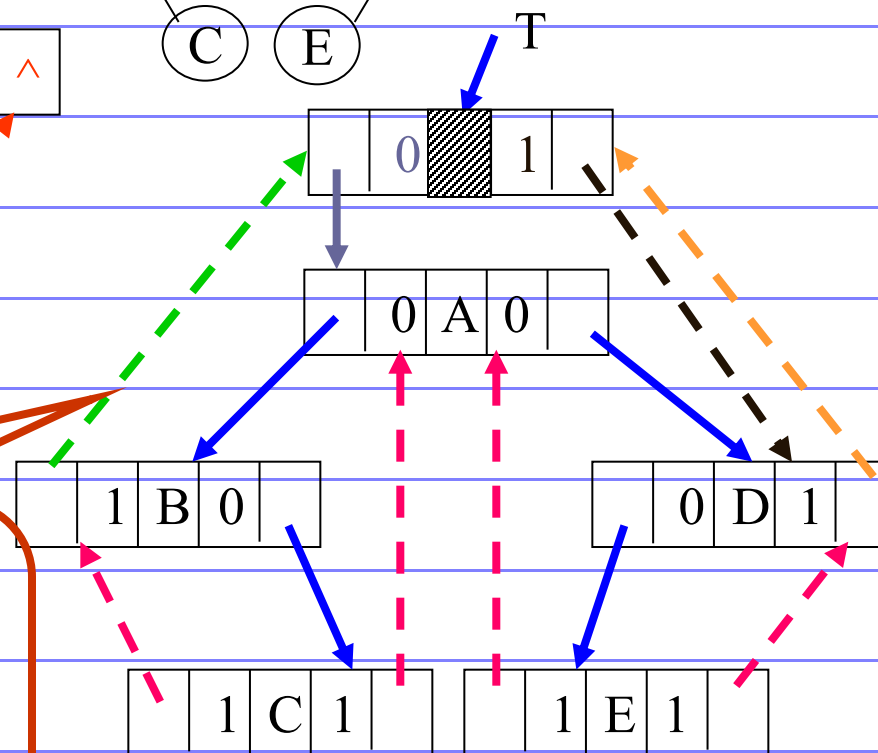
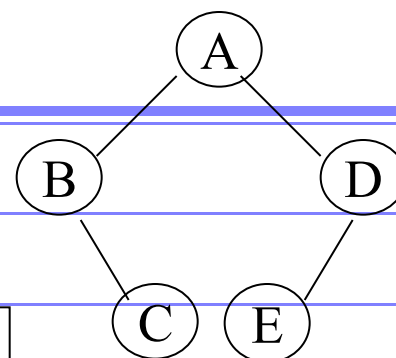
```
    struct BiThrNode *lchild, *rchild;
```

```
} BiThrNode, *BiThrTree;
```

后序序列: CBEDA
后序线索二叉树



中序序列: BCAED
中序线索二叉树



中序序列: BCAED
带头结点的中序线索二叉树

头结点:
lt=0, lc指向根结点
rt=1, rc指向遍历序列中最后一个结点

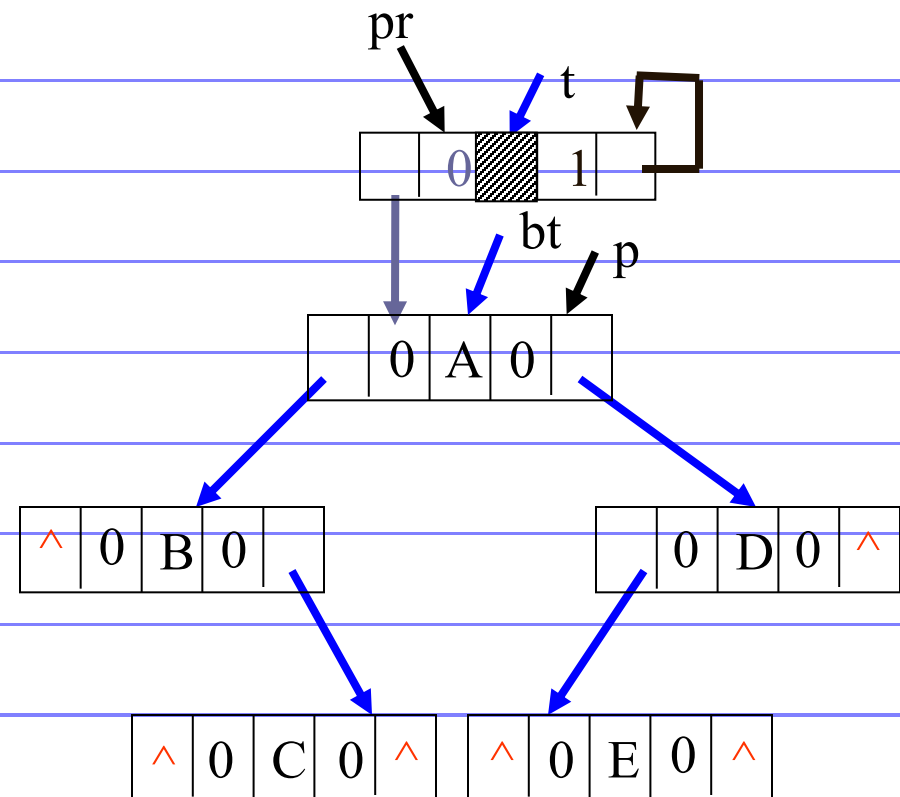
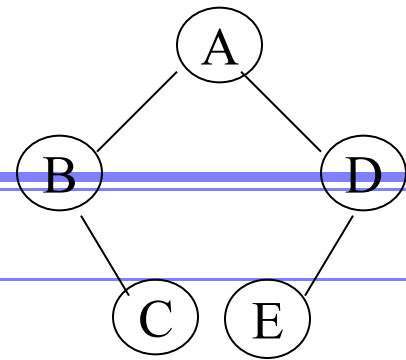
遍历序列中第一个结点的lc域和最后一个结点的rc域都指向头结点

✧ 算法

✧ 按中序线索化二叉树

i

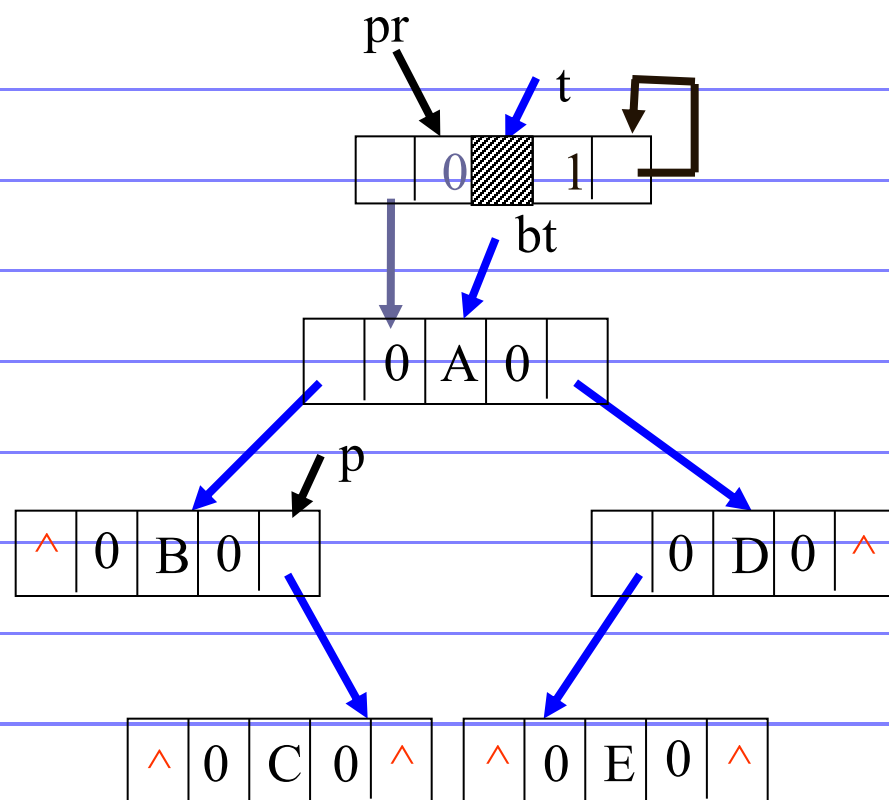
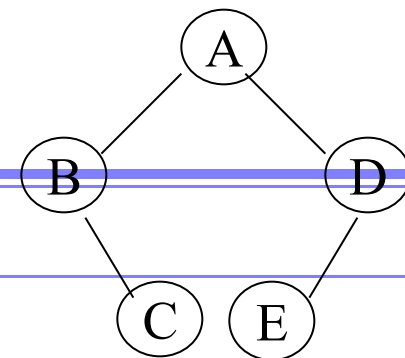
P->A



✧ 算法

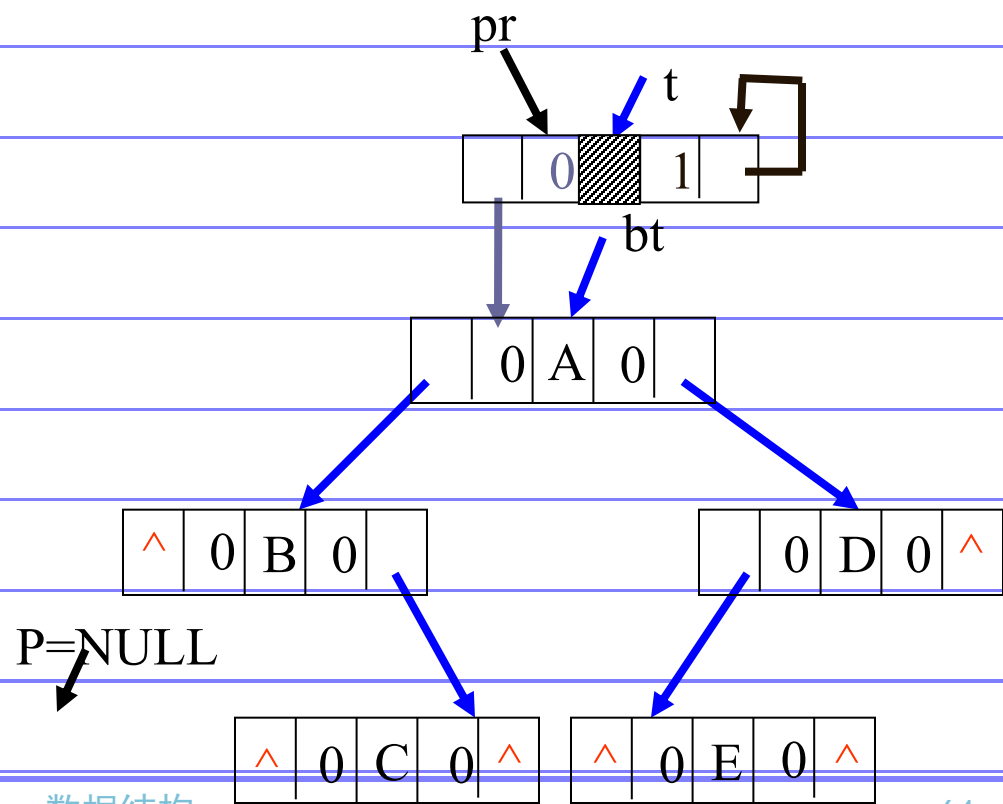
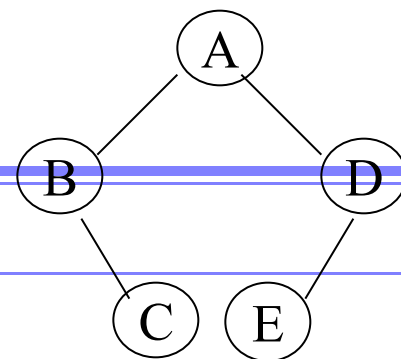
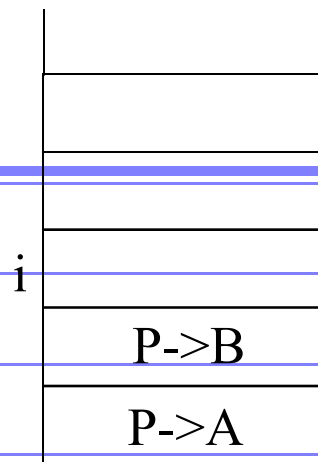
✧ 按中序线索化二叉树

i
P->B
P->A



✧ 算法

✧ 按中序线索化二叉树

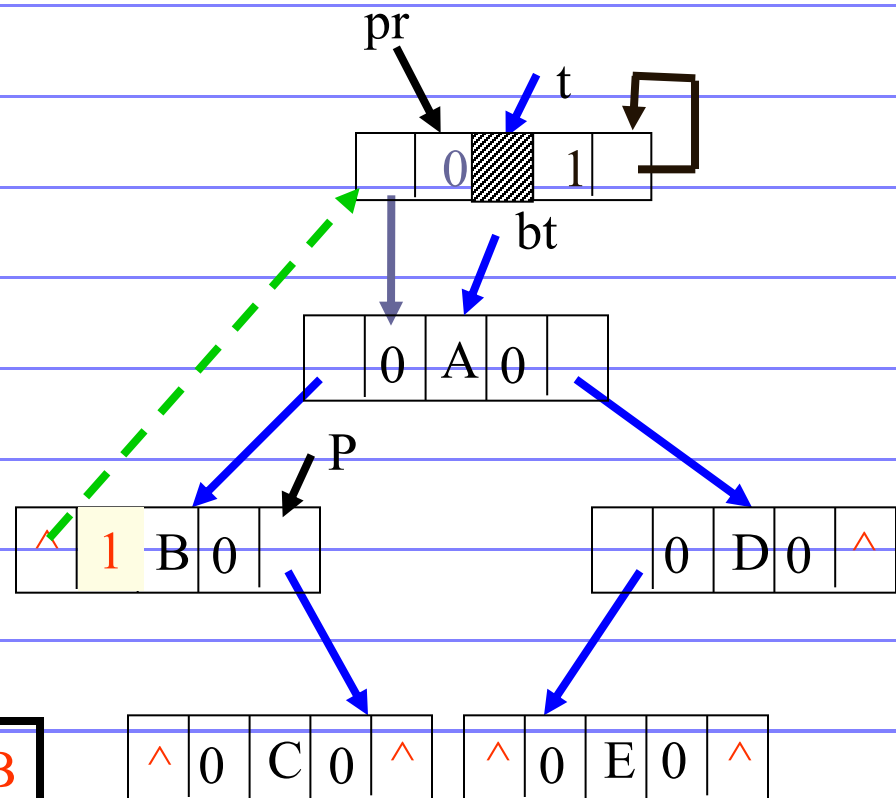
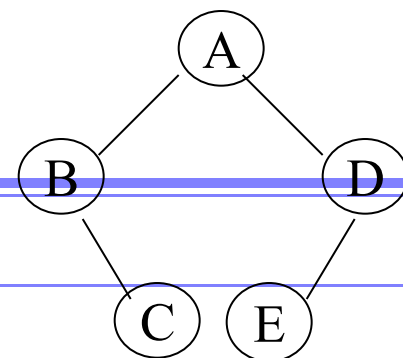


✧ 算法

✧ 按中序线索化二叉树

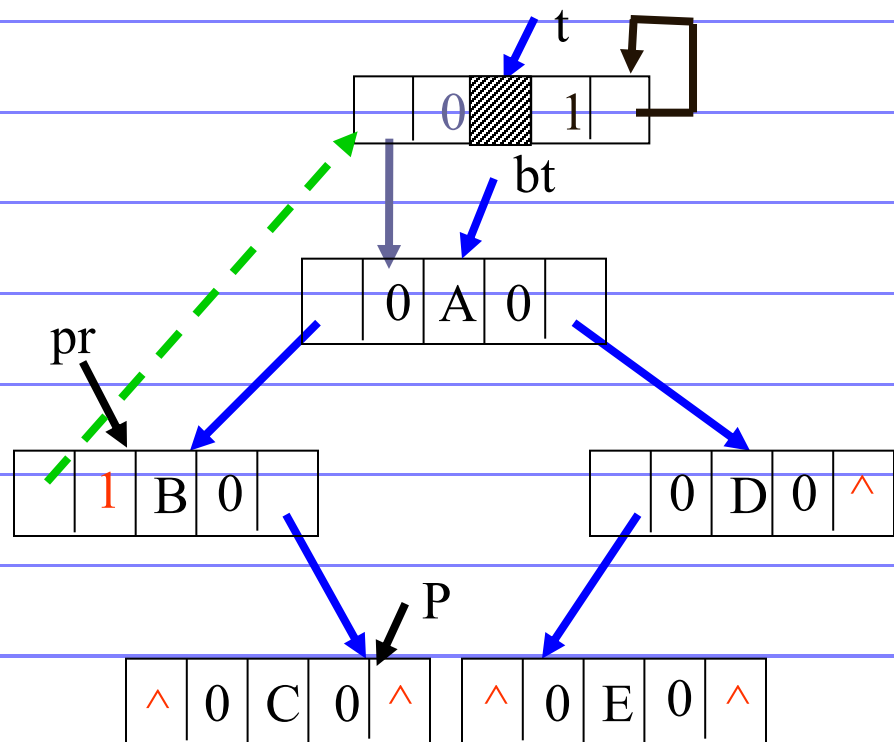
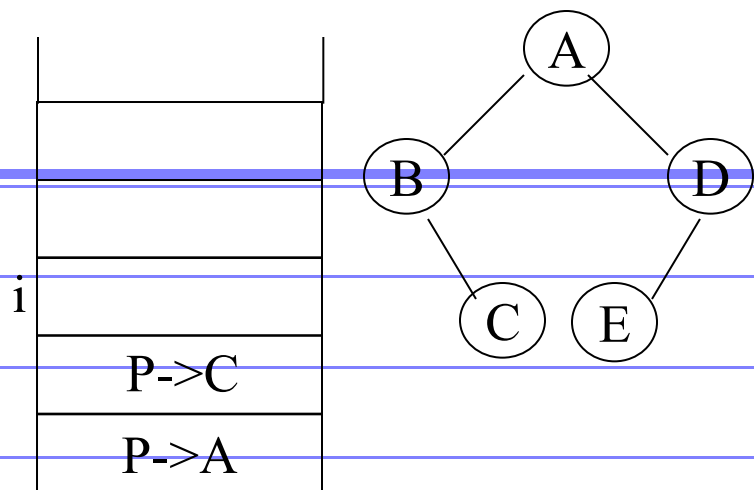
i

P->A



✧ 算法

✧ 按中序线索化二叉树



输出:B

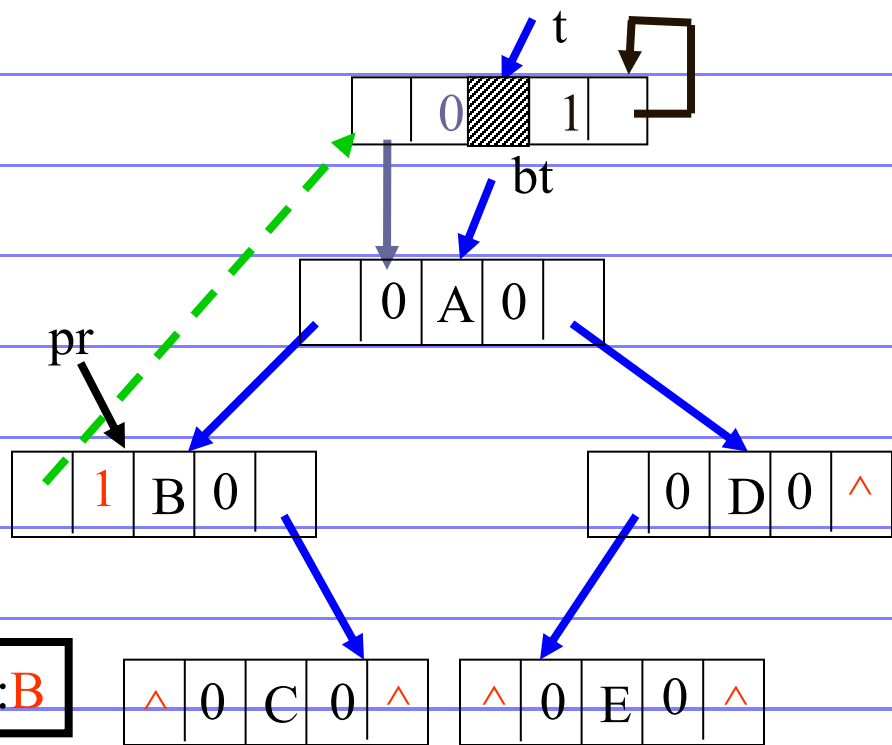
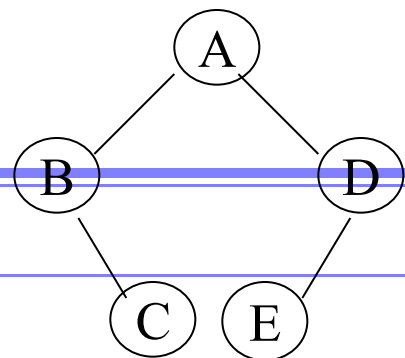
✧ 算法

✧ 按中序线索化二叉树

i

P->C

P->A



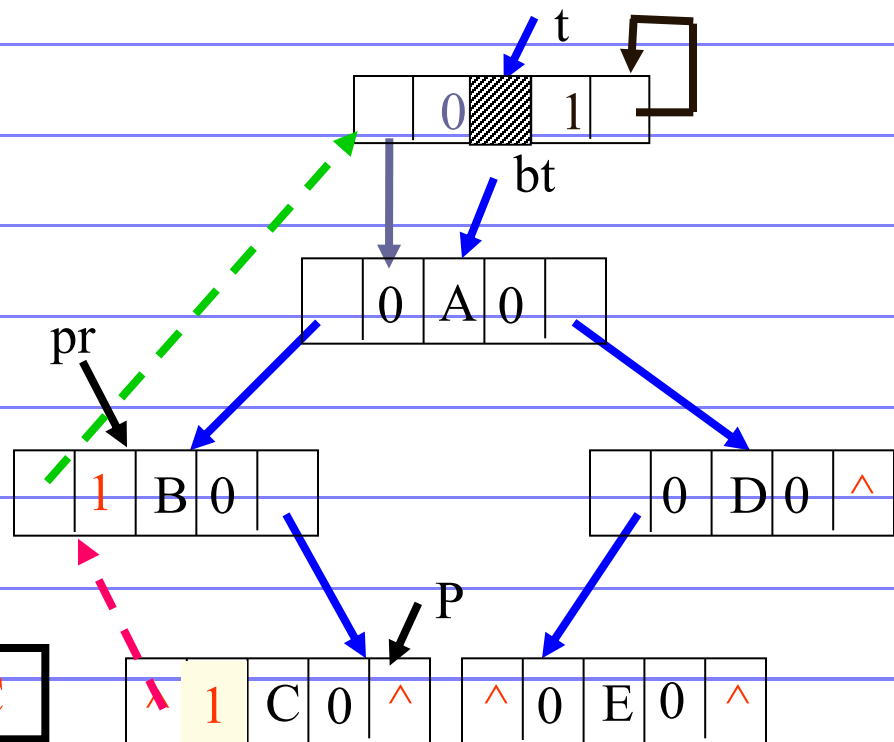
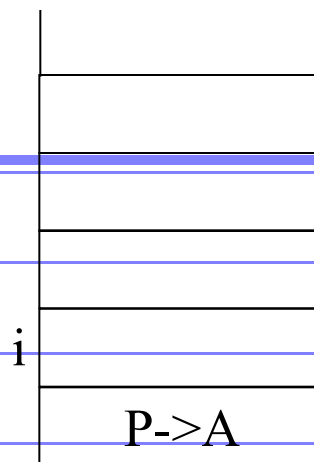
输出: **B**

P=NULL

数据结构

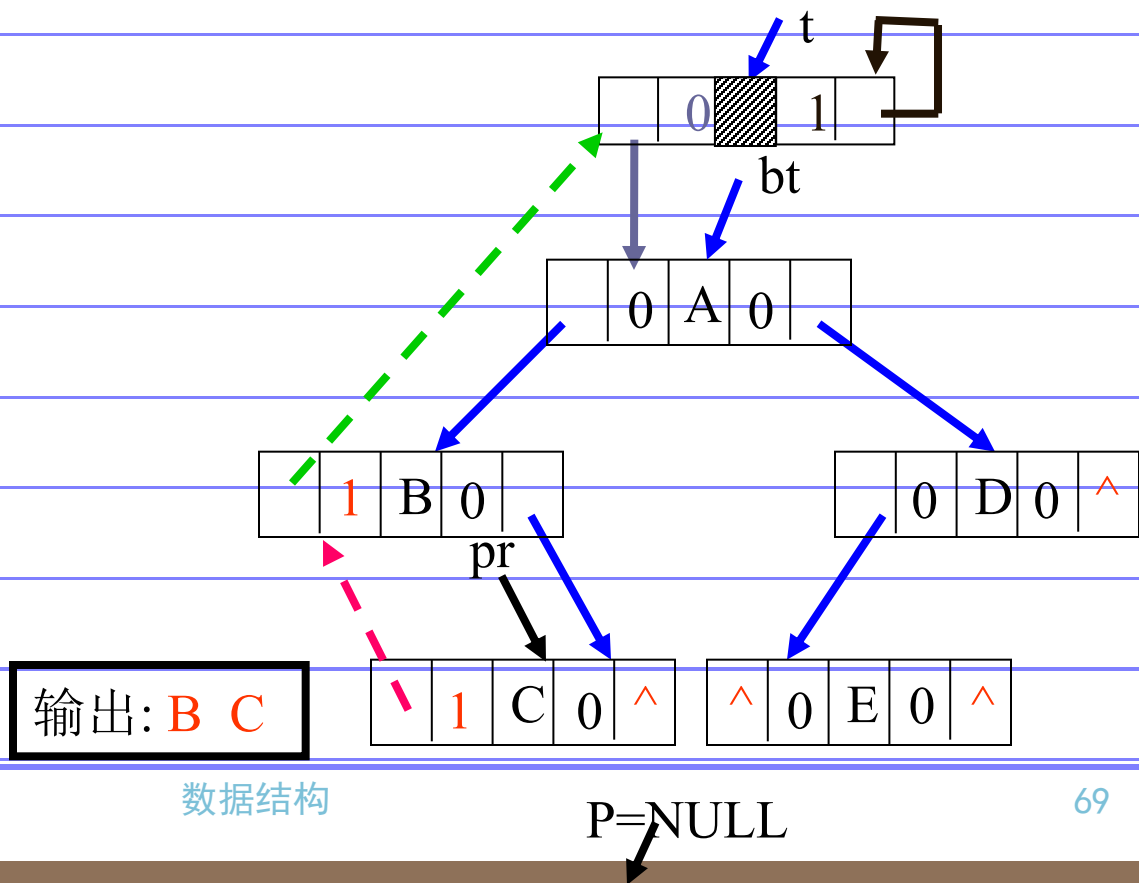
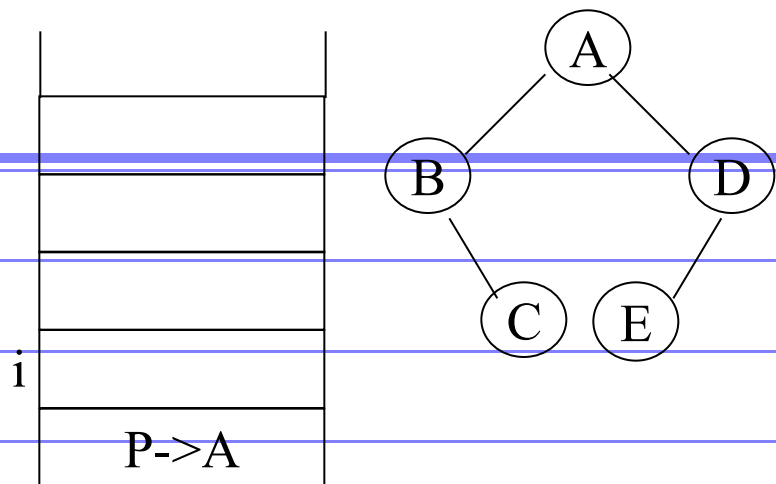
✧ 算法

✧ 按中序线索化二叉树



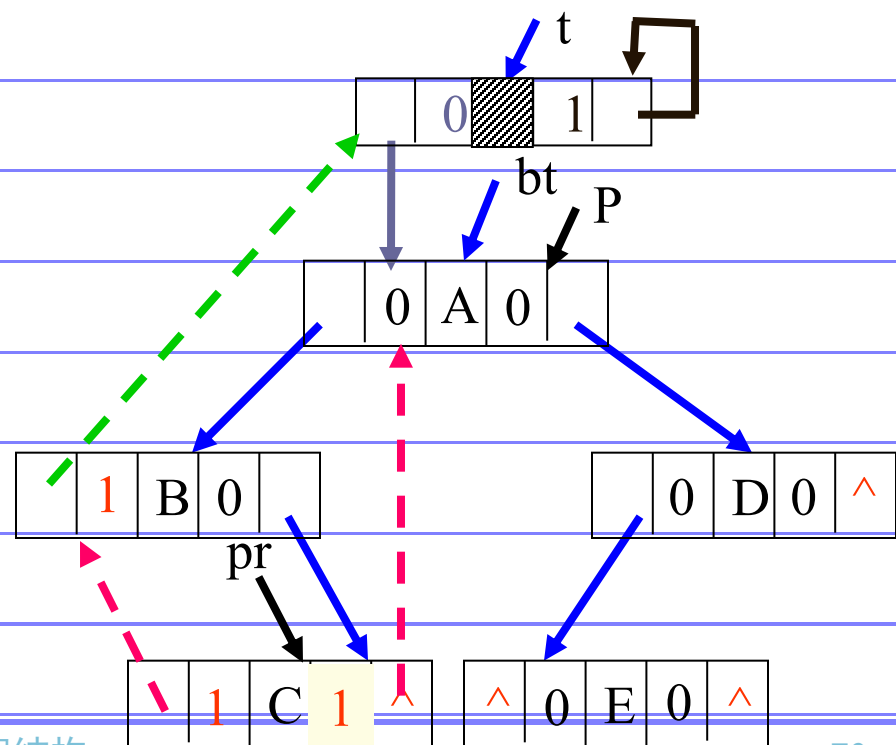
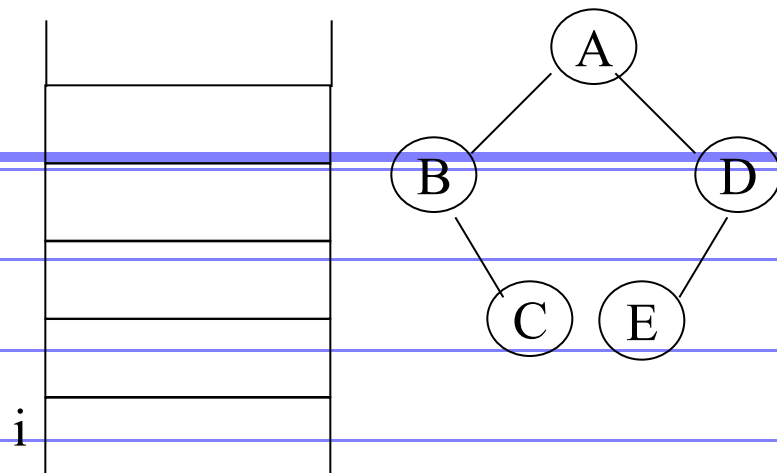
✧ 算法

✧ 按中序线索化二叉树



✧ 算法

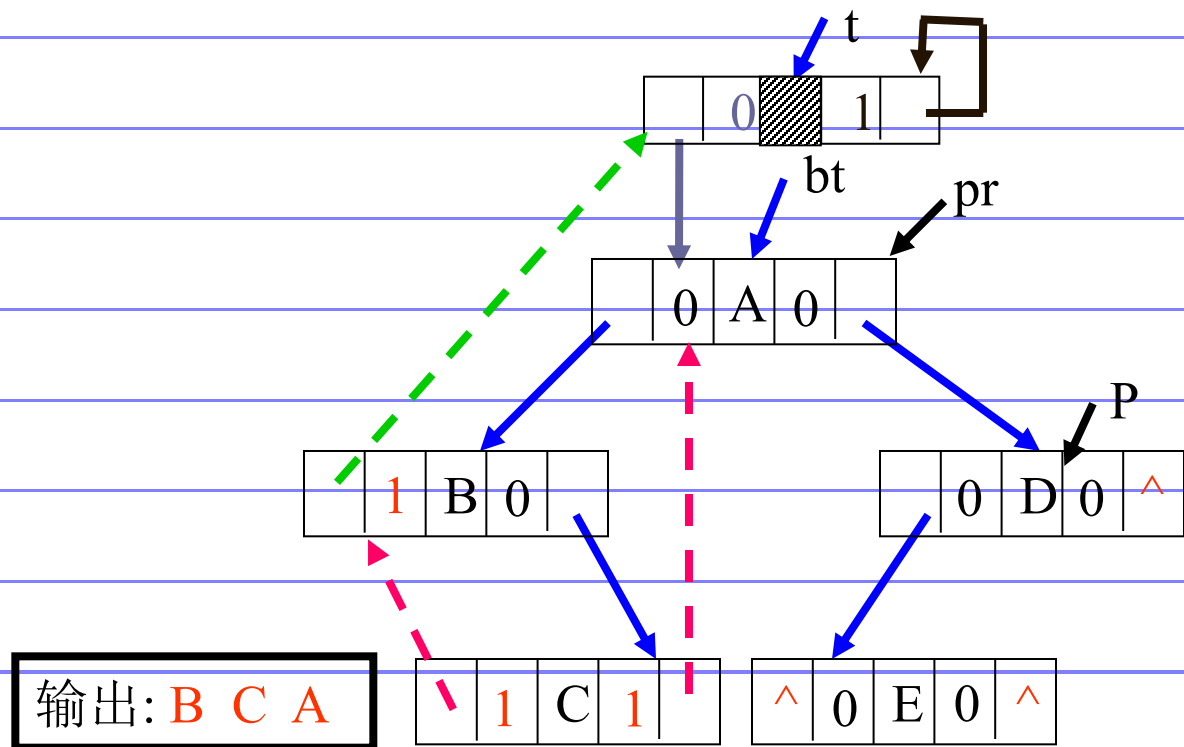
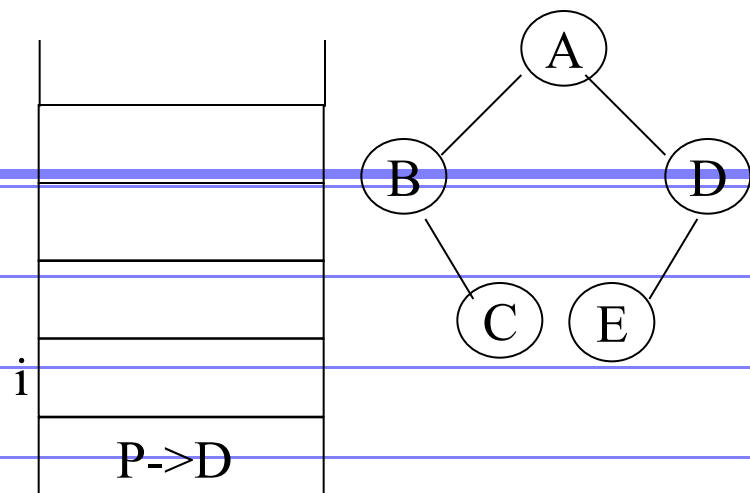
✧按中序线索化二叉树



输出: B C A

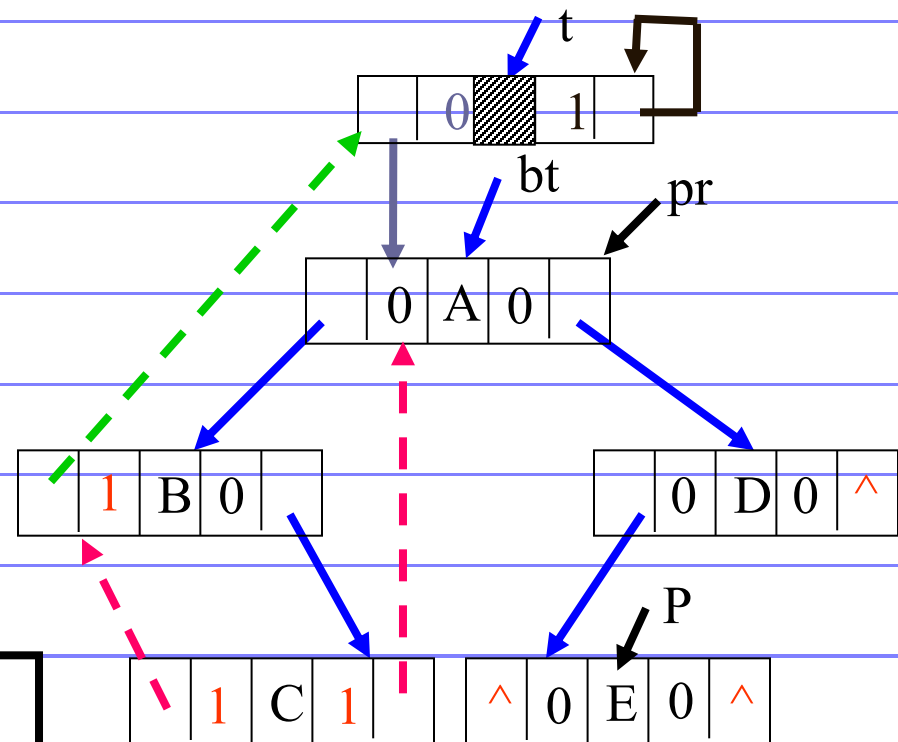
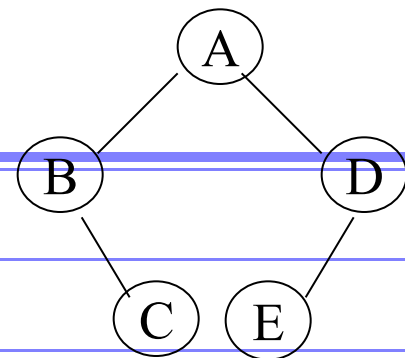
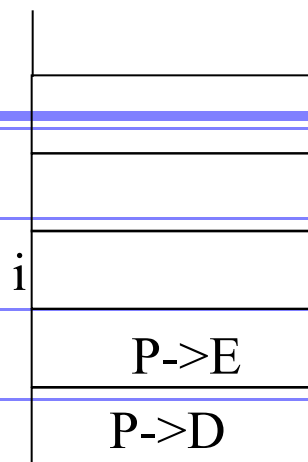
✧ 算法

✧ 按中序线索化二叉树



✧ 算法

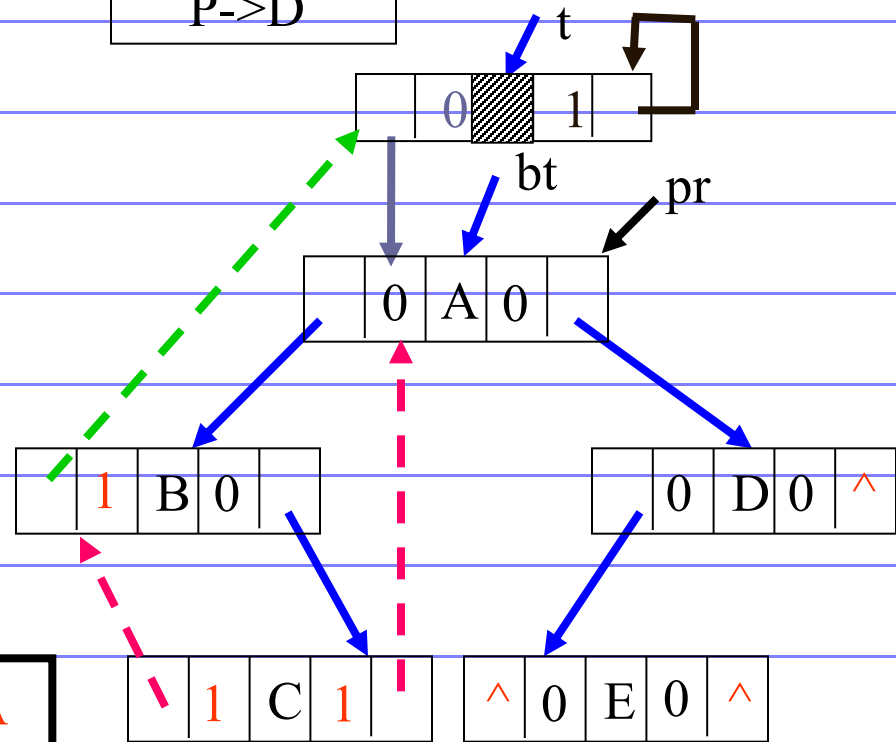
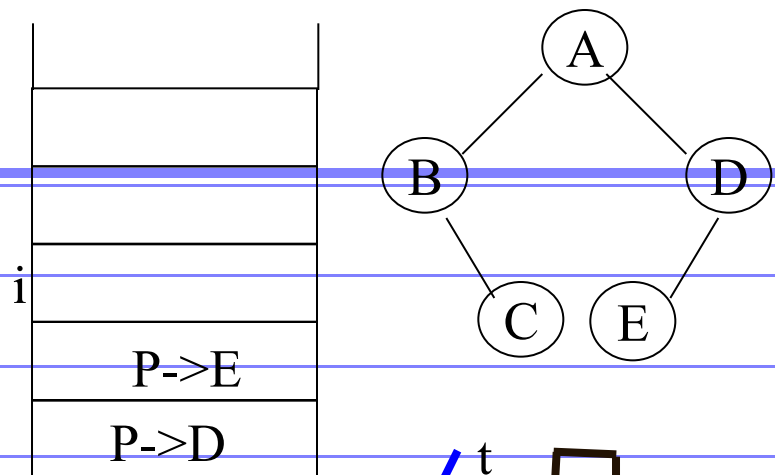
✧ 按中序线索化二叉树



输出: B C A

✧ 算法

✧ 按中序线索化二叉树

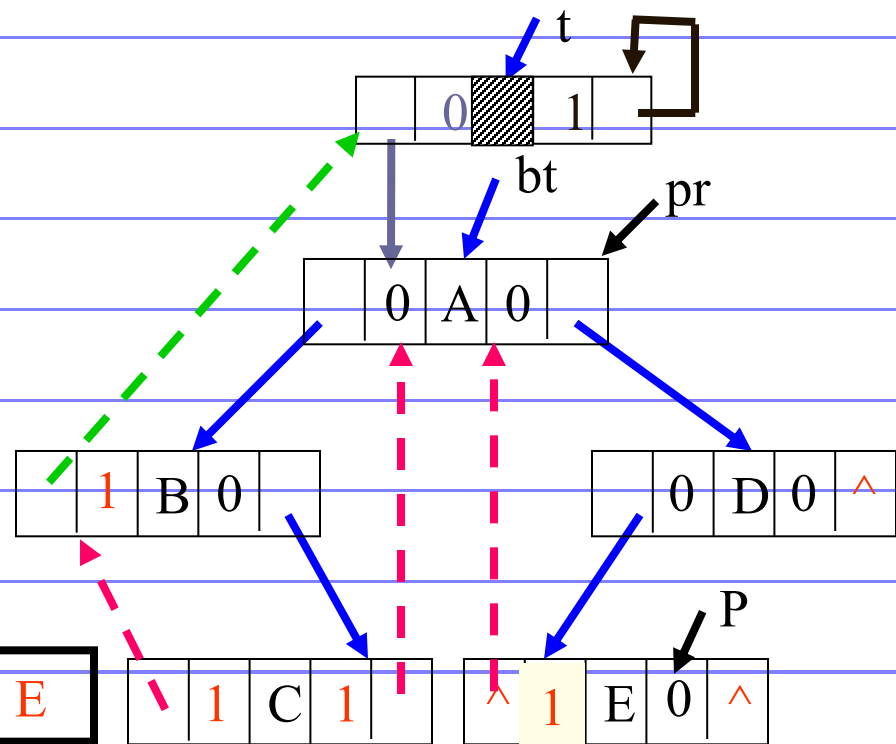
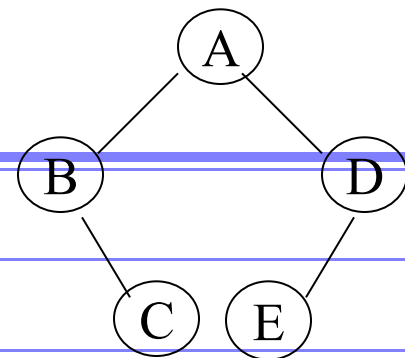
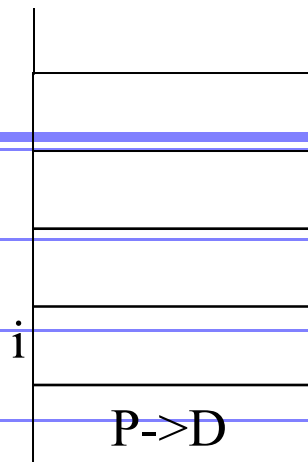


输出: B C A

P=NULL

✧ 算法

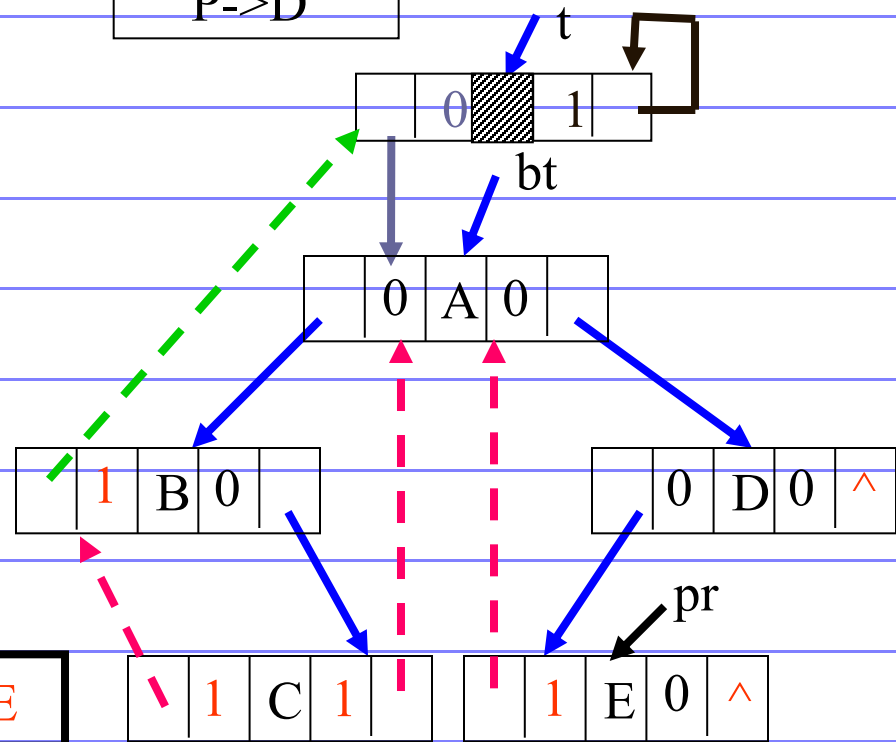
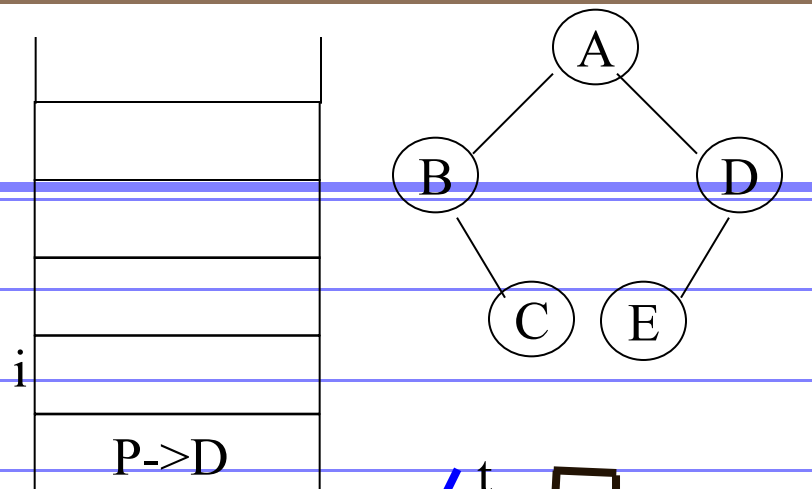
✧ 按中序线索化二叉树



输出: B C A E

✧ 算法

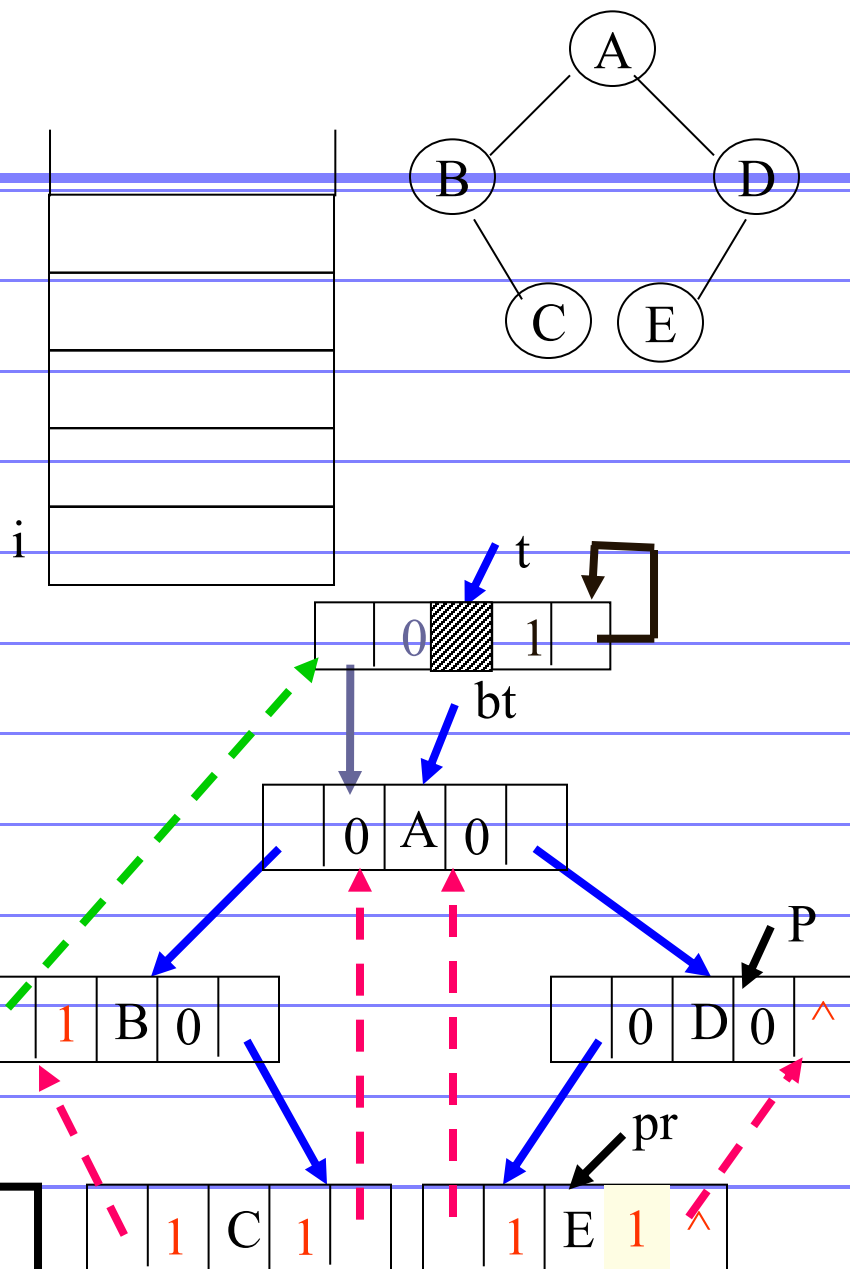
✧ 按中序线索化二叉树



输出: B C A E

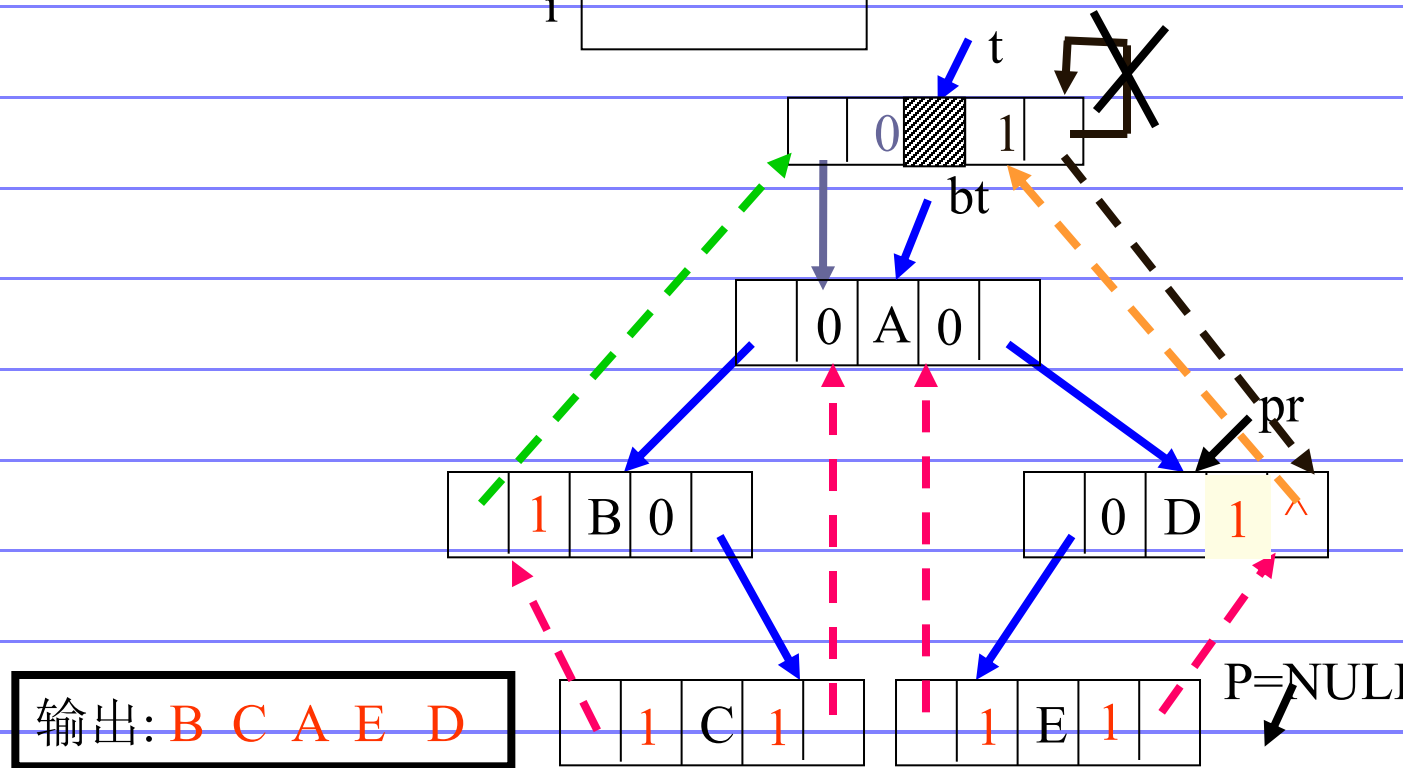
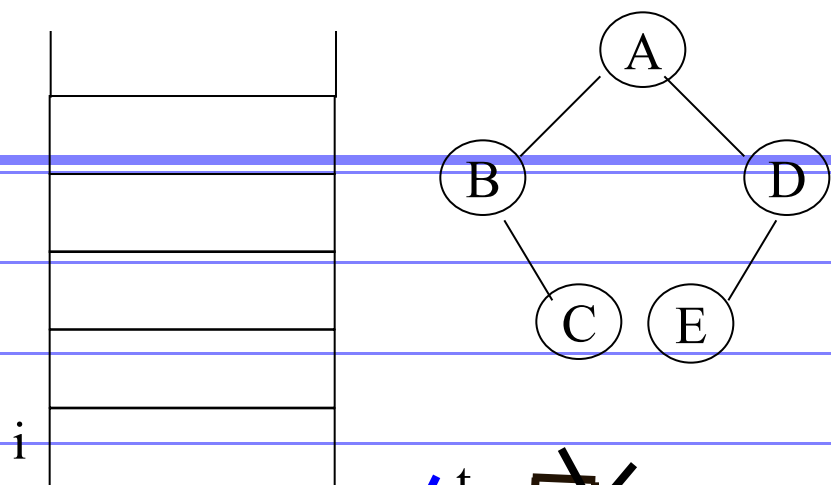
✧ 算法

✧ 按中序线索化二叉树



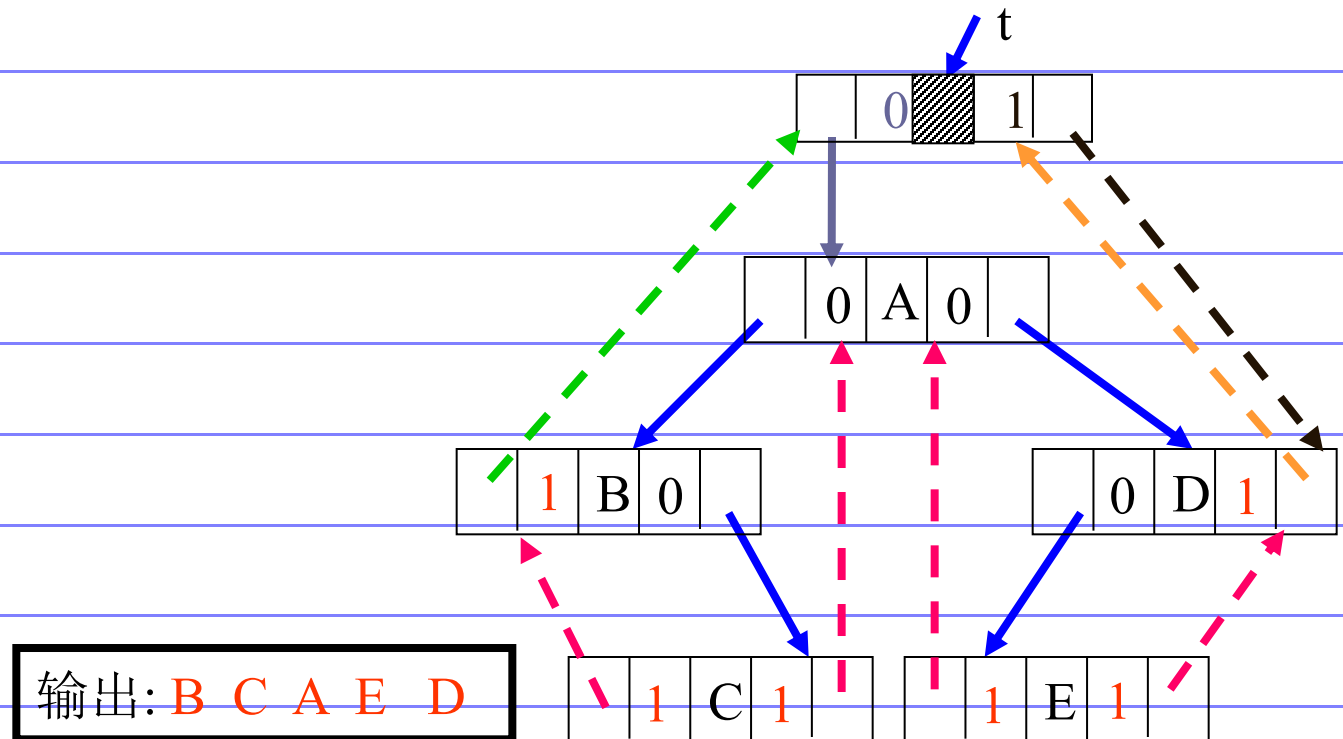
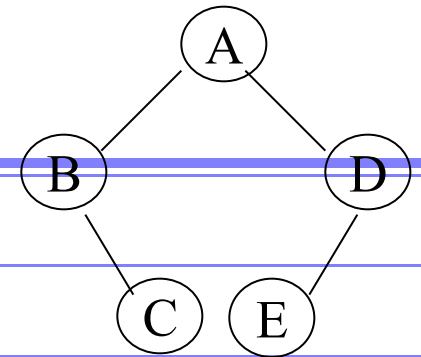
✧ 算法

✧ 按中序线索化二叉树



✧ 算法

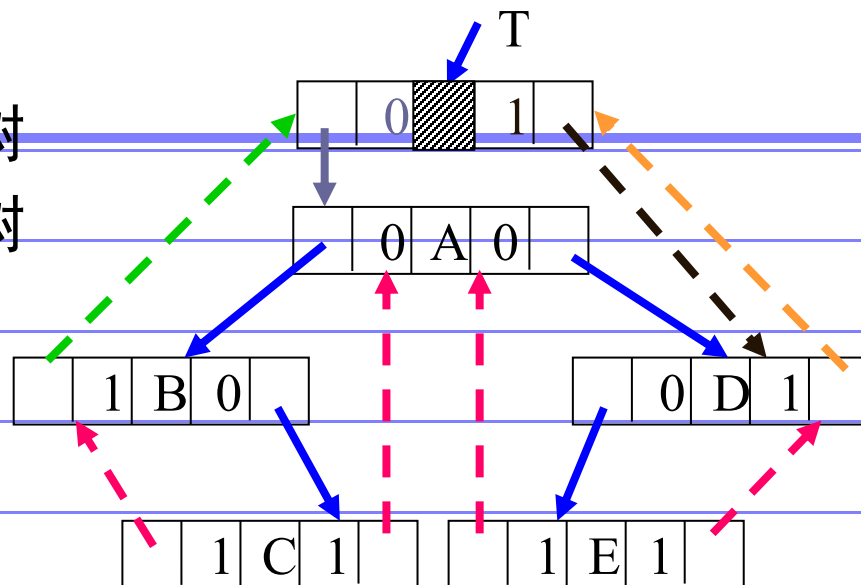
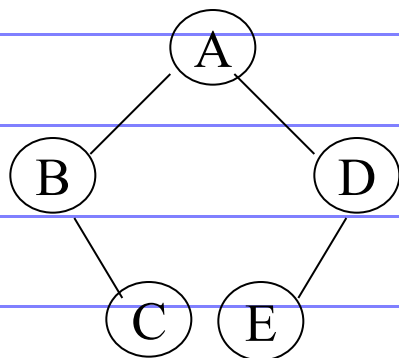
✧ 按中序线索化二叉树



◇ 算法

□ 按中序线索化二叉树

□ 遍历中序线索二叉树



在中序线索二叉树中找结点后继的方法:

- (1) 若 $rt=1$, 则 rc 域直接指向其后继
- (2) 若 $rt=0$, 则结点的后继应是其右子树的左链尾 ($lt=1$)的结点

在中序线索二叉树中找结点前驱的方法:

- (1) 若 $lt=1$, 则 lc 域直接指向其前驱
- (2) 若 $lt=0$, 则结点的前驱应是其左子树的右链尾 ($rt=1$)的结点

中序线索二叉树的中序遍历

```
void inorder_Thread_bt(BiThrNode *T)
{
    BiThrNode *p;
    if (T!=NULL)
    {
        p=T;
        while (p->Ltag==0 )
            p=p->Lchild; /* 寻找最左的结点 */
        while (p!=NULL)
        {
            visit(p->data);
            if (p->Rtag==1)
                p=p->Rchild; /* 通过右线索找到后继 */
            else /* 否则，右子树的最左结点为后继 */
            {
                p=p->Rchild;
                while (p->Ltag==0 ) p=p->Lchild;
            }
        }
    }
}
```

第三节 树的存储结构

□ 双亲表示法

■ 实现:定义结构数组存放树的结点,每个结点含两个域:

◇ 数据域:存放结点本身信息

◇ 双亲域:指示本结点的双亲结点在数组中位置

■ 特点:找双亲容易, 找孩子难

□ 孩子表示法

■ 多重链表:每个结点有多个指针域,分别指向其子树的根

◇ 结点同构: 结点的指针个数相等, 为树的度 D

◇ 结点不同构: 结点指针个数不等, 为该结点的度 d

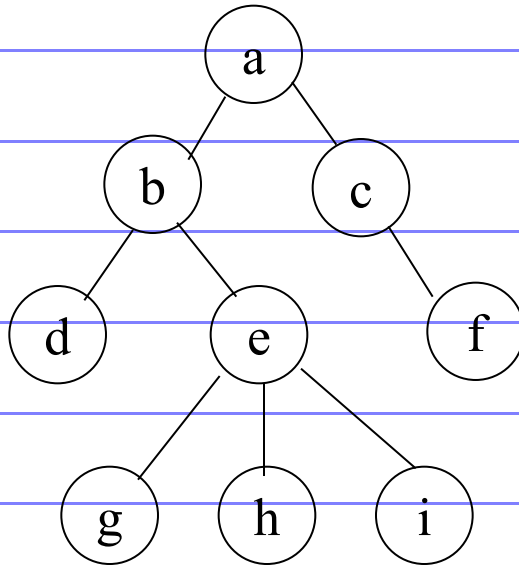
■ 孩子链表: 每个结点的孩子结点用单链表存储, 再用含 n 个元素的结构数组指向每个孩子链表

□ 父子表示法

树的双亲表示法

```
#define MAX_TREE_SIZE 100
typedef struct PTNode {    //节点结构
    TElemType data;
    int parent;    //双亲位置域
} PTNode;
typedef struct {    //树结构
    PTNode nodes[MAX_TREE_SIZE];
    int r, n;    //根的位置和节点数
} PTree;
```

双亲表示法例



	data	parent
0	0	9
1	a	0
2	b	1
3	c	1
4	d	2
5	e	2
6	f	3
7	g	5
8	h	5
9	i	5

0号单元不用或
存结点个数

如何找孩子结点

树的孩子表示法

```
typedef struct CTNode { //孩子节点
```

```
    int child;
```

```
    struct CTNode *next;
```

```
} *ChildPtr;
```

```
typedef struct {
```

```
    TElemType data;
```

```
    ChildPtr firstchild; //孩子链表头指针
```

```
} CTBox;
```

```
typedef struct {
```

```
    CTBox nodes[MAX_TREE_SIZE];
```

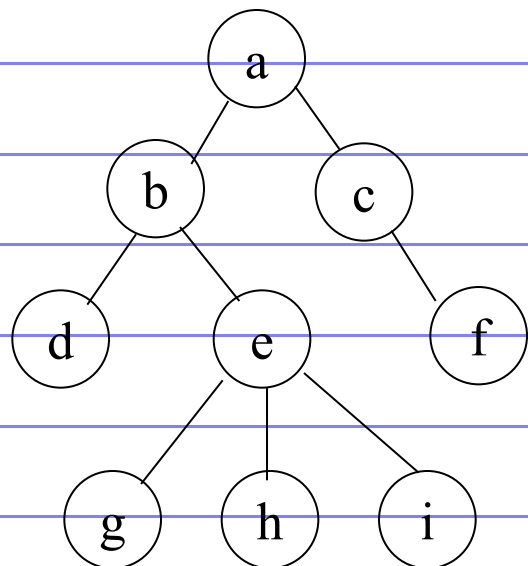
```
    int n, r;
```

```
} CTree;
```

data	child1	child2		childD
------	--------	--------	-------	--------

data	degree	child1	child2		childd
------	--------	--------	--------	-------	--------

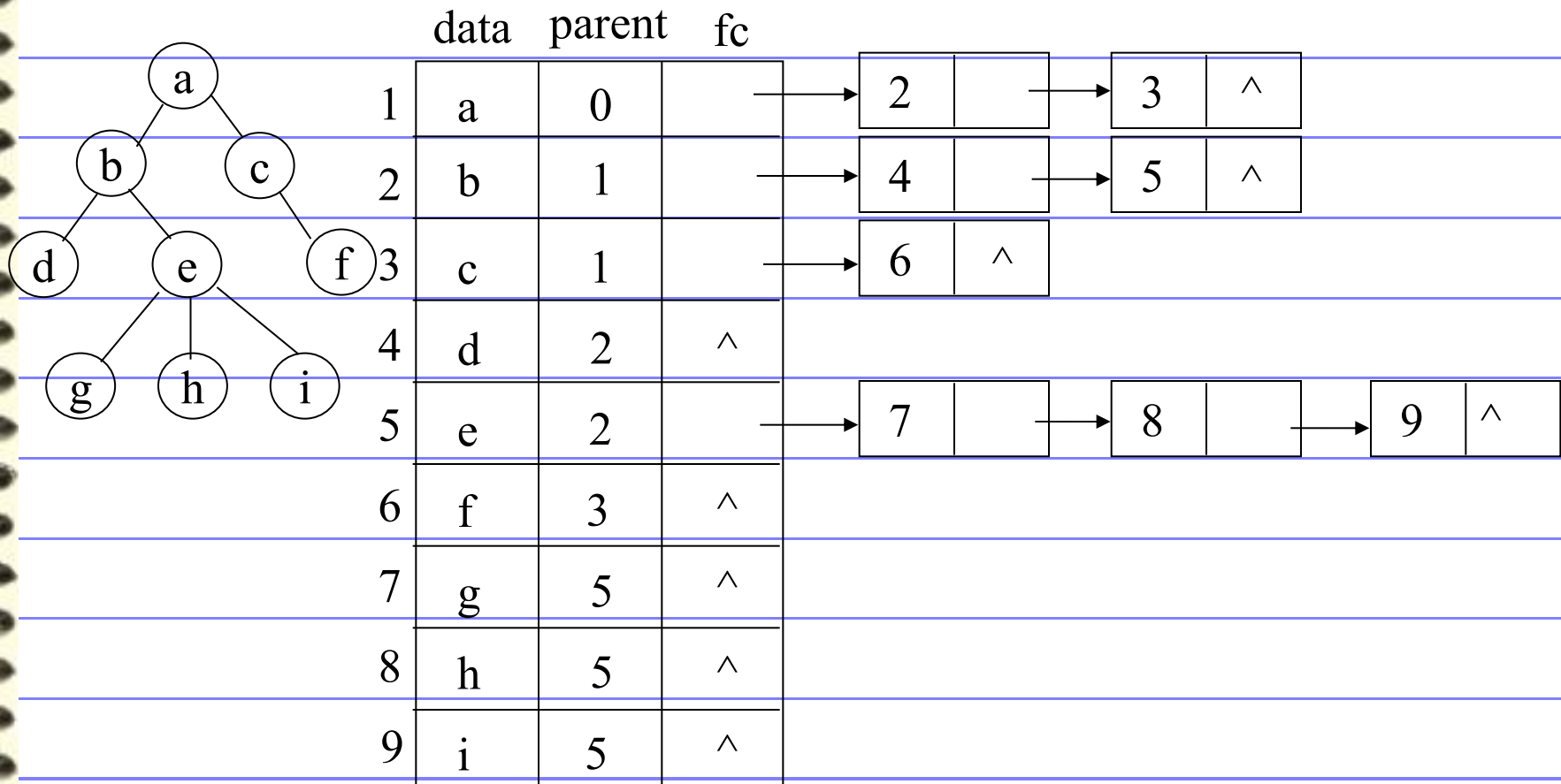
孩子表示法例



	data	fc
0		
1	a	→ 2 → 3 ^
2	b	→ 4 → 5 ^
3	c	→ 6 ^
4	d	^
5	e	→ 7 → 8 → 9 ^
6	f	^
7	g	^
8	h	^
9	i	^

如何找双亲结点

带双亲的孩子链表



树的父子兄弟表示法

```
typedef struct CSNode
{
    ElemType data;
    struct CSNode *firstchild, *nextsibling;
} CSNode, *CSTree;
```

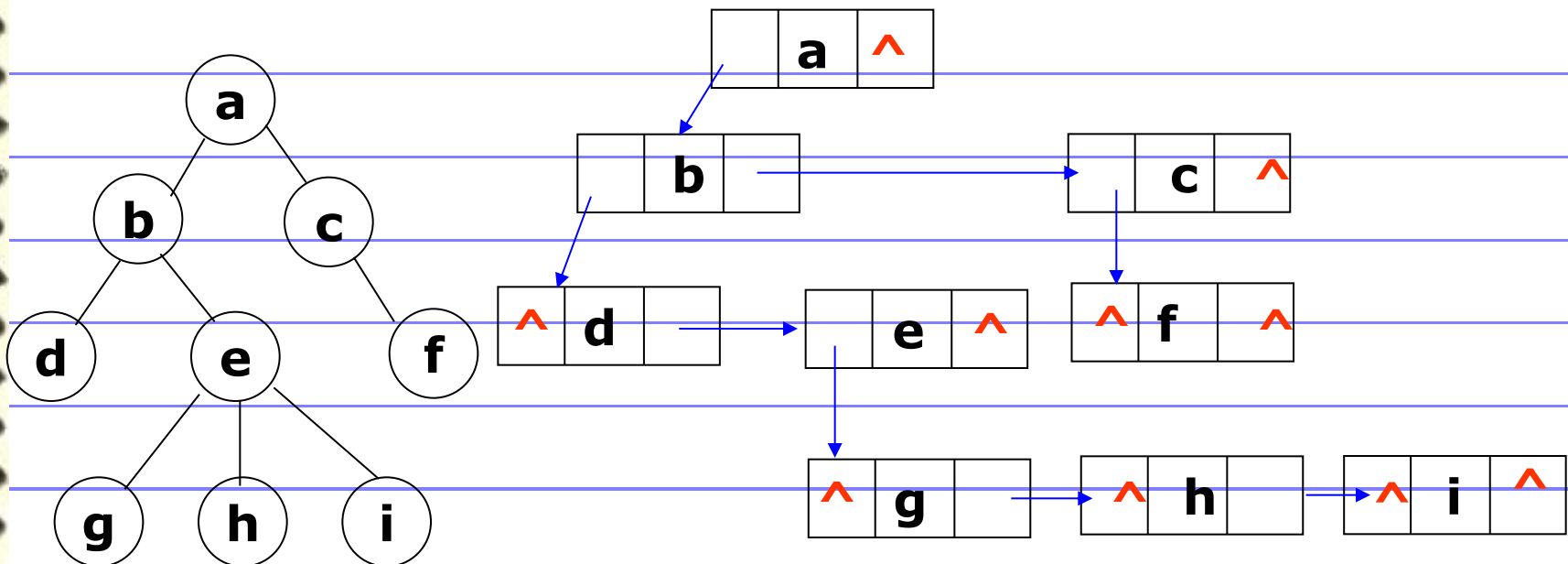
树的父子兄弟表示法

□ 实现：用二叉链表作树的存储结构，链表中每个结点的两个指针域分别指向其第一个孩子结点和下一个兄弟结点

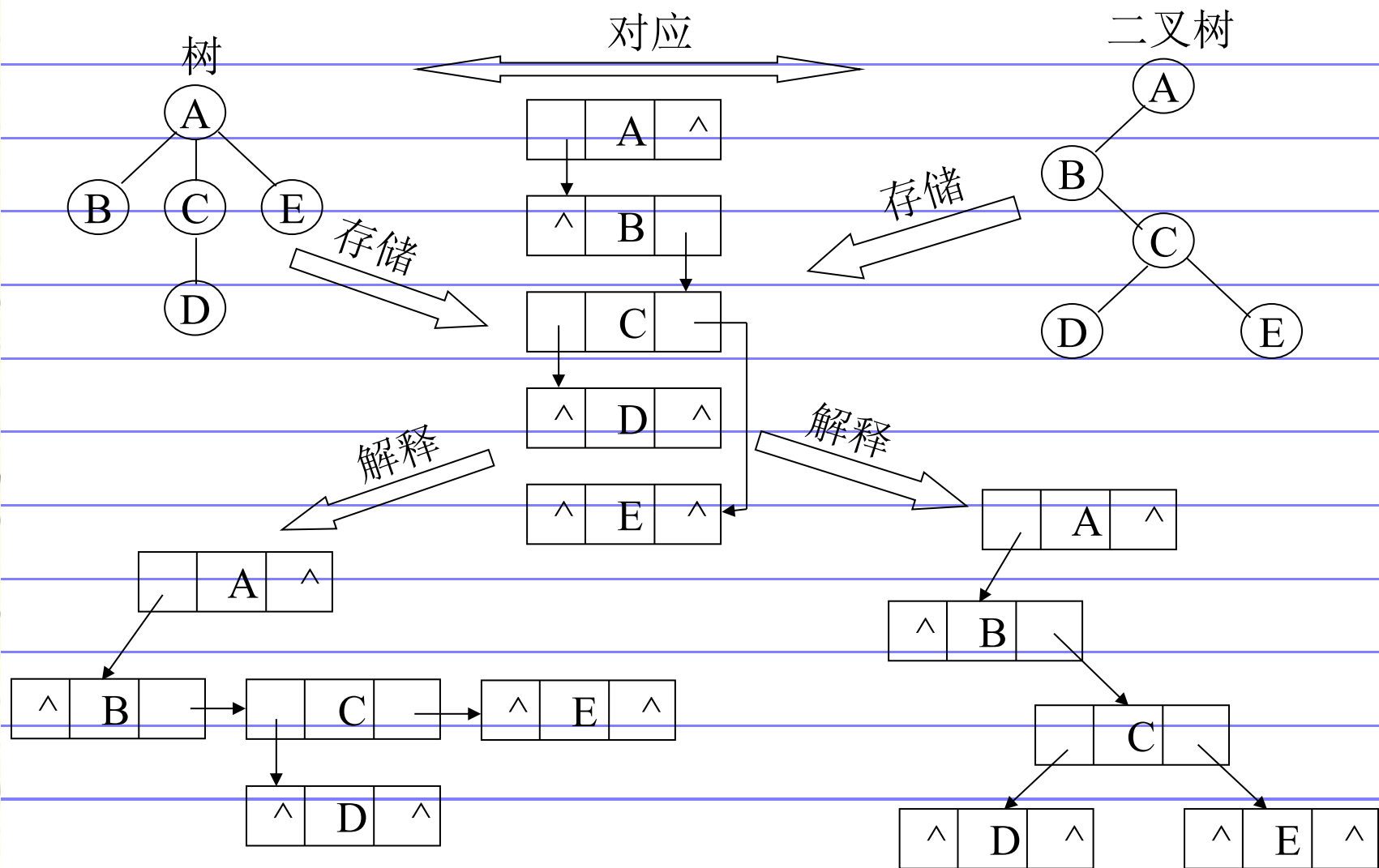
□ 特点

■ 操作容易

■ 破坏了树的层次

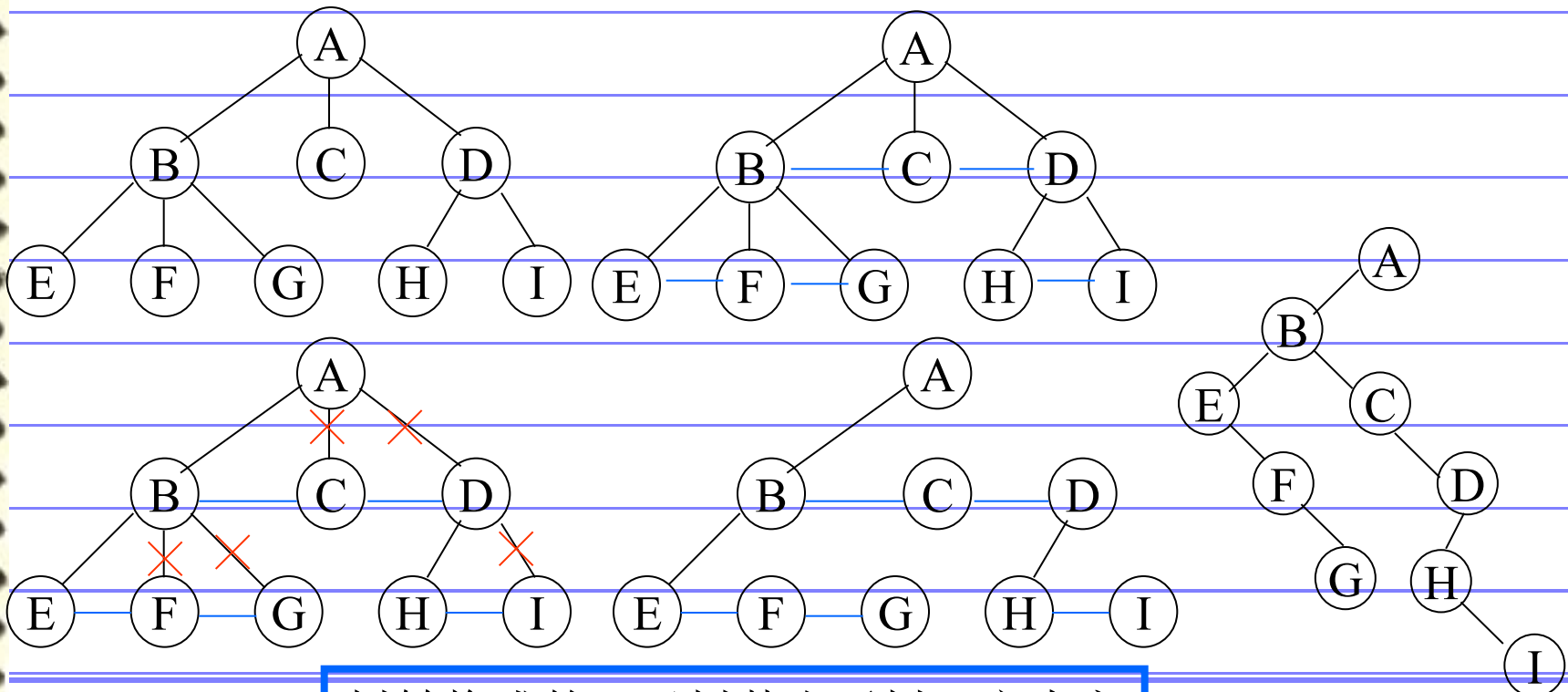


树与二叉树转换



将树转换成二叉树的步骤

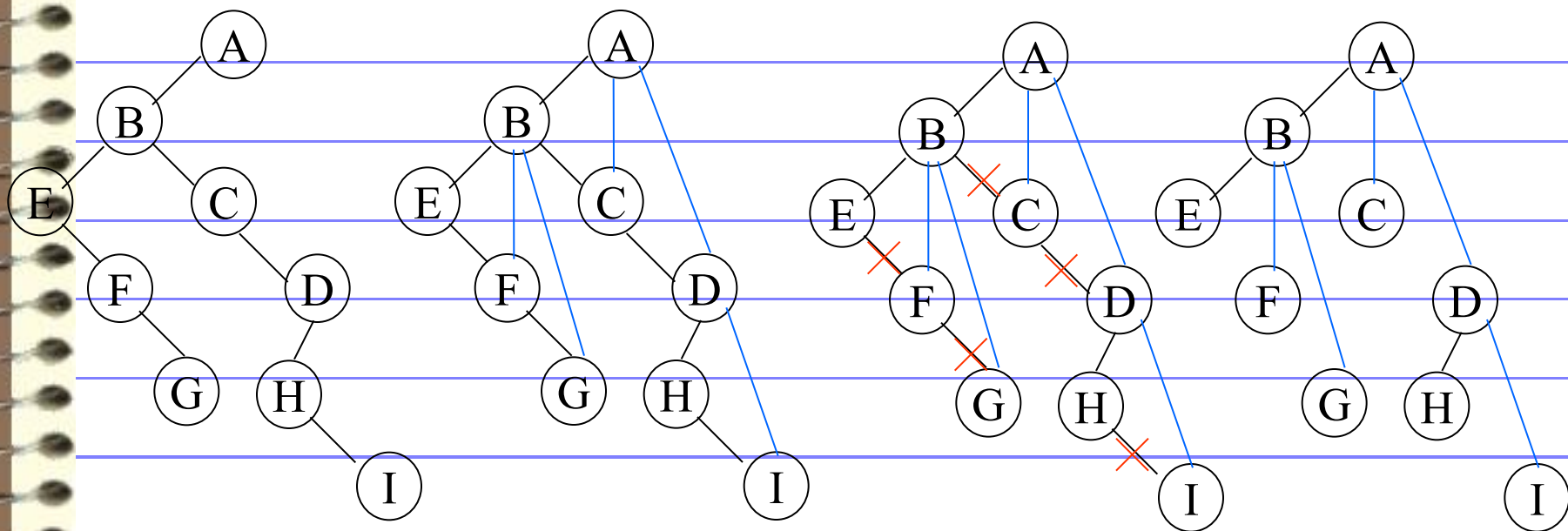
- 加线：在兄弟之间加一连线
- 抹线：对每个结点，除了其左孩子外，去除其与其余孩子之间的关系
- 旋转：以树的根结点为轴心，将整树顺时针转45°



树转换成的二叉树其右子树一定为空

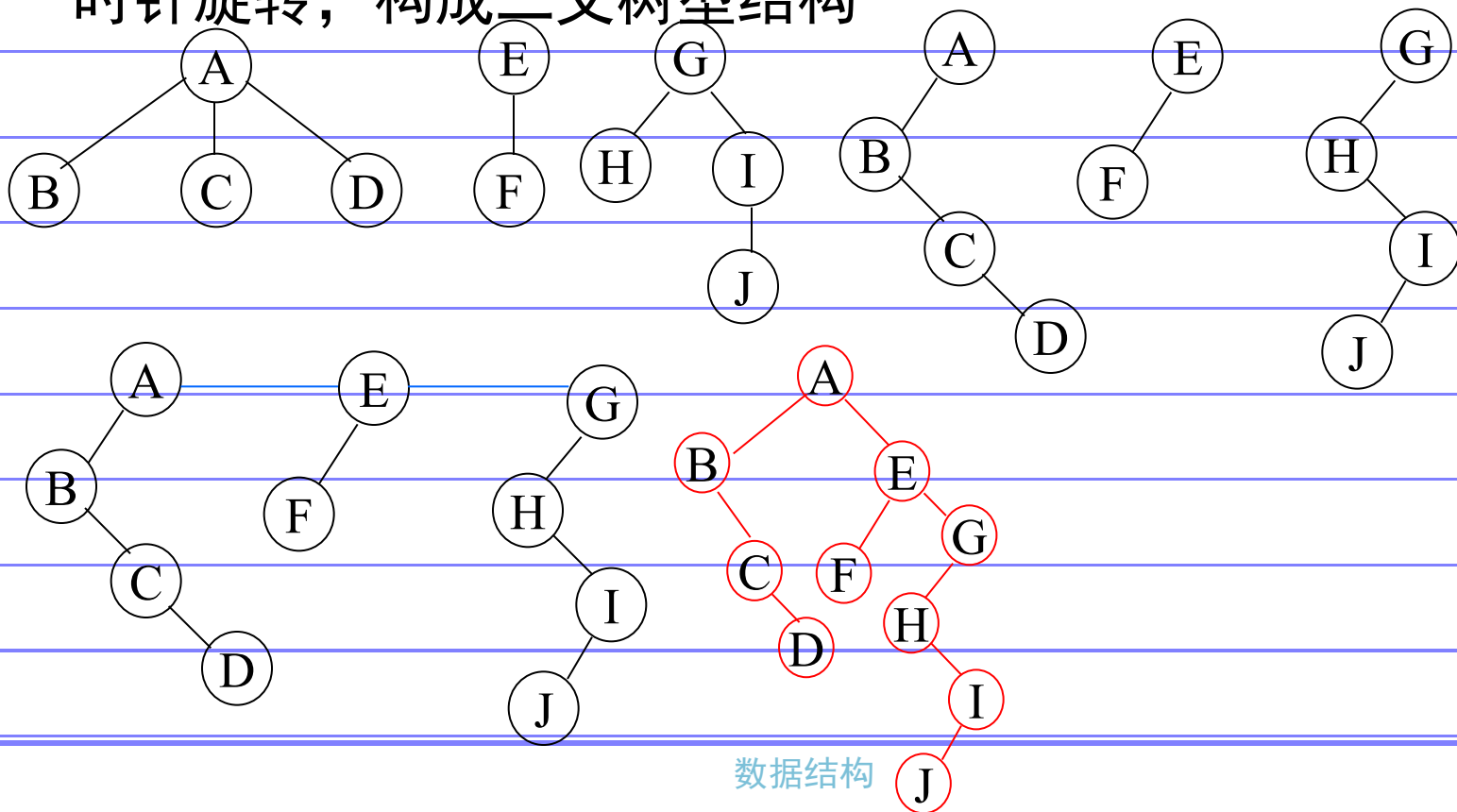
将二叉树转换成树的步骤

- ❑ 加线：若p结点是双亲结点的左孩子，则将p的右孩子，右孩子的右孩子，……沿分支找到的所有右孩子，都与p的双亲用线连起来
- ❑ 抹线：抹掉原二叉树中双亲与右孩子之间的连线
- ❑ 调整：将结点按层次排列，形成树结构



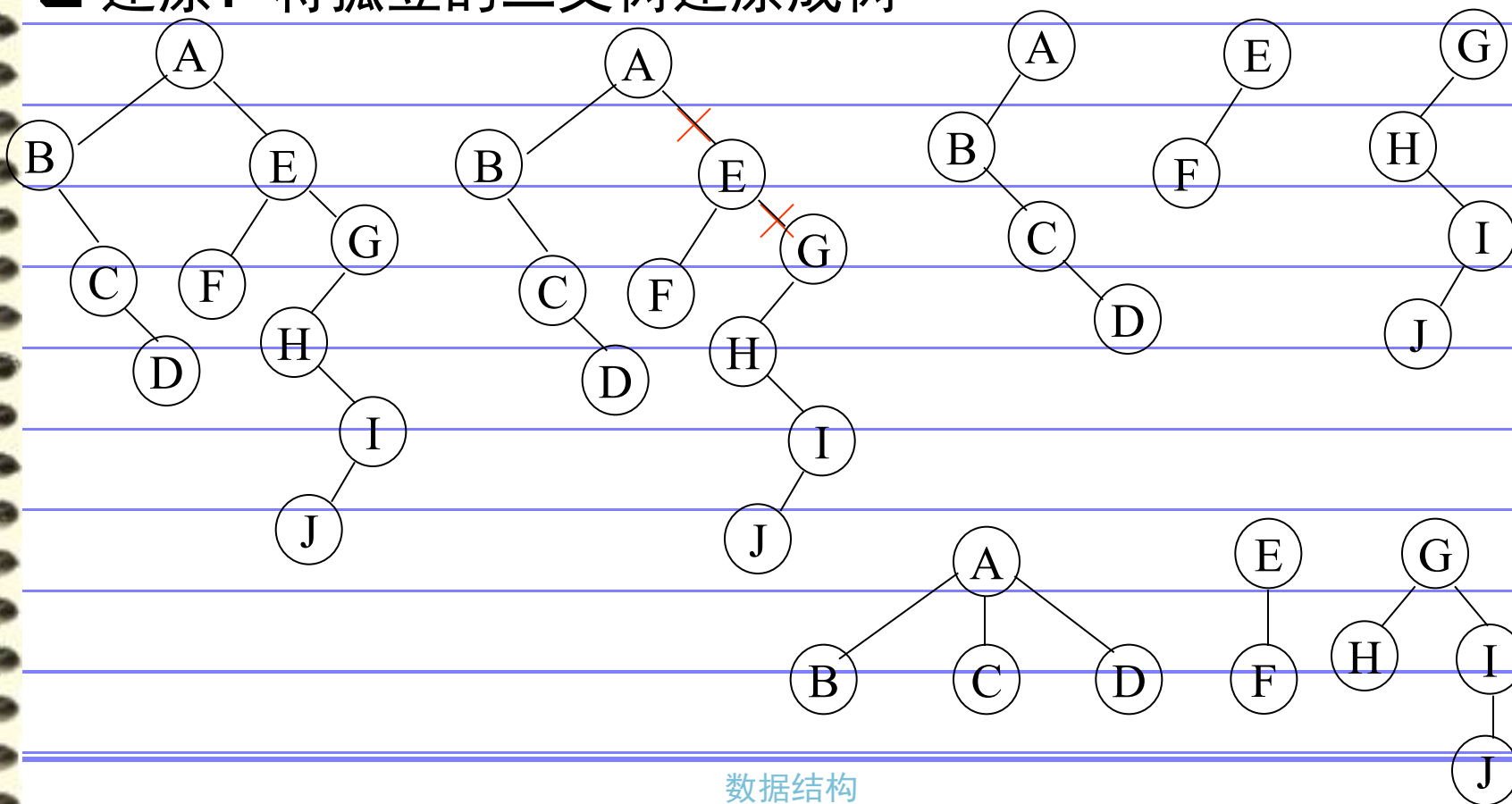
将森林转换成二叉树的步骤

- ❑ 将各棵树分别转换成二叉树
- ❑ 将每棵树的根结点用线相连
- ❑ 以第一棵树根结点为二叉树的根，再以根结点为轴心，顺时针旋转，构成二叉树型结构



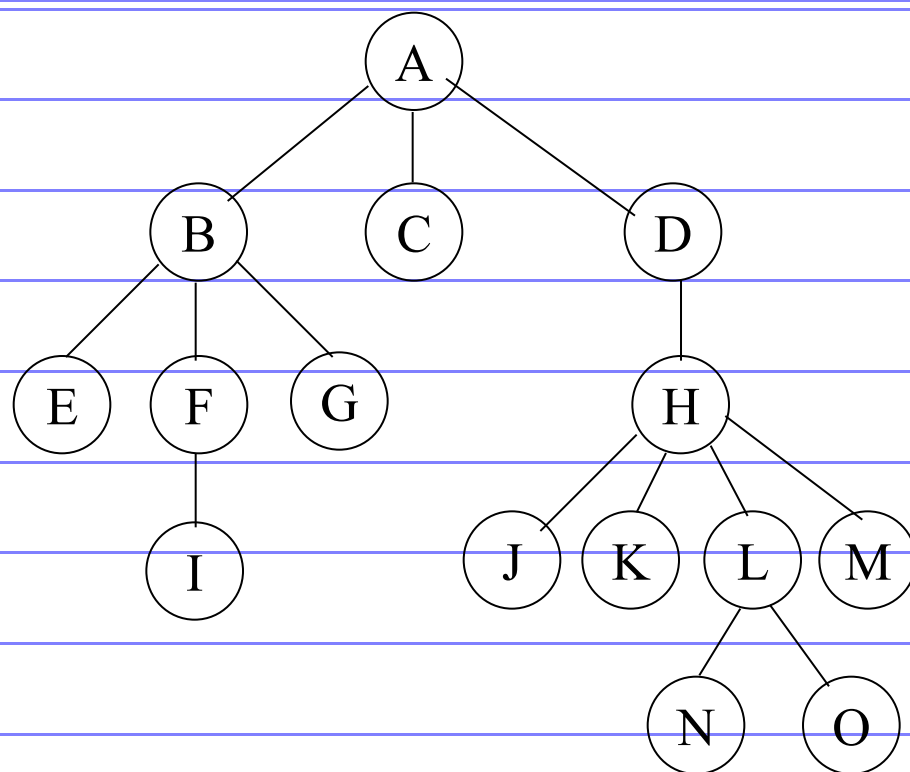
将二叉树转换成森林的步骤

- 抹线：将二叉树中根结点与其右孩子连线，及沿右分支搜索到的所有右孩子间连线全部抹掉,使之变成孤立的二叉树
- 还原：将孤立的二叉树还原成树



第四节 树和二叉树的遍历

- 树的遍历——按一定规律走遍树的各个顶点，且使每一顶点仅被访问一次，即找一个完整而有规律的走法，以得到树中所有结点的一个线性排列
- 常用方法
 - 先根（序）遍历：先访问树的根结点，然后依次先根遍历根的每棵子树
 - 后根（序）遍历：先依次后根遍历每棵子树，然后访问根结点
 - 按层次遍历：先访问第一层上的结点，然后依次遍历第二层，……第n层的结点



先序遍历: **ABEF I GCDHJ KLNOM**

后序遍历: **E I F G B C J K N O L M H D A**

层次遍历: **A B C D E F G H I J K L M N O**

5.5 二叉树的应用

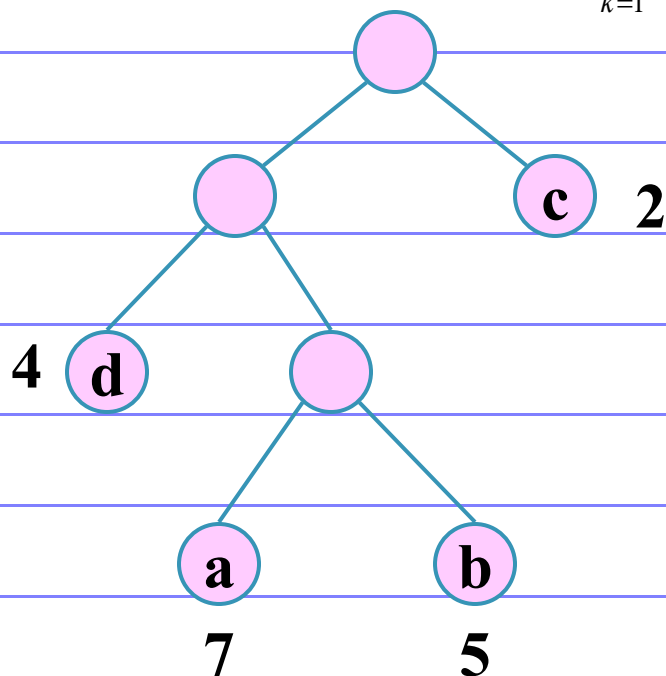
■ 哈夫曼树(Huffman)——带权路径长度最短的树

- 路径：从树中一个结点到另一个结点之间的分支构成这两个结点间的~
- 路径长度：路径上的分支数
- 树的路径长度：从树根到每一个结点的路径长度之和
- 树的带权路径长度：树中所有带权结点的路径长度之和
- Huffman树——设有n个权值 $\{w_1, w_2, \dots, w_n\}$ ，构造一棵有n个叶子结点的二叉树，每个叶子的权值为 w_i ，则wpl最小的二叉树叫~

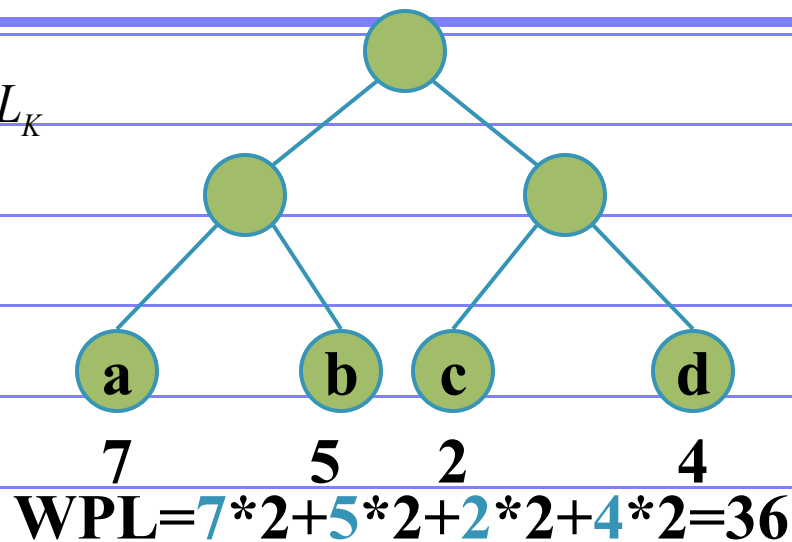
$$WPL = \sum_{k=1}^n W_K L_K$$

例 有4个结点，权值分别为7，5，2，4，构造有4个叶子结点的二叉树

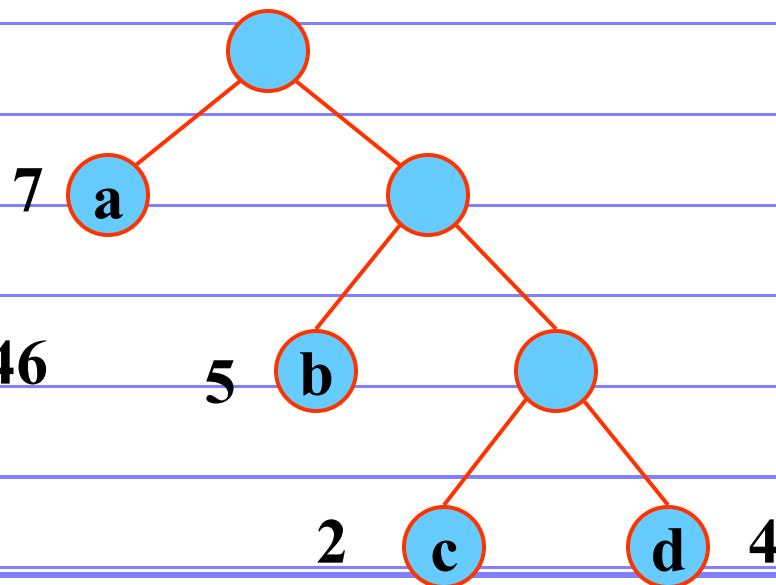
$$WPL = \sum_{k=1}^n W_K L_K$$



$$WPL = 7*3 + 5*3 + 2*1 + 4*2 = 46$$



$$WPL = 7*2 + 5*2 + 2*2 + 4*2 = 36$$



$$WPL = 7*1 + 5*2 + 2*3 + 4*3 = 35$$

◇构造Huffman树的方法——Huffman算法

✧构造Huffman树步骤

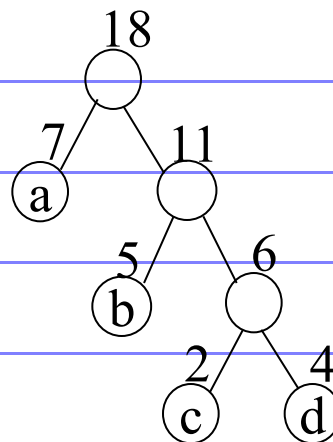
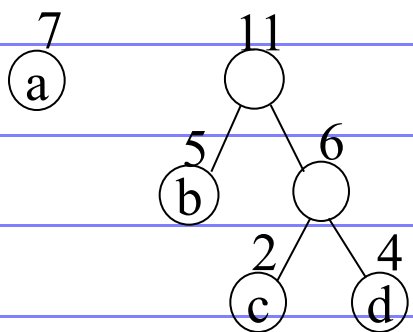
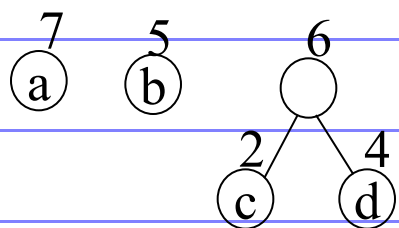
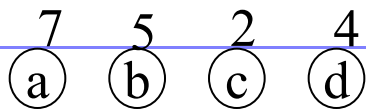
◇根据给定的 n 个权值 $\{w_1, w_2, \dots, w_n\}$ ，构造 n 棵只有根结点的二叉树，令其权值为 w_j

◇在森林中选取两棵根结点权值最小的树作左右子树，构造一棵新的二叉树，置新二叉树根结点权值为其左右子树根结点权值之和

◇在森林中删除这两棵树，同时将新得到的二叉树加入森林中

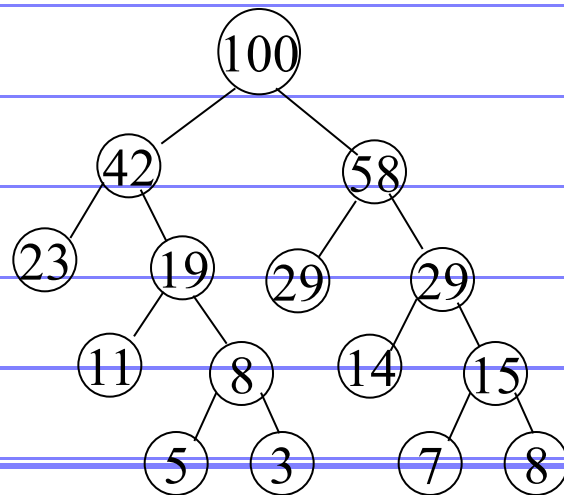
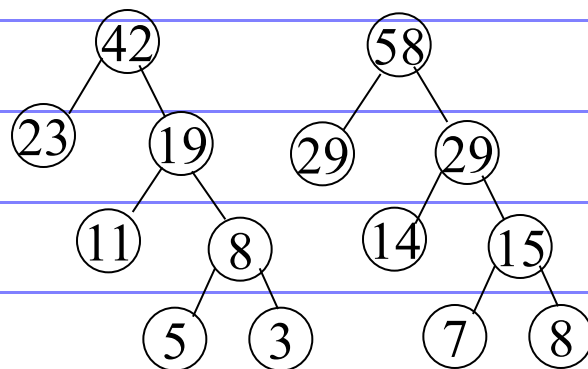
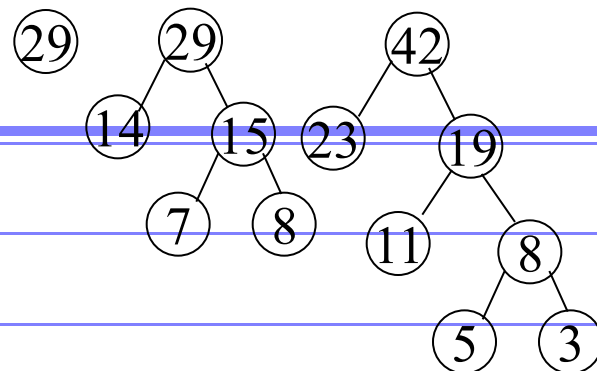
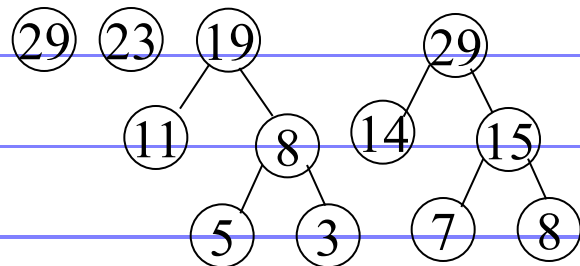
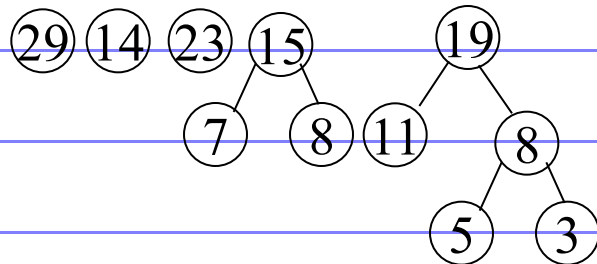
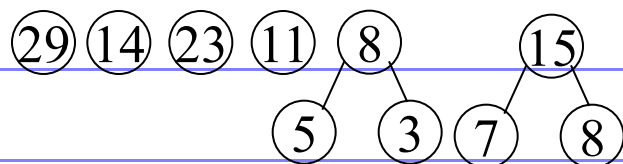
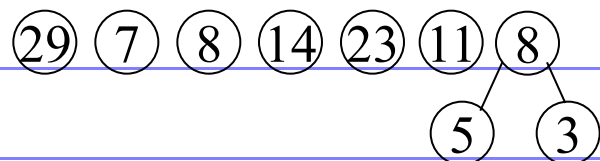
◇重复上述两步，直到只含一棵树为止，这棵树即哈夫曼树

例



例 $w = \{5, 29, 7, 8, 14, 23, 3, 11\}$

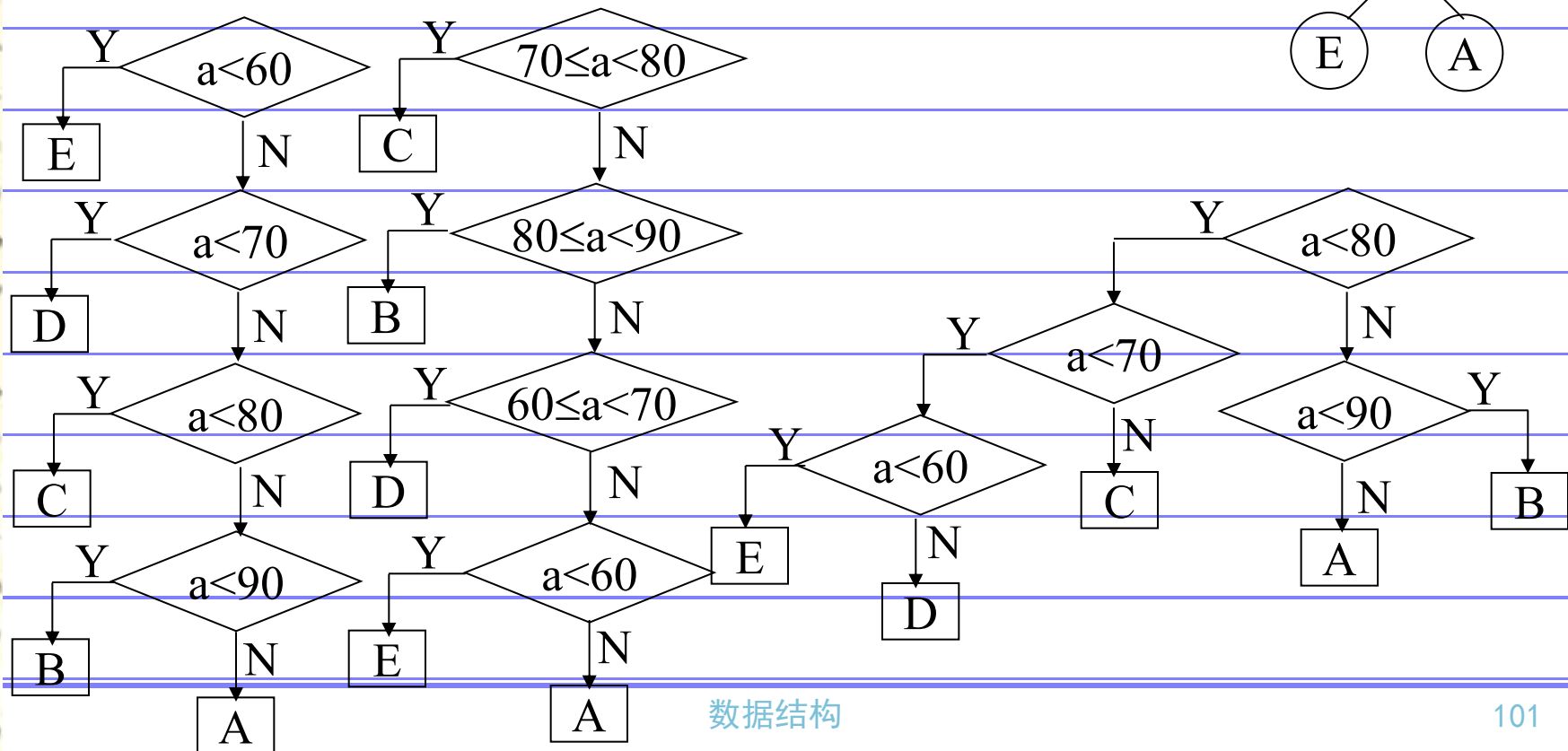
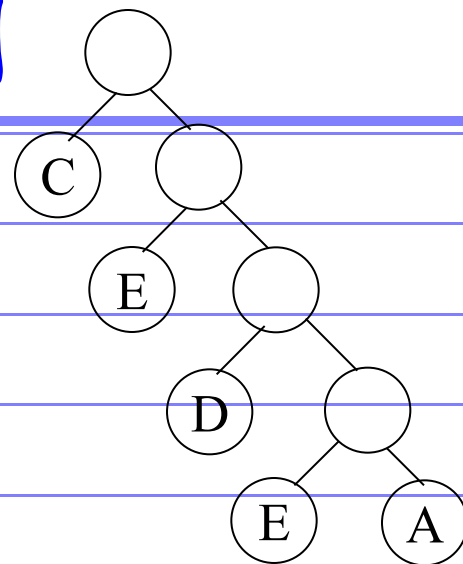
⑤ ②⑨ ⑦ ⑧ ⑭ ⑳ ③ ⑪



Huffman树应用

□最佳判定树

等级	E	D	C	B	A
分数段	0~59	60~69	70~79	80~89	90~100
比例	0.05	0.15	0.40	0.30	0.10



Huffman算法实现

- ✧ 一棵有 n 个叶子结点的Huffman树有 $2n-1$ 个结点
- ✧ 采用顺序存储结构——一维结构数组
- ✧ 结点类型定义

```
typedef struct {  
    unsigned int weight;  
    unsigned int parent, lchild, rchild;  
} HTNode, *HuffmanTree;
```

	lc	data	rc	pa
1	0	7	0	0
2	0	5	0	0
3	0	2	0	0
4	0	4	0	0
5	0	0	0	0
6	0	0	0	0
7	0	0	0	0

(1)

	lc	data	rc	pa
1	0	7	0	0
2	0	5	0	0
3	0	2	0	5
4	0	4	0	5
k→5	3	6	4	0
6	0	0	0	0
7	0	0	0	0

(2)

x1=3,x2=4

	lc	data	rc	pa
1	0	7	0	0
2	0	5	0	6
3	0	2	0	5
4	0	4	0	5
5	3	6	4	6
k→6	2	11	5	0
7	0	0	0	0

(3)

x1=2,x2=5

m1=5,m2=6

m1=2,m2=4

	lc	data	rc	pa
1	0	7	0	7
2	0	5	0	6
3	0	2	0	5
4	0	4	0	5
5	3	6	4	6
6	2	11	5	7
k→7	1	18	6	0

(4)

x1=1,x2=6

m1=7,m2=11

```

void Create_Huffman(unsigned n, HTNode HT[ ], unsigned m) // 创建一棵叶子结点数为n的Huffman树
{ unsigned int w ; int k , j ;
  for (k=1 ; k<m ; k++)
  { if (k<=n) { printf("\n Please Input Weight : w=?"); scanf("%d", &w) ;HT[k].weight=w ; }
    else HT[k].weight=0; /* 非叶子结点没有权值 */
    HT[k].Parent=HT[k].Lchild=HT[k].Rchild=0 ;
  } /* 初始化向量HT */
  for (k=n+1; k<m ; k++)
  { unsigned w1=32767 , w2=w1 ;/* w1 , w2分别保存权值最小的两个权值 */
    int p1=0 , p2=0 ; /* p1 , p2保存两个最小权值的下标 */
    for (j=1 ; j<=k-1 ; j++)
    { if (HT[j].Parent==0) /* 尚未合并 */
      { if (HT[j].Weight<w1) { w2=w1 ; p2=p1 ; w1=HT[j].Weight ; p1=j ; }
        else if (HT[j].Weight<w2)
        { w2=HT[j].Weight ; p2=j ; }
      } /* 找到权值最小的两个值及其下标 */
    }
    HT[k].Lchild=p1 ; HT[k].Rchild=p2 ;
    HT[k].weight=w1+w2 ;
    HT[p1].Parent=k ; HT[p2].Parent=k ;
  }
}

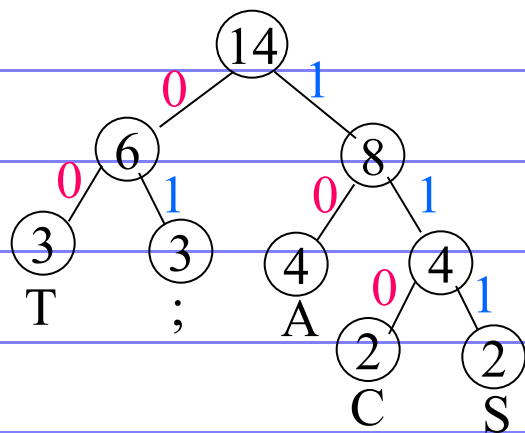
```

说明：生成Huffman树后，树的根结点的下标是 $2n-1$ ，即 $m-1$ 。

✧ Huffman编码：数据通信用的二进制编码

✧思想：根据字符出现频率编码，使电文总长最短

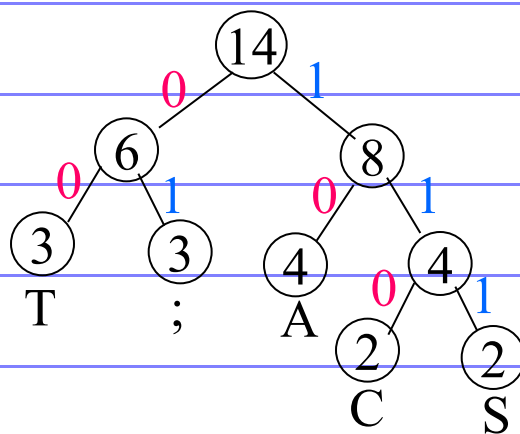
✧编码：根据字符出现频率构造Huffman树，然后将树中结点引向其左孩子的分支标“0”，引向其右孩子的分支标“1”；每个字符的编码即为从根到每个叶子的路径上得到的0、1序列



例 要传输的字符集 $D=\{C,A,S,T,;\}$
字符出现频率 $w=\{2,4,2,3,3\}$

T : 00
; : 01
A : 10
C : 110
S : 111

◇译码：从Huffman树根开始，从待译码电文中逐位取码。若编码是“0”，则向左走；若编码是“1”，则向右走，一旦到达叶子结点，则译出一个字符；再重新从根出发，直到电文结束



T : 00
; : 01
A : 10
C : 110
S : 111

例 电文是{CAS;CAT;SAT;AT}
其编码 “11010111011101000011111000011000”
电文为 “1101000”
译文只能是 “CAT”

```

void Huff_coding(unsigned n , Hnode HT[] , unsigned m)
    /* m应为n+1,编码的最大长度n加1 */
{
    int k , sp , fp ;
    char *cd , *HC[m] ;
    cd=(char *)malloc(m*sizeof(char)) ; /* 动态分配求编码的工作空间 */
    cd[n]='\0' /* 编码的结束标志 */
    for (k=1 ; k<n+1 ; k++) /* 逐个求字符的编码 */
    {
        sp=n ; p=k ; fp=HT[k].parent ;
        for ( ; fp!=0 ; p=fp , fp=HT[p].parent) // 从叶子结点到根逆向求编码
            if (HT[fp]. Lchild==p) cd[--sp]='0' ;
            else cd[--sp]='1' ;
        HC[k]=(char *)malloc((n-sp)*sizeof(char)) ; // 为第k个字符分配保存编码的空间
        strcpy(HC[k] , &cd[sp]) ;
    }
    free(cd) ;
}

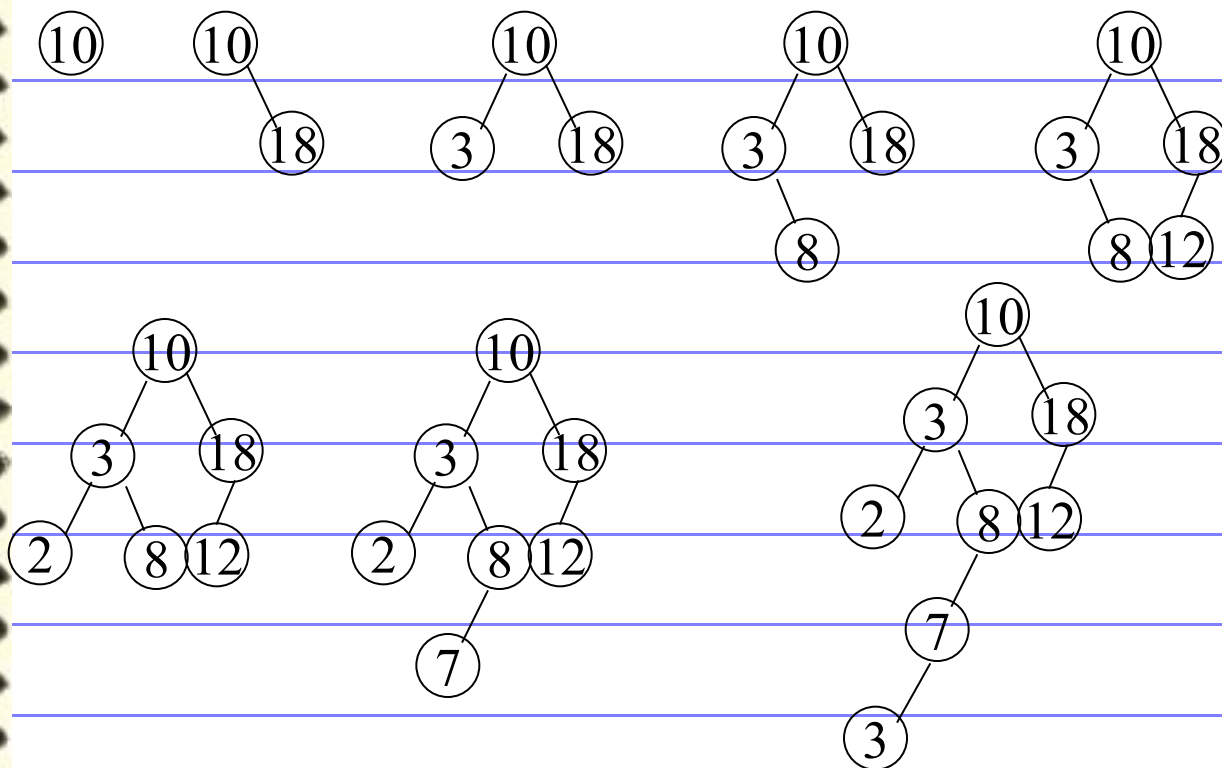
```

二叉排序树

- 定义：二叉排序树或是一棵空树，或是具有下列性质
 - 若它的左子树不空，则左子树上所有结点的值均小于它的根结点的值
 - 若它的右子树不空，则右子树上所有结点的值均大于或等于它的根结点的值
 - 它的左、右子树也分别为二叉排序树
- 二叉排序树的插入
 - 插入原则：若二叉排序树为空，则插入结点应为新的根结点；否则，继续在其左、右子树上查找，直至某个叶子结点的左子树或右子树为空为止，则插入结点应为该叶子结点的左孩子或右孩子
 - 二叉排序树生成：从空树出发，经过一系列的查找、插入操作之后，可生成一棵二叉排序树

□ 插入算法

例 {10, 18, 3, 8, 12, 2, 7, 3}



中序遍历二叉排序树可得到一个关键字的有序序列

◇ 二叉排序树的删除

要删除二叉排序树中的p结点，分三种情况：

✧ p为叶子结点，只需修改p双亲f的指针

f->lchild=NULL

f->rchild=NULL

✧ p只有左子树或右子树

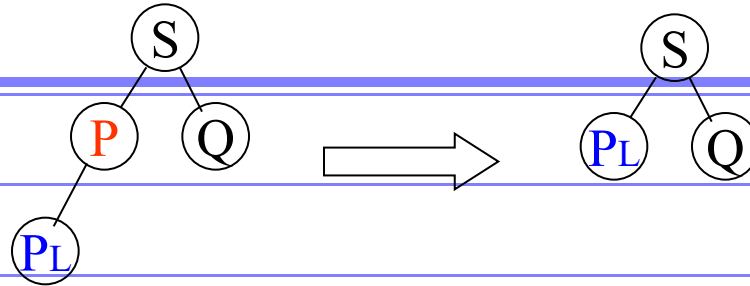
◇ p只有左子树，用p的左孩子代替p (1)(2)

◇ p只有右子树，用p的右孩子代替p (3)(4)

✧ p左、右子树均非空

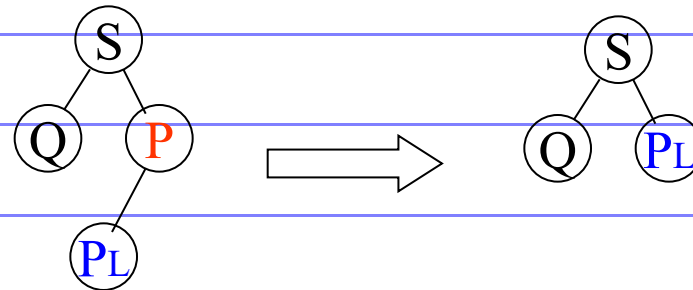
◇ 沿p左子树的根C的右子树分支找到S，S的右子树为空，将S的左子树成为S的双亲Q的右子树，用S取代p (5)

◇ 若C无右子树，用C取代p (6)



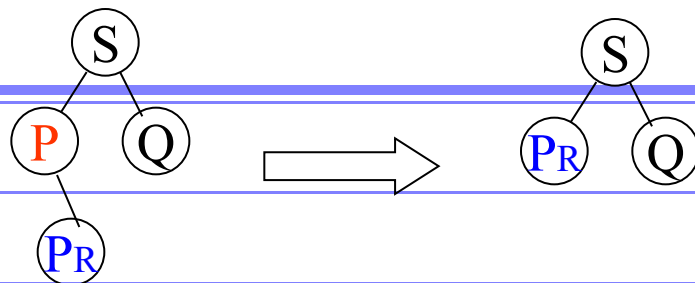
中序遍历: $P_L P S Q$ 中序遍历: $P_L S Q$

(1)

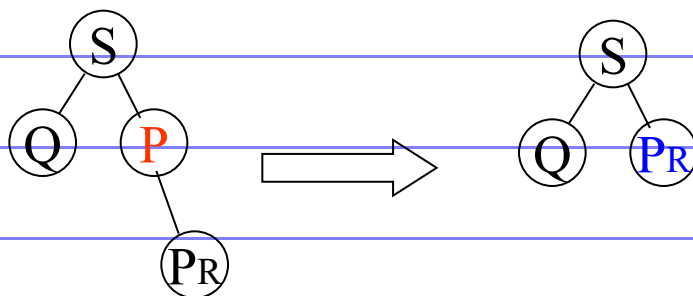


中序遍历: $Q S P_L P$ 中序遍历: $Q S P_L$

(2)

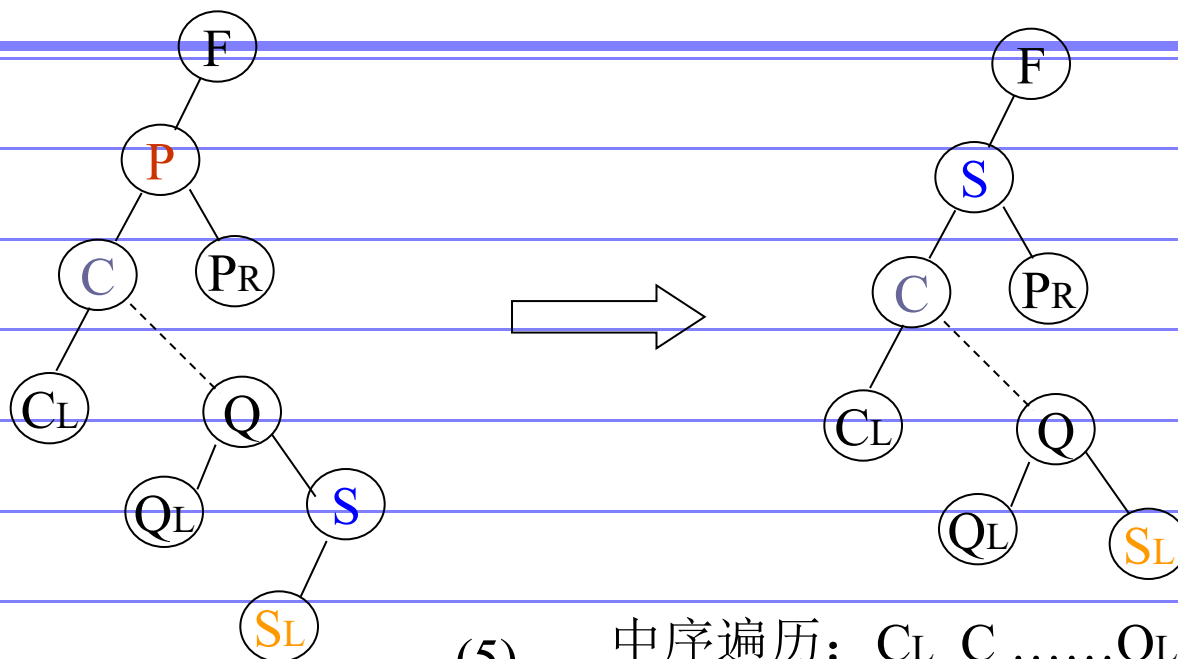


中序遍历: P P_R S Q 中序遍历: P_R S Q
(3)

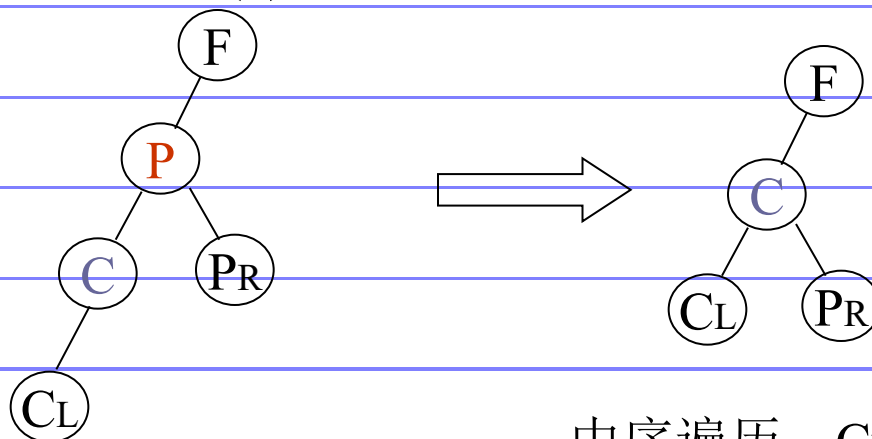


中序遍历: Q S P P_R 中序遍历: Q S P_R
(4)

中序遍历: $C_L \ C \ \dots \ Q_L \ Q \ S_L \ S \ P \ P_R \ F$



中序遍历: $C_L \ C \ \dots \ Q_L \ Q \ S_L \ S \ P_R \ F$

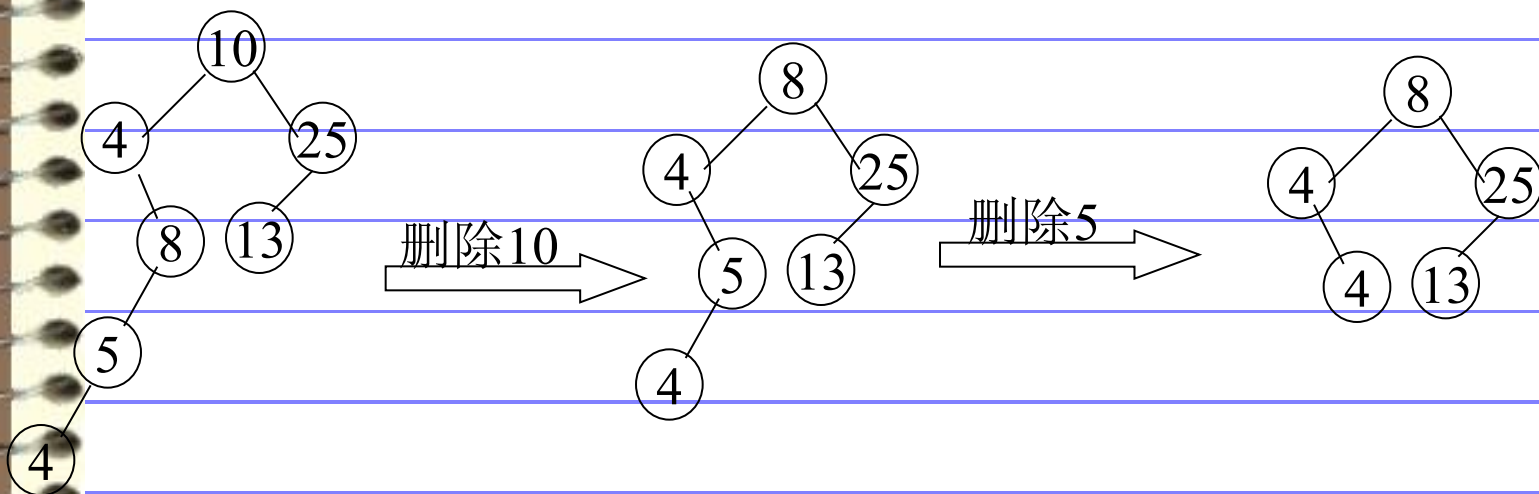
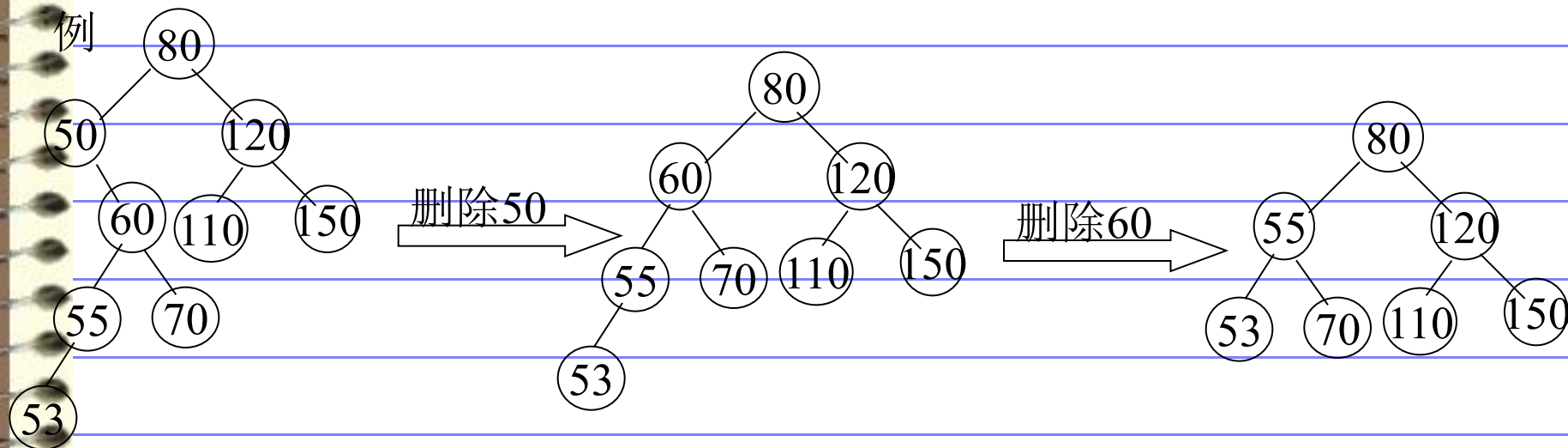


中序遍历: $C_L \ C \ P_R \ F$

中序遍历: $C_L \ C \ P \ P_R \ F$ (6)

✧ 删除算法

例



习题六

(1) 假设在树中，结点 x 是结点 y 的双亲时，用 (x,y) 来表示树边。已知一棵树的树边集合为 $\{ (e,i), (b,e), (b,d), (a,b), (g,j), (c,g), (c,f), (h,l), (c,h), (a,c) \}$ ，用树型表示法表示该树，并回答下列问题：

① 哪个是根结点？哪些是叶子结点？哪个是 g 的双亲？哪些是 g 的祖先？哪些是 g 的孩子？哪些是 e 的子孙？哪些是 e 的兄弟？哪些是 f 的兄弟？

② b 和 n 的层次各是多少？树的深度是多少？以结点 c 为根的子树的深度是多少？

(2) 一棵深度为 h 的满 k 叉树有如下性质：第 h 层上的结点都是叶子结点，其余各层上每个结点都有 k 棵非空子树。如果按层次顺序(同层自左至右)从1开始对全部结点编号，问：

① 各层的结点数是多少？

② 编号为 i 的结点的双亲结点(若存在)的编号是多少？

③ 编号为 i 的结点的第 j 个孩子结点(若存在)的编号是多少？

④ 编号为 i 的结点的有右兄弟的条件是什么？其右兄弟的编号是多少？

(3) 设有如图6-27所示的二叉树。

① 分别用顺序存储方法和链接存储方法画出该二叉树的存储结构。

② 写出该二叉树的先序、中序、后序遍历序列。

(4) 已知一棵二叉树的先序遍历序列和中序遍历序列分别为ABDGHCEFI和GDHBAECIF，请画出这棵二叉树，然后给出该树的后序遍历序列。

(5) 设一棵二叉树的中序遍历序列和后序遍历序列分别为BDCEAFHG和DECBGHFA，请画出这棵二叉树，然后给出该树的先序序列。

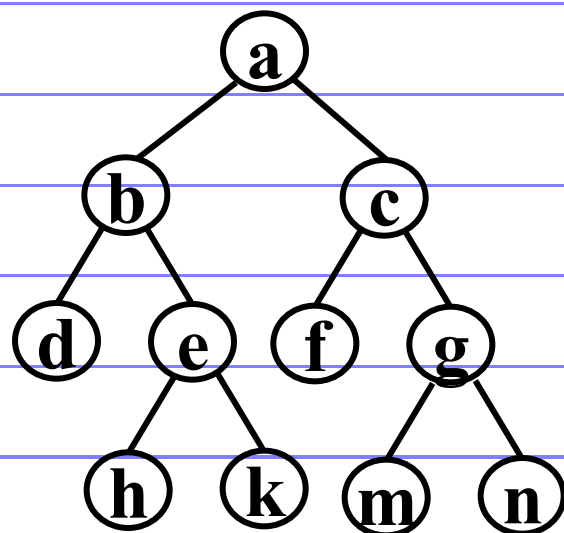


图6-27 二叉树

- (6) 已知一棵二叉树的中序遍历序列和后序遍历序列分别为dgbaekchif和gdbkeihfca，请画出这棵二叉树对应的中序线索树和后序线索树。
- (7) 以二叉链表为存储结构，请分别写出求二叉树的结点总数及叶子结点总数的算法。
- (8) 设图6-27所示的二叉树是森林F所对应的二叉树，请画出森林F。
- (9) 设有一棵树，如图6-28所示。
- ① 请分别用双亲表示法、孩子表示法、孩子兄弟表示法给出该树的存储结构。
 - ② 请给出该树的先序遍历序列和后序遍历序列。
 - ③ 请将这棵树转换成二叉树。

(10) 设给定权值集合 $w=\{3,5,7,8,11,12\}$ ，请构造关于 w 的一棵huffman树，并求其加权路径长度WPL。

(11) 假设用于通信的电文是由字符集 $\{a, b, c, d, e, f, g, h\}$ 中的字符构成，这8个字符在电文中出现的概率分别为 $\{0.07, 0.19, 0.02, 0.06, 0.32, 0.03, 0.21, 0.10\}$ 。

① 请画出对应的huffman树(按左子树根结点的权小于等于右子树根结点的权的次序构造)。

② 求出每个字符的huffman编码。

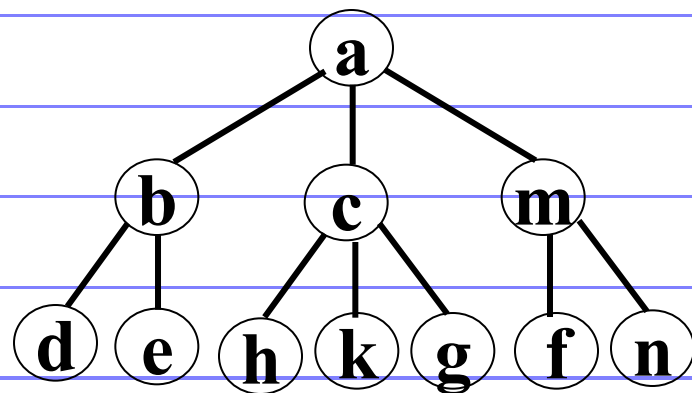


图6-28 一般的树

数据结构题集（C语言版）：

6. 23, 6. 27, 6. 28, 6. 37, 6. 43, 6. 44